Теория кодирования

Конспект курса

По материалам лекций

Содержание

1 Обозначения и соглашения ... 5
1.1 Коды и их параметры ... 5
1.2 Вероятность и информация ... 5
1.3 Каналы ... 5
1.4 Алгебра ... 5
1.5 Прочее ... 6
2 Системы передачи информации и основы теории кодирования ... 6
2.1 Системы передачи данных ... 6
2.2 Модуляция и модели канала ... 6
2.3 Решающие области и вероятность ошибки ... 7
2.4 Вероятность ошибки 2-АМ в канале с АБГШ ... 7
2.5 Отношение сигнал/шум ... 8
2.6 Способы снижения вероятности ошибки: посимвольное повторение ... 8
2.7 Теоремы кодирования ... 8
2.8 Пропускная способность некоторых каналов ... 9
2.9 Энергетический выигрыш кодирования ... 9
2.10 Жесткое и мягкое декодирование ... 9
2.11 Спектральная эффективность ... 9
2.12 Основные понятия теории кодирования ... 10
2.13 Линейные коды ... 11
2.14 Декодирование ... 13
2.15 Декодирование по информационным совокупностям ... 14
2.16 Дуальные коды и весовой спектр ... 15
2.17 Границы на параметры кодов ... 16
2.18 Заключение ... 18
3 Универсальные методы декодирования линейных кодов ... 18
3.1 Критерии мягкого декодирования ... 18
3.2 Метод порядковых статистик ... 19
3.3 Декодирование по решеткам ... 19
3.4 Декодирование с мягким выходом ... 22
4 Отказоустойчивое хранение данных: локально восстанавливаемые коды ... 24
4.1 Мотивация ... 24
4.2 Локальность ... 24
4.3 Минимальное расстояние в терминах столбцов ... 24
4.4 Граница Гопалана—Хуанга—Симитци—Еханина ... 25
4.5 Пирамидальные коды ... 26
4.6 Коды Тамо—Барга ... 27
4.7 Выводы ... 28
5 Сверточные коды ... 28
5.1 Конструкция кодов ... 28
5.2 Декодирование ... 30
5.3 Вероятность ошибки ... 30
5.4 Выводы ... 32
6 Методы модификации и комбинирования кодов ... 32
6.1 Конструкция Плоткина ... 32
6.2 Выкалывание, укорочение и расширение ... 33
6.3 Каскадные коды ... 34
6.4 Турбо-коды ... 36
6.5 Выводы ... 38
7 Полярные коды ... 38

7.1 Модели каналов ... 38
7.2 Параметр Бхаттачарьи ... 38
7.3 Пропускная способность ... 39
7.4 Поляризация канала ... 39
7.5 Полярный код и алгоритм последовательного исключения ... 40
7.6 Построение полярных кодов ... 41
7.7 Минимальное расстояние ... 42
7.8 Субоптимальность алгоритма ПИ ... 42
7.9 Списочное декодирование ... 43
7.10 Полярные коды с CRC ... 44
7.11 Полярные подкоды ... 44
7.12 Последовательное декодирование ... 46
7.13 Выводы ... 47
8 Циклические коды ... 47
8.1 Алгебраические сведения ... 47
8.2 Циклические коды ... 51
8.3 Коды БЧХ и Рида—Соломона ... 53
8.4 Декодирование кодов БЧХ ... 54
8.5 Приложения ... 59
8.6 Выводы ... 59
9 Альтернантные коды и криптосистема Мак-Элиса ... 60
9.1 Обобщенные коды Рида—Соломона ... 60
9.2 Альтернантные коды ... 61
9.3 Коды Гоппы ... 63
9.4 Криптосистема Мак-Элиса ... 64
9.5 Выводы ... 65
10 Коды с малой плотностью проверок на четность (МППЧ, LDPC) ... 66
10.1 Понятие МППЧ ... 66
10.2 Регулярные и нерегулярные коды ... 66
10.3 Декодирование ... 67
10.4 Эволюция плотностей ... 70
10.5 Конструкции кодов ... 72
10.6 Выводы ... 73
11 Коды для стирающего канала ... 73
11.1 Постановка задачи ... 73
11.2 LT-коды ... 74
11.3 Хищные коды (Raptor) ... 77
11.4 Выводы ... 77
12 Кодовая модуляция ... 77
12.1 Теоретико-информационные основы ... 77
12.2 Методы модуляции ... 78
12.3 Битоперемеженная кодовая модуляция ... 78
12.4 Многоуровневые коды ... 79
12.5 Выводы ... 80
13 Сетевое кодирование ... 81
13.1 Постановка задачи ... 81
13.2 Теоретические пределы ... 81
13.3 Линейные сетевые коды ... 82
13.4 Выводы ... 83
14 Регенерирующие коды ... 83
14.1 Мотивация ... 83

14.2 Сетевое кодирование и СХД 83
14.3 Комбинирующая конструкция (piggybacking) 84
14.4 Конструкция на основе произведения матриц 85
14.5 Восстановление в кодах Рида—Соломона 87
14.6 Выводы 90

1 Обозначения и соглашения

В конспекте используются следующие обозначения. Все прочие символы вводятся словами в тексте при первом использовании; если формула содержит величину, определенную ранее, это оговаривается явно.

1.1 Коды и их параметры

- **(n, k, d)-код** — блоковый код длины n (число символов в кодовом слове), размерности k (число информационных символов, из которых строится кодовое слово) с минимальным расстоянием d (минимальное расстояние Хемминга между различными кодовыми словами, то есть минимальное число позиций, в которых они различаются).
- **Скорость кода** $R = k/n$ — доля информационных символов в кодовом слове.
- C — код (множество кодовых слов); c — кодовое слово; $a = (a_0, \ldots, a_{k-1})$ — информационный вектор (данные до кодирования); $y = (y_0, \ldots, y_{n-1})$ — принятый вектор (кодовое слово с ошибками канала); e — вектор ошибок, так что $y = c + e$.
- $\mathrm{wt}(c)$ — вес Хемминга вектора c (число ненулевых позиций); $d(x, y)$ — расстояние Хемминга между векторами x и y.
- $\mathrm{GF}(q)$ — конечное поле из q элементов; для двоичных кодов $q = 2$. $\mathbb{Z}_N$ — кольцо вычетов по модулю N.
- G — порождающая матрица кода (строки линейно комбинируются в кодовые слова: $c = aG$); H — проверочная матрица (ее столбцы задают проверки: $Hc^T = 0$ для любого кодового слова).
- МДР-код — код с максимальным достижимым расстоянием $d = n - k + 1$.

1.2 Вероятность и информация

- $P\{\text{событие}\}$ — вероятность события; $p_{Y|X}(y \mid x)$ — условная вероятность (или плотность) того, что случайная величина Y примет значение y, если случайная величина X равна x; X, Y — заглавные буквы для случайных величин, строчные — для их конкретных значений.
- $\hat{x}$ — оценка величины x (например, результат решения демодулятора или декодера).
- $\sim N(\mu, \sigma^2)$ — «распределено нормально со средним μ и дисперсией σ^2»; σ — среднеквадратическое отклонение шума.
- $Q(x)$ — гауссовский хвост: $Q(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\,\mathrm{d}t$ — вероятность того, что нормальная величина с нулевым средним и единичной дисперсией превысит x. $\mathrm{erfc}(x)$ — дополнительная функция ошибок, $\mathrm{erfc}(x) = 2Q\bigl(\sqrt{2}x\bigr)$.
- $h(p)$ — двоичная функция энтропии: $h(p) = -p\log_2 p - (1-p)\log_2(1-p)$.
- $M[X]$ (или $M(X)$) — математическое ожидание случайной величины X; $D[X]$ — ее дисперсия.
- Логарифм без указания основания берется по основанию 2, если из контекста не следует иное.

1.3 Каналы

- **ДСК** — двоичный симметричный канал: каждый бит переворачивается с вероятностью p независимо от остальных.
- **БСЭ** — binary erasure channel, канал со стиранием: символ либо доходит верно, либо стирается (с вероятностью p) с известной приемнику пометкой.
- **АБГШ** — канал с аддитивным белым гауссовским шумом: $y_i = x_i + \eta_i$, где $\eta_i \sim N(0, \sigma^2)$ — независимые гауссовские шумовые компоненты.
- E_b, E_s — энергии на бит и на символ; N_0 — спектральная плотность мощности шума ($\sigma^2 = N_0/2$); ОСШ — отношение сигнал/шум, E_b/N_0 или E_s/N_0, измеряется в децибелах (дБ): $10\log_{10}(E_b/N_0)$.

1.4 Алгебра

- $\deg f$ — степень многочлена f; $f \equiv g \bmod m$ — делимость разности $f - g$ на m с остатком, равным нулю.
- НОД, НОК — наибольший общий делитель и наименьшее общее кратное.
- $\mathrm{rank}(M)$ — ранг матрицы M; матрицы обозначаются заглавными буквами, векторы — строчными.

- $A \otimes B$ — произведение Кронекера матриц A и B (блочная матрица из блоков $a_{ij}B$); $A^{\otimes m}$ — m-кратное кронекеровское возведение в степень. Исключение: в главе об LDPC-кодах символ $\otimes$, примененный к плотностям распределений, означает **свертку** (плотность суммы независимых величин).
- 1_A — индикатор события A: равен 1, если событие выполнено, и 0 иначе.
- $|S|$ — мощность конечного множества S; $\subset$ — строгое включение множеств.

1.5 Прочее

- $O(f(n))$ — стандартная асимптотическая оценка сложности алгоритмов.
- ЛОПП — логарифмическое отношение правдоподобия: $L = \ln \frac{P\{x=0 \mid y\}}{P\{x=1 \mid y\}}$; положительное значение говорит в пользу нуля, отрицательное — в пользу единицы.
- FER — частота ошибок по кадрам (вероятность неверно декодированного кодового слова); BER — частота ошибок по битам.
- В proof-блоках ($\Longrightarrow$) и ($\Longleftarrow$) отмечают доказательство прямой и обратной импликации соответственно.

2 Системы передачи информации и основы теории кодирования

2.1 Системы передачи данных

Система передачи данных (СПД) состоит из источника, кодера источника, кодера канала, модулятора, канала, демодулятора, декодера канала и декодера источника.

- **Кодер источника** устраняет избыточность из передаваемых данных (с потерями или без них).
- **Кодер канала** вносит избыточность, позволяющую впоследствии исправлять ошибки: он отображает последовательности информационных символов в более длинные кодовые последовательности.

Каналом ПД называется часть СПД, не контролируемая разработчиком; она моделируется как стохастический преобразователь. При проектировании системы некоторая совокупность ее блоков также может рассматриваться как канал.

Каждый блок СПД (кодер, модулятор, сам канал и т.д.) преобразует проходящие через него данные. Удобная формальная модель такого блока — **оператор канала $(i+1)$-го уровня**:

$$C_{i+1} = R_i C_i T_i,$$

где C_i — матрица данных на входе блока (строки — отдельные последовательности символов, например кадры или кодовые слова; столбцы — позиции внутри последовательностей), C_{i+1} — матрица данных на выходе, R_i — матрица преобразования строк (умножение слева смешивает и комбинирует последовательности), T_i — матрица преобразования столбцов (умножение справа преобразует символы внутри каждой последовательности). Индекс i нумерует последовательные блоки системы: выход C_{i+1} одного блока служит входом следующего. Это схематическая модель каскада блоков; конкретный вид матриц R_i, T_i (или замена их стохастическим преобразованием для неконтролируемой части — канала) определяется устройством каждого блока. Далее по курсу такая матричная запись не используется — она нужна лишь чтобы подчеркнуть: вся СПД есть цепочка преобразований данных.

Теория помехоустойчивого кодирования изучает методы внесения избыточности в передаваемые данные, а также методы ее использования для исправления ошибок.

2.2 Модуляция и модели канала

Передаваемый сигнал есть сумма сдвинутых во времени сигнальных функций:

$$x(t) = \sum_i S_i(t - iT),$$

где $S_i(t)$ — сигнальная функция, соответствующая передаваемому символу x_i (например, импульс нужной амплитуды и частоты), T — продолжительность символьного интервала, а суммирование ведется по всем символьным интервалам.

М-ичная амплитудно-импульсная модуляция (М-АИМ):

$$S_i(t) = \alpha(2i + 1 - M)g(t)\sin(2\pi f t),$$

где $g(t)$ — сигнальный импульс (например, единичный импульс продолжительностью T), f — несущая частота, α — коэффициент, определяющий энергию передаваемого сигнала.

Модель канала в непрерывном времени: $y(t) = x(t) + \eta(t)$. Модель в дискретном времени (канал с аддитивным белым гауссовским шумом, АБГШ):

$$y_i = \alpha(2x_i + 1 - M) + \eta_i, \quad \eta_i \sim N(0, \sigma^2),$$

где передаваемый символ $x_i \in \{0, 1, ..., M-1\}$ выбирает уровень $\alpha(2x_i + 1 - M)$ из сигнального множества, а η_i — гауссовская случайная величина (шум) с нулевым средним и дисперсией σ^2.

2.3 Решающие области и вероятность ошибки

Приемник наблюдает на выходе канала вектор $y = (y_0, ..., y_{n-1})$. Канал характеризуется условным распределением $p_{Y|X}(y|x)$, где X, Y — случайные величины, соответствующие векторам переданных и принятых символов (если выход канала непрерывен, $p_{Y|X}$ — условная плотность распределения).

Приемник реализует разбиение векторного пространства выходов $\mathbb{R}^N$ на **решающие области** $\mathcal{R}_x$: если принятый вектор y попал в область $\mathcal{R}_x$, приемник выносит решение $\hat{x} = x$. Вероятность ошибки:

$$P_e = \int_{\mathbb{R}^N} P_e(y) p_Y(y)\,\mathrm{d}y = \sum_x \int_{\mathcal{R}_x} \big(1 - p_{X|Y}\{x|y\}\big) p_Y(y)\,\mathrm{d}y = 1 - \sum_x \int_{\mathcal{R}_x} p_{X|Y}\{x|y\} p_Y(y)\,\mathrm{d}y,$$

где $P_e(y)$ — вероятность ошибки при условии, что принят именно вектор y; $p_Y(y)$ — плотность распределения (или вероятности) принятого вектора; $p_{X|Y}\{x|y\}$ — апостериорная вероятность того, что был передан вектор x, при условии принятия y. Первое равенство — усреднение $P_{e(y)}$ по всем возможным принятым векторам; во втором для $y \in \mathcal{R}_x$ ошибкой является любое решение, кроме x.

Критерий максимума апостериорной вероятности (критерий идеального наблюдателя) — выбор областей

$$\mathcal{R}_x = \big\{y \mid p_{X|Y}(x|y) > p_{X|Y}(x'|y), x' \neq x\big\} = \big\{y \mid P_X(x) p_{Y|X}(y|x) > P_X(x') p_{Y|X}(y|x'), x' \neq x\big\},$$

где $P_X(x)$ — априорная вероятность передачи вектора x (второе равенство — формула Байеса). Он обеспечивает минимизацию вероятности ошибки. **Критерий максимума правдоподобия** (МП) использует области

$$\mathcal{R}_x = \big\{y \mid p_{Y|X}(y|x) > p_{Y|X}(y|x'), x' \neq x\big\}.$$

Он совпадает с критерием идеального наблюдателя, если все сообщения равновероятны: $P_X(x) = \text{const}$.

2.4 Вероятность ошибки 2-АМ в канале с АБГШ

Пусть $y_i = \alpha(2x_i - 1) + \eta_i$, $\eta_i \sim N(0, \sigma^2)$, $x_i \in \{0, 1\}$. Тогда

$$p_{Y|X}(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \alpha(2x-1))^2}{2\sigma^2}\right).$$

Считая сообщения равновероятными, получаем приемник максимального правдоподобия с решающими областями $\mathcal{R}_0 = \{y \mid y < 0\}$, $\mathcal{R}_1 = \{y \mid y \geq 0\}$ (множества значений y, при которых решением объявляется 0 или 1). Вероятность ошибки:

$$
\begin{aligned}
P_e &= P_X(0)P\{Y \geq 0 \mid X = 0\} + P_X(1)P\{Y < 0 \mid X = 1\} \\
&= \int_0^\infty \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y+\alpha)^2}{2\sigma^2}\right) \mathrm{d}y = \int_{\frac{\alpha}{\sigma}}^\infty \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right) \mathrm{d}y \\
&= Q(\alpha/\sigma) = \frac{1}{2}\operatorname{erfc}\left(\frac{\alpha}{\sqrt{2}\sigma}\right).
\end{aligned}
$$

Здесь $Q(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{t^2}{2}\right) \mathrm{d}t$ — функция Q, а $\operatorname{erfc}(x) = 2Q\left(\sqrt{2}x\right)$.

2.5 Отношение сигнал/шум

Значение сигнала соответствует уровню напряжения; мощность сигнала пропорциональна $E_s = \alpha^2$, мощность шума пропорциональна $\sigma^2 = N_0/2$. Используются величины:

- **ОСШ на символ**: E_s/N_0 (измеряется в децибелах);
- **ОСШ на бит**: $E_b/N_0 = E_s/(RN_0)$, где R — количество бит полезной информации, представляемых одним символом.

Для 2-АМ: $P_e = Q\left(\sqrt{2E_s/N_0}\right)$. ОСШ измеряется в децибелах:

$$\text{ОСШ} = 10\log_{10}(E_b/N_0).$$

2.6 Способы снижения вероятности ошибки: посимвольное повторение

Пусть каждый символ повторяется m раз: $y_{mi+j} = \alpha(2x_i - 1) + \eta_{mi+j}$, $0 \leq j < m$.

Голосование по отдельным решениям. Принимаем решения по каждому y_{mi+j} и проводим голосование. Вероятность ошибки:

$$P_v(m) = \sum_{j=\lceil \frac{m}{2} \rceil}^{m} \binom{m}{j} P_e^j (1 - P_e)^{m-j}.$$

Усреднение наблюдений. Принимаем решение по сумме $\sum_{j=0}^{m-1} y_{mi+j} = m\alpha(2x_i - 1) + \sum_{j=0}^{m-1} \eta_{mi+j}$. Вероятность ошибки:

$$P_a(m) = Q\left(\frac{m\alpha}{\sqrt{m}\sigma}\right) = Q\left(\sqrt{\frac{2mE_s}{N_0}}\right) = Q\left(\sqrt{\frac{2E_b}{N_0}}\right).$$

Поскольку при повторении на каждый повтор приходится энергия E_b/m, суммарная энергия на бит не меняется: **выигрыша нет**. Отсюда вывод: избыточность нужно вносить не на уровне отдельных битов, а на уровне их блоков — этим занимается теория кодов, исправляющих ошибки.

2.7 Теоремы кодирования

Пусть для передачи используется код $C \not\subset X^n$ длины n — множество из $M = |C|$ кодовых слов, каждое из которых есть вектор длины n над алфавитом X, — выбираемых с одинаковой вероятностью. Внимание: буквой C обозначается и код, и пропускная способность канала (в теоремах ниже); различаем по контексту. Скорость кода — $R = \log_{|X|} \frac{M}{n}$ бит (символов) на позицию кодового слова.

> **Теорема 1** (обратная). Для дискретного постоянного канала с пропускной способностью C для любого $\delta > 0$ существует $\varepsilon > 0$ такое, что для любого кода со скоростью $R > C + \delta$ средняя вероятность ошибки $\overline{P_e} \geq \varepsilon$.

> **Теорема 2** (прямая). Для дискретного постоянного канала с пропускной способностью C для любых $\varepsilon, \delta > 0$ существует достаточно большое $n_0 > 0$ такое, что для всех натуральных $n \geq n_0$ существует код длины n со скоростью $R \geq C - \delta$, средняя вероятность ошибки которого $P_e \leq \varepsilon$.

2.8 Пропускная способность некоторых каналов

Двоичный симметричный канал (ДСК): $X, Y \in \{0,1\}$, $p_{Y|X}(y|x) = \begin{cases} p & y \neq x \\ 1-p & y = x \end{cases}$. Пропускная способность:

$$C_{\text{ДСК}} = 1 + p\log_2 p + (1-p)\log_2(1-p).$$

Идеальный частотно ограниченный гауссовский канал: $y(t) = x(t) + \eta(t)$, где $\eta(t)$ — гауссовский случайный процесс со спектральной плотностью мощности $S(f) = \frac{N_0}{2}$ при $|f| < W$ и 0 иначе:

$$C_{\text{АБГШ}} = W\log_2\left(1 + \frac{E_s}{WN_0}\right), \quad \lim_{W\to\infty} C_{\text{АБГШ}} = \frac{E_s}{N_0 \ln 2}.$$

Код со скоростью R обеспечивает надежную передачу при $R < C = \frac{E_s}{N_0 \ln 2} = E_b \frac{R}{N_0 \ln 2}$, откуда минимальное допустимое ОСШ на бит:

$$\frac{E_b}{N_0} > \ln 2 \sim 0.693 \quad (\text{то есть} \quad -1.59 \text{ дБ})$$

— так называемый **предел Шеннона**.

2.9 Энергетический выигрыш кодирования

Энергетический выигрыш кодирования показывает, во сколько раз использование кодирования позволяет уменьшить необходимое ОСШ для достижения заданной вероятности ошибки по сравнению со случаем отсутствия кодирования. Реальный выигрыш снижается из-за:

- использования кодов конечной длины;
- несовершенства существующих кодовых конструкций;
- субоптимальных алгоритмов декодирования;
- дискретизации выхода канала (жесткое декодирование).

2.10 Жесткое и мягкое декодирование

Канал с АБГШ: $y_i = (2x_i - 1) + \eta_i$, $x_i \in \{0,1\}$.

- **Мягкое декодирование**: декодер непосредственно использует величины y_i.
- **Жесткое декодирование**: декодер использует оценки $\hat{x}_i$; канал с АБГШ превращается в ДСК с вероятностью ошибки $p = Q\left(\sqrt{2\frac{E_s}{N_0}}\right)$.

При жестком декодировании пропускная способность эквивалентного ДСК вблизи $p = \frac{1}{2}$ аппроксимируется так. Из разложения двоичной энтропии $h_2(p) = 1 - \left(\frac{2}{\ln}2\right)\left(\frac{1}{2} - p\right)^2 + o\left(\left(\frac{1}{2} - p\right)^2\right)$ следует:

$$C = 1 - h_2(p) \sim \left(\frac{2}{\ln}2\right)\left(\frac{1}{2} - p\right)^2.$$

При малых x имеем $Q(x) \sim \frac{1}{2} - \frac{x}{\sqrt{2\pi}}$, поэтому $\frac{1}{2} - p \sim \sqrt{\frac{E_s}{\pi N_0}}$ и достижимая скорость удовлетворяет

$$R < \ln 2\left(\frac{2E_s}{\pi N_0}\right) = \ln 2 \frac{2RE_b}{\pi N_0} \Longrightarrow \frac{E_b}{N_0} = \frac{E_s}{RN_0} > \frac{\pi \ln 2}{2} \approx 1.09 = 0.37 \text{ дБ}.$$

Потеря по сравнению с мягким декодированием (-1.59 дБ) составляет **1.96 дБ**.

2.11 Спектральная эффективность

Для идеального частотно ограниченного гауссовского канала полоса частот, занимаемых сигналом, зависит от длительности символьного интервала T, которая определяется в том числе скоростью передачи данных. **Спектральная эффективность** кодирования:

$$\beta = \frac{R}{W} \quad \left[\frac{\text{бит}}{\text{сГц}}\right].$$

Из $\frac{R}{W} < \log_2\left(1 + \frac{R}{W} \cdot \frac{E_b}{N_0}\right)$ следует предел на ОСШ на бит при заданной спектральной эффективности:

$$\frac{E_b}{N_0} > \frac{2^\beta - 1}{\beta}.$$

2.12 Основные понятия теории кодирования

> **Определение 1**. Код — множество допустимых последовательностей символов алфавита X (конечных или бесконечных; на практике ограничиваются последовательностями конечной длины n). Не всякая последовательность символов из X является кодовой.

Кодер — устройство, реализующее отображение информационных последовательностей символов алфавита B в кодовые; различным информационным последовательностям сопоставляются различные кодовые последовательности. **Декодер** — устройство, восстанавливающее по принятой последовательности наиболее вероятную кодовую (или информационную) последовательность; для этого необходима модель канала.

Скорость кода — отношение длин информационной и кодовой последовательностей.

2.12.1 Критерии декодирования

- **Критерий минимального расстояния** ($X = Y$): декодер ищет $c = \arg\min_{c \in C} d(c, y)$.
- **Списочное декодирование**: декодер ищет все кодовые слова в сфере заданного радиуса вокруг принятого вектора.
- **Побитовое декодирование**: критерий идеального наблюдателя для отдельных символов кодового слова; часто дополняется вычислением логарифмических отношений правдоподобия для отдельных символов:

$$L_i = \ln \frac{\sum_{c \in C, c_i = 0} P\{c|y\}}{\sum_{c \in C, c_i = 1} P\{c|y\}}.$$

Алгоритм называется **алгоритмом полного декодирования** по критерию K, если он способен найти решение соответствующей оптимизационной задачи для любого возможного принятого сигнала; здесь K — правило выбора решения (например, минимум расстояния Хемминга или максимум правдоподобия).

2.12.2 Метрики

> **Определение 2**. Функция $d(x, y)$ называется метрикой, если: $d(x, y) \geq 0$ и $d(x, y) = 0 \Leftrightarrow x = y$; $d(x, y) = d(y, x)$; $d(x, y) + d(y, z) \geq d(x, z)$ (неравенство треугольника). Метрическое пространство — множество X с определенной на нем метрикой.

Примеры метрик:

- **Расстояние Хемминга**: $d_H(x, y) = |\{i \mid x_i \neq y_i\}|$.
- **Расстояние Евклида**: $d_E(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$ — для аддитивного гауссовского канала с АИМ ($Y = \mathbb{R}^n$).
- **Расстояние Ли**: $d_L(x, y) = \sum_{i=1}^n \min(|x_i - y_i|, q - |x_i - y_i|)$ на $A = \mathrm{GF}(q)^n$ — для аддитивного гауссовского канала с q-ичной фазовой модуляцией.
- **Ранговое расстояние**: $d_R(x, y) = \mathrm{rank}(x - y)$ на $A = \mathrm{GF}(q)^{n \times m}$ — для сетевых кодов.

Для ДСК с $p < 0.5$ декодирование по максимуму правдоподобия сводится к минимуму расстояния Хемминга:

$$\hat{c} = \arg\max_{c \in C} \prod_{i=1}^{n} P\{y_i \mid c_i\} = \arg\max_{c \in C} \prod_{i=1}^{n} (1-p)^{1-|y_i - c_i|} p^{|y_i - c_i|} = \arg\max_{c \in C} (1-p)^n \prod_{i=1}^{n} \left(\frac{p}{1-p}\right)^{|y_i - c_i|}$$

$$= \arg\min_{c \in C} \sum_{i=1}^{n} |y_i - c_i| = \arg\min_{c \in C} d_H(c, y).$$

2.12.3 Блоковые и сверточные коды

Операция канального кодирования вносит в информационную последовательность избыточность, необходимую для исправления возможных ошибок (forward error correction). Обычно входной и выходной алфавиты кодера совпадают.

- **Блоковые коды** преобразуют блок из k символов в блок из n символов; преобразование отдельных блоков выполняется независимо.
- **Сверточные коды** преобразуют блок из k символов в блок из n символов, но преобразование зависит от предыдущих блоков. В большинстве случаев блоками являются один или несколько битов.

2.12.4 Параметры блоковых кодов

- n — длина кода C (для исправления ошибок не все $|X|^n$ последовательностей должны быть кодовыми словами);
- мощность кода (число различных кодовых слов) $M = |C|$;
- скорость кода $R = \log_{|} X| \ M/n$;
- **минимальное расстояние** — минимальное расстояние Хемминга между различными кодовыми словами.

> **Пример 1**. Для $C = \{000, 111\}$ имеем $d_{\min}(C) = 3$.

Хеммингов шар радиуса $d_{\min} - 1$ вокруг кодового слова $c \in C$ не содержит других кодовых слов. Хемминговы шары радиуса $\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor$ вокруг кодовых слов не пересекаются. Если в результате канальных искажений слово c превратилось в точку своего шара, оно может быть восстановлено декодером по критерию минимального расстояния Хемминга.

> **Итог.** Код с минимальным расстоянием d способен исправить $\left\lfloor \frac{d-1}{2} \right\rfloor$ ошибок.

2.13 Линейные коды

> **Определение 3**. Линейным $(n, k)_q$ кодом C называется k-мерное линейное подпространство n-мерного линейного пространства над полем $\mathrm{GF}(q)$. Число кодовых слов равно q^k; k — размерность кода, $\frac{k}{n}$ — скорость кода.

Код задается **порождающей** матрицей полного ранга G размера $k \times n$ и **проверочной** матрицей H размера $r \times n$:

$$C = \{y = xG \mid x \in \mathrm{GF}(q)^k\}, \quad C = \{y \in \mathrm{GF}(q)^n \mid yH^T = 0\}, \quad r \geq n - k = \mathrm{rank}(H), \quad GH^T = 0.$$

Иногда проверочная матрица определяется над расширением поля $\mathrm{GF}(q^m)$. Код может иметь много различных порождающих и проверочных матриц.

Коды называются **эквивалентными**, если один получается из другого перестановкой координат (позиций) во всех кодовых словах и умножением отдельных координат на ненулевые константы поля; эквивалентные коды имеют одинаковые (n, k, d). С помощью перестановок столбцов и линейных операций над строками (не меняющих порождаемое пространство) порождающая матрица приводится к виду $G = (I \mid A)$, где I — единичная матрица размера $k \times k$, A — матрица размера $k \times (n-k)$. Тогда **систематическое кодирование** имеет вид

$$xG = (x \mid xA),$$

то есть информационный вектор является подвектором кодового слова, что упрощает декодирование. При этом $H = (A^T \mid -I)$.

2.13.1 Свойства линейных кодов

Теорема 3. Минимальное расстояние линейного блокового кода C равно минимальному весу ненулевого кодового слова:

$$d = \min_{c' \neq c'', c', c'' \in C} d(c', c'') = \min_{c \in C \setminus \{0\}} \mathrm{wt}(c),$$

где $\mathrm{wt}(c)$ — вес Хемминга (число ненулевых символов).

Теорема 4. Пусть H — проверочная матрица кода длины n с r строками. Код имеет размерность $n - r$ тогда и только тогда, когда существуют r линейно независимых столбцов матрицы H, а любые $r + 1$ столбцов линейно зависимы.

Теорема 5. Код имеет минимальное расстояние d тогда и только тогда, когда любые $1, 2, \ldots, d - 1$ столбцов H линейно независимы, но существуют d линейно зависимых столбцов. (Принадлежность коду $yH^T = 0$ эквивалентна записи линейной зависимости столбцов.)

Теорема 6. Граница Синглтона (верхняя): для любого (n, k, d) линейного кода $n - k \geq d - 1$. (Ранг матрицы H — максимальное число линейно независимых столбцов — не может превосходить $n - k$.) Для произвольных кодов: $A_q(n, d) \leq q^{n-d+1}$.

Коды с $n - k = d - 1$ называются **разделимыми кодами с максимальным достижимым расстоянием** (МДР).

2.13.2 Простейшие коды

- Пусть G — обратимая $n \times n$ матрица. Она порождает код $(n, n, 1)$.
- **Код с n-кратным повторением** $(n, 1, n)$: $G' = (1 \quad 1 \quad \ldots \quad 1)$; проверочная матрица H' размера $(n - 1) \times n$ имеет в i-й строке нуль на позиции i и единицы на остальных.
- **Код с проверкой на четность** $(n, n - 1, 2)$: $G'' = (I_{n-1} \mid 1)$ (к каждой строке единичной матрицы приписана единица), $H'' = (1 \quad 1 \quad \ldots \quad 1)$.

2.13.3 Код Хемминга

Выберем в качестве столбцов матрицы H все ненулевые двоичные векторы длины r. Получаем **код Хемминга**:

$$n = 2^r - 1, \quad k = n - r = 2^r - r - 1, \quad d = 3.$$

Код лежит на границе Хемминга, то есть является **совершенным**:

$$\sum_{i=0}^{1} \binom{n}{i} = n + 1 = 2^{n-k}.$$

Кодирование: информационные символы располагаются во всех позициях, кроме степеней двойки 2^i. Декодирование: синдром задает номер ошибочной позиции (если столбцы выписаны в соответствии с двоичной записью номеров).

Симплексный код — дуальный к коду Хемминга код $(2^m - 1, m, 2^{m-1})$: его порождающей матрицей служит проверочная матрица H кода Хемминга. Все его кодовые слова равноудалены друг от друга.

2.14 Декодирование

2.14.1 Жесткое и мягкое декодирование

Пусть используется код $C \not\subset X^n$. Модулятор преобразует символы из конечного алфавита X в сигналы; выходной алфавит большинства каналов — вещественные или комплексные числа.

- **Жесткое декодирование**: демодулятор на каждом символьном интервале находит наиболее правдоподобный символ из X; декодер пытается исправить ошибочные решения демодулятора.
- **Мягкое декодирование**: демодулятор преобразует принятый сигнал в форму, пригодную для декодера; окончательные решения не принимаются, либо формируется информация о надежности принятых решений (логарифмические отношения правдоподобия, мягкое детектирование). Декодер ищет кодовое слово, наиболее правдоподобное при условии сигналов, принятых на всех n символьных интервалах.

2.14.2 Синдромное декодирование и стандартная расстановка

Рассмотрим ДСК с переходной вероятностью $p < 0.5$. Принятый вектор $y = xG + e$, где e — вектор ошибки.

Декодирование перебором кодовых слов (выбор ближайшего) имеет сложность $O\left(2^k n\right)$. Более эффективно **синдромное декодирование**. Синдром принятого вектора:

$$S = yH^T = xGH^T + eH^T = eH^T$$

зависит только от вектора ошибки. Построим **стандартную расстановку**: рассмотрим все возможные векторы e и их синдромы; для каждого возможного значения синдрома отсортируем соответствующие векторы e по весу. В качестве решения выбираем самый легкий вектор e, соответствующий вычисленному синдрому (лидер смежного класса). Идея в том, что декодеру достаточно хранить таблицу «синдром $\to$ лидер смежного класса» размера $2^r = 2^{n-k}$ вместо перебора всех 2^k кодовых слов: приняв y, вычисляем S, находим $\hat{e}$ и полагаем $\hat{c} = y - \hat{e}$.

2.14.3 Декодирование при стираниях

Иногда известно, что отдельные символы кодового слова продетектированы ненадежно; такие символы заменяются на ε перед декодированием. Стираниями также считаются потери пакетов в компьютерных сетях. Декодеру не нужно искать местоположение этих потенциальных ошибок — достаточно найти их значения. Если нестертые символы образуют информационную совокупность и других ошибок не было, кодовое слово восстанавливается.

> **Теорема 7**. (n, k, d) код может исправить любую комбинацию из t ошибок и v стираний, если
>
> $$d \geq 2t + v + 1.$$
>
> **Доказательство.** Стирание эквивалентно выкалыванию кода на v позиций, после чего минимальное расстояние уменьшается не более чем на v: остается $d - v \geq 2t + 1$, что достаточно для исправления t ошибок. □

Для двоичных кодов декодирование ошибок и стираний можно выполнить за два прохода: положить все стертые позиции равными 0 и исправить ошибки; затем положить их равными 1 и повторить; выбрать результат декодирования, ближайший к принятому вектору. Иногда возможно построить алгоритм, одновременно исправляющий ошибки и стирания.

2.14.4 Весовой спектр и вероятности ошибки

Весовой спектр кода: $A_i = |\{c \in C \mid \mathrm{wt}(c) = i\}|$.

Для ДСК с переходной вероятностью p:

- вероятность **необнаружения** ошибки (синдром нулевой, но принятое слово отлично от переданного):

$$P_{[\text{undetect}]} = P\{S = 0\} = \sum_{i=d}^{n} A_i p^i (1-p)^{n-i} \le \sum_{i=d}^{n} \binom{n}{i} p^i (1-p)^{n-i};$$

- вероятность **правильного декодирования** — вероятность того, что вектор ошибки является лидером смежного класса:

$$P_{[\text{correct}]} = \sum_{i=0}^{l} L_i p^i (1-p)^{n-i},$$

где L_i — число лидеров смежных классов веса i, l — максимальный вес лидера. Для совершенных кодов $l = \left\lfloor \frac{d-1}{2} \right\rfloor$, $L_i = \binom{n}{i}$. В общем случае $P_{[\text{correct}]} \ge \sum_{i=0}^{l} \binom{n}{i} p^i (1-p)^{n-i}$.

2.14.5 Объединенная верхняя граница

Для аддитивного гауссовского канала с двоичной модуляцией $y_i = (2c_i - 1) + \eta_i$, $\eta_i \sim N(0, \sigma^2)$, вероятность ошибки мягкого декодирования по максимуму правдоподобия линейного блокового кода ограничена:

$$P \le \sum_{i=d}^{n} A_i Q\left(\sqrt{\frac{2iE_s}{N_0}}\right) = \sum_{i=d}^{n} A_i Q\left(\sqrt{2iR\frac{E_b}{N_0}}\right) = \frac{1}{n}\sum_{i=d}^{n} A_i \operatorname{erfc}\left(\sqrt{\frac{iRE_b}{N_0}}\right).$$

2.15 Декодирование по информационным совокупностям

Полное декодирование по минимальному расстоянию — нахождение по произвольному y ближайшего кодового слова $c = \arg\min_{c \in C} d(c, y)$.

> **Определение 4**. Информационной совокупностью (ИС) называется множество из k позиций кодового слова, значения на которых однозначно определяют значения символов на остальных позициях. Если $\gamma = \{j_1, \ldots, j_k\}$ — ИС, то дополнение $\{1, \ldots, n\} \setminus \gamma$ образует проверочную совокупность.

γ — ИС тогда и только тогда, когда матрица из столбцов $j_1, \ldots, j_k$ порождающей матрицы обратима. Пусть $G(\gamma) = M(\gamma)G$ — порождающая матрица с единичной подматрицей на столбцах γ, где $M(\gamma)$ — подходящая обратимая матрица. ИС **свободна от ошибок**, если $e(\gamma) = 0$ (соответствующие позиции вектора ошибки нулевые).

> **Алгоритм 1** (декодирование по ИС). Ищем $c = \arg\min_{c \in C} d(c, y)$ для $y = xG + e$.
>
> 1. Первоначальный кандидат: $c = 0$.
> 2. Выбрать ИС γ, вычислить $c' = y(\gamma)G(\gamma)$.
> 3. Если $d(c', y) < d(c, y)$, положить $c = c'$.
> 4. Перейти к следующей ИС. Если все ИС проверены, возвратить c.

Не всякие k позиций образуют ИС. Для коротких кодов можно заранее построить список ИС, достаточный для исправления всех конфигураций ошибок; для длинных кодов целесообразно перебирать ИС, добавляя и удаляя на каждом шаге по одному элементу (экономия на приведении порождающей матрицы к каноническому виду $G(\gamma)$).

> **Теорема 8**. Алгоритм декодирования по ИС обеспечивает полное декодирование по минимальному расстоянию.

Доказательство. Достаточно показать, что для всякого исправимого вектора ошибки существует ИС, свободная от ошибок. Пусть c — единственное решение задачи декодирования по минимальному расстоянию, $e = y - c$ — вектор ошибки, $E = \operatorname{supp}(e)$ — множество позиций ненулевых элементов e, $|E| \le n - k$, $N = \{1, \dots, n\}$. Если бы $N \setminus E$ не содержало ИС, то существовали бы различные кодовые слова, отличающиеся от принятого вектора только в позициях E, что противоречит единственности c. □

2.15.1 Покрытия

Определение 5. Покрытием $M(n, m, t)$ называется набор $F \not\subset 2^{N_n}$ из подмножеств мощности m множества $N_n = \{1, \dots, n\}$, такой, что всякое t-элементное подмножество N_n содержится в одном из $f \in F$. Здесь N_n — множество позиций кодового слова, а 2^{N_n} — множество всех его подмножеств (булеан).

Декодирование по ИС с исправлением не более t ошибок требует покрыть все исправимые конфигурации ошибок; элементы покрытия задают проверочные совокупности.

Пример 2. Для $(7, 4, 3)$-кода все конфигурации единичных ошибок покрываются проверочными совокупностями $M(7, 3, 1) = \{\{1, 2, 4\}, \{5, 6, 7\}, \{3, 4, 5\}\}$. Им соответствуют ИС $\{3, 5, 6, 7\}$, $\{1, 2, 3, 4\}$, $\{1, 2, 6, 7\}$ и преобразованные порождающие матрицы, каждая из которых имеет единичную подматрицу на соответствующей ИС. Декодирование принятого вектора $y = (0, 1, 1, 0, 1, 0, 0)$ дает кандидатов $(0, 1, 1, 1, 1, 0, 0)$, $(0, 1, 1, 0, 0, 1, 1)$, $(0, 1, 1, 1, 1, 0, 0)$; выбирается ближайший.

Построение минимального покрытия — NP-полная задача. Итеративное (приближенное) построение: на каждом шаге выбирается множество, содержащее максимальное количество непокрытых элементов.

2.16 Дуальные коды и весовой спектр

Определение 6. Пусть задан (n, k) код с проверочной матрицей H. Дуальным к нему называется $(n, n - k)$ код с порождающей матрицей H. Кодовые слова дуального кода — множество всех проверок на четность исходного кода. Самодуальным называется код, совпадающий со своим дуальным.

Код с проверочной матрицей $H = (A \mid I)$ самодуален тогда и только тогда, когда A — квадратная матрица, удовлетворяющая $AA^T = -I$ (для $\mathrm{GF}(2)$ — $AA^T = I$), поскольку $HH^T = AA^T + I$.

Весовой спектр линейного кода — последовательность A_i, $i = 0, \dots, n$, где A_i — число кодовых слов веса i. Он нужен для оценки вероятностей ошибки. **Весовая функция кода**:

$$W_C(x, y) = \sum_{i=0}^{n} A_i x^{n-i} y^i = \sum_{u \in C} x^{n - \mathrm{wt}(u)} y^{\mathrm{wt}(u)}.$$

Теорема 9. Мак-Вильямса (для двоичных линейных кодов). Весовая функция кода C связана с весовой функцией дуального кода $C^\perp$ соотношением

$$W_{C^\perp}(x, y) = \frac{1}{|C|} \cdot W_C(x + y, x - y).$$

Таким образом, весовой спектр дуального кода полностью определяется весовым спектром исходного.

2.17 Границы на параметры кодов

2.17.1 Граница Хемминга

Теорема 10. Для любого q-ичного кода с минимальным расстоянием $d = 2t + 1$ число кодовых слов удовлетворяет

$$A_q(n, d) \leq \frac{q^n}{\sum_{i=0}^{t} \binom{n}{i} (q-1)^i}.$$

Доказательство. Если код исправляет t ошибок, то вокруг всех кодовых слов можно описать непересекающиеся хемминговы шары радиуса t; каждый шар содержит $\sum_{i=0}^{t} \binom{n}{i} (q-1)^i$ векторов. □

При $n \to \infty$ скорость q-ичного кода удовлетворяет $R \leq 1 - h_q\left(\frac{d}{2n}\right)$, где

$$h_q(x) = -x \log_q x - (1-x) \log_q (1-x).$$

(Следует из аппроксимации Стирлинга $n! \approx \left(\frac{n}{e}\right)^n$ и оценки знаменателя старшим членом $\binom{n}{\lfloor \frac{d-1}{2} \rfloor} (q-1)^{\frac{d-1}{2}}$.)

2.17.2 Граница Варшамова—Гилберта

Теорема 11. Существует q-ичный код длины n с минимальным расстоянием d, число слов которого удовлетворяет

$$A_q(n, d) \geq \frac{q^n}{\sum_{i=0}^{d-1} \binom{n}{i} (q-1)^i}.$$

Доказательство. Если код C имеет максимальную мощность, то для любого вектора $x^{\neg} \in C$ существует кодовое слово c с $d(x, c) \leq d - 1$. Итеративное построение: $A := \mathrm{GF}(q)^n$; выбрать произвольный $c \in A$, добавить его в код; удалить из A шар радиуса $d-1$ с центром в c (число элементов в шаре $\sum_{i=0}^{d-1} \binom{n}{i} (q-1)^i$; фактически удаляется меньше, так как часть шара могла быть удалена ранее); повторять, пока A непусто. □

Теорема 12. Варшамов—Гилберт для линейных кодов. Если выполнено $q^r > \sum_{i=0}^{d-2} \binom{n-1}{i} (q-1)^i$, то существует линейный код над $\mathrm{GF}(q)$ длины n с минимальным расстоянием не менее d и не более чем $r = n - k$ проверочными символами.

Доказательство. Построим $(n-k) \times n$ матрицу H, у которой любые $d-1$ столбцов линейно независимы. Первый столбец — произвольный ненулевой вектор. Если уже выбраны j столбцов, то в качестве $(j+1)$-го нельзя использовать линейные комбинации любых $d-2$ выбранных столбцов, число которых $\sum_{i=0}^{d-2} \binom{j}{i} (q-1)^i < \sum_{i=0}^{d-2} \binom{n-1}{i} (q-1)^i < q^r = q^{n-k}$. Значит, запрещены еще не все q^{n-k} векторов, и можно выбрать еще один столбец. □

Итог: существует (n, k, d) код над $\mathrm{GF}(q)$ с $A_q(n, d) \geq q^k$, где k — наибольшее целое с $q^k < q^n / \sum_{i=0}^{d-2} \binom{n-1}{i} (q-1)^i$. Существует бесконечная последовательность двоичных линейных кодов со скоростью $R \geq 1 - h\left(\frac{d}{n}\right)$: граница ВГ асимптотически достижима.

2.17.3 Граница Плоткина

Пусть $K(n, d)$ — максимально возможное число информационных символов двоичного линейного кода длины n с минимальным расстоянием d.

Лемма 1. Для любого двоичного кода из M слов с минимальным расстоянием $d > \frac{n}{2}$ мощность ограничена:

$$M \leq \frac{2d}{2d-n}.$$

Доказательство. Просуммируем попарные расстояния по всем упорядоченным парам различных слов. Каждая пара дает не менее d, поэтому сумма не меньше $M(M-1)d$. С другой стороны, в каждой координате не более половины слов имеют данное значение (для линейного кода — ровно половина, так как $x \mapsto (xG)_i$ — линейный функционал), поэтому каждая координата различает не более $\frac{M^2}{4}$ упорядоченных пар, и сумма не больше $n\frac{M^2}{4}$. Итак, $M(M-1)d \leq n\frac{M^2}{4}$, откуда $M \leq \frac{2d}{2d-n}$. □

Для линейных кодов $M = 2^k$, поэтому при $d > \frac{n}{2}$

$$k \leq \log_2\left(2\frac{d}{2d-n}\right).$$

Лемма 2. K(n, d) <= K(n-1, d) + 1.

Доказательство. Возьмем (n, k, d) код с наибольшим $k = K(n, d)$. Рассмотрим подкод слов с фиксированным значением (скажем, 0) в первой координате: он содержит не менее половины кодовых слов (для линейного кода его размерность не менее $k-1$). Выколов первую координату у всех слов этого подкода, мы не изменим попарные расстояния (слова подкода совпадают в первой координате) и получим $(n-1, k-1, d)$ код. Значит, $K(n-1, d) \geq K(n, d) - 1$. □

Применяя лемму о мощности и рекуррентное соотношение, получаем **границу Плоткина**: для любого $(n, Rn, \delta n)$ кода

$$R \leq 1 - 2\delta, \quad K(n, d) \leq K(2d-1, d) + n - 2d + 1.$$

При $\delta > \frac{1}{2}$ длинных кодов с $R > 0$ не существует: $\lim_{n\to\infty} \frac{K(2d-1,d)}{n} = 0$, поскольку $K(2d-1, d) \leq \log_2\left(2\frac{d}{2d-(2d-1)}\right) = \log_2(2d) = o(n)$.

2.17.4 Граница Грайсмера

Пусть $N(k, d)$ — минимальная длина двоичного линейного кода размерности k с минимальным расстоянием d.

Теорема 13. N(k, d) >= d + N(k - 1, ceil(d/2)), и, следовательно,

$$N(k, d) \geq \sum_{i=0}^{k-1} \left\lceil \frac{d}{2^i} \right\rceil.$$

Доказательство. Пусть $C - (n, k, d)$ код наименьшей длины $n = N(k, d)$. Возьмем кодовое слово минимального веса d и объявим его первой строкой порождающей матрицы; перестановкой столбцов добьемся вида $u = (1, ..., 1, 0, ..., 0)$ с единицами на первых d позициях:

$$G = \begin{bmatrix} 1 & \dots & 1 & 0 & \dots & 0 \\ a_0 & \dots & a_{d-1} & a_d & \dots & a_{n-1} \\ \vdots & & \vdots & \vdots & & \vdots \end{bmatrix},$$

где оставшиеся $k-1$ строк записаны как $(a \mid G')$, G' — матрица размера $(k-1)\times(n-d)$. Выколов первые d столбцов, G' порождает $(n-d, k-1)$ код C'; обозначим его минимальное расстояние d'. Пусть $w \in C'$, $\mathrm{wt}(w) = d'$, и $(xa \mid w) \in C$ — соответствующее кодовое слово, где xa — ограничения тех же строк на первые d столбцов. Тогда:

- $\mathrm{wt}(xa) + d' \geq d$, так как $(xa \mid w) \in C$;
- $\mathrm{wt}((1,\dots,1) - xa) + d' \geq d$, так как $(xa + (1,\dots,1) \mid w) \in C$ — это сумма $(xa \mid w)$ с первой строкой u; его вес равен $d - \mathrm{wt}(xa) + d'$.

Складывая оба неравенства, получаем $2d' \geq d$, то есть $d' \geq \lceil \frac{d}{2} \rceil$. Значит, $N\big(k-1, \lceil \frac{d}{2} \rceil\big) \leq N(k,d) - d$. □

2.17.5 Границы МРРВ и Бассалыго—Элайеса

Для двоичных кодов известны более сильные (нелинейные по δ) верхние границы:

- **Граница Мак-Элиса—Родемича—Рамсея—Уэлча** (МРРВ):

$$R \leq h_2\Big(\frac{1}{2} - \sqrt{\delta(1-\delta)}\Big), \quad \delta = \frac{d}{n};$$

- **Граница Бассалыго—Элайеса**, объединяющая идеи границ Хемминга и Плоткина (и дающая вторую границу МРРВ):

$$R \leq h_2\Big(\frac{1}{2} - \sqrt{\delta(1-2\delta)}\Big) \quad \Big(\text{при } \delta \leq \frac{1}{2}\Big).$$

Асимптотическая диаграмма границ для двоичных кодов в координатах (δ, R): сверху вниз — граница Варшамова—Гилберта (нижняя, область существования), затем Хемминга, Бассалыго—Элайеса, МРРВ, Плоткина ($R \leq 1 - 2\delta$), Синглтона ($R \leq 1 - \delta$).

2.18 Заключение

- Задача теории кодирования — построение методов кодирования данных, позволяющих восстановить их после передачи по зашумленному каналу.
- Основные параметры кода: алфавит, число кодовых слов, длина, минимальное расстояние.
- Линейные коды допускают компактное задание; их параметры: поле $\mathrm{GF}(q)$, размерность k, длина n, скорость $R = \frac{k}{n}$, минимальное расстояние.
- Существуют фундаментальные ограничения (границы) на параметры кодов.
- Декодирование линейных кодов в общем случае — NP-полная задача.
- Весовой спектр дуального кода определяется весовым спектром исходного кода (теорема Мак-Вильямса).

3 Универсальные методы декодирования линейных кодов

В этой лекции рассматриваются методы, применимые к **произвольному** линейному блоковому коду: метод порядковых статистик, декодирование по решеткам (алгоритм Витерби) и декодирование с мягким входом и мягким выходом (алгоритм Бала—Коке—Елинека—Равива, BCJR).

3.1 Критерии мягкого декодирования

Декодирование кода C по максимуму правдоподобия в канале с АБГШ эквивалентно декодированию по критерию минимального расстояния Евклида. Рассмотрим передачу по каналу с АБГШ символов $(-1)^{c}_i$, $c_i \in \{0,1\}$, то есть $y_i = (-1)^{c}_i + \eta_i$:

$$\arg\min_{c\in C} \sum_{i=0}^{n-1} \big(y_i - (-1)^{c}_i\big)^2 = \arg\min_{c\in C} \sum_{i=0}^{n-1} \big(y_i^2 - 2(-1)^{c}_i y_i + (-1)^{2c_i}\big) = \arg\max_{c\in C} \sum_{i=0}^{n-1} (-1)^{c}_i y_i.$$

Пусть $\hat{c}_i = \begin{cases} 0 & y_i > 0 \\ 1 & y_i \leq 0 \end{cases}$ — жесткие решения. Тогда

$$\arg\max_{c\in C}\sum_{i=0}^{n-1}(-1)_i^c y_i = \arg\max_{c\in C}\sum_{i=0}^{n-1}\left((-1)_i^c y_i - (-1)_i^{\hat{c}} y_i\right) = \arg\min_{c\in C} E(c,y),$$

где **корреляционная невязка**

$$E(c,y) = \sum_{i:c_i\neq\hat{c}_i} |y_i|,$$

— сумма модулей принятых значений на позициях, где кодовое слово c расходится с жестким решением (разность под знаком суммы равна $2|y_i|$ при $c_i \neq \hat{c}_i$ и нулю при совпадении, а общий член $\sum_i (-1)_i^{\hat{c}} y_i$ от c не зависит).

Вместо y_i может использоваться логарифмическое отношение правдоподобия $L_i = \log\frac{P\{c_i=0 \mid y_i\}}{P\{c_i=1 \mid y_i\}}$ в обеих формулах.

3.2 Метод порядковых статистик

Рассмотрим передачу кодовых слов $(c_0, \ldots, c_{n-1})$ двоичного (n,k) кода с помощью 2-АМ по каналу без памяти, например $y_i = (-1)_i^c + \eta_i$, $\eta_i \sim N(0,\sigma^2)$. Пусть L_i — логарифмические отношения правдоподобия, $\hat{c}_i$ — жесткие решения. Вероятность ошибки в $\hat{c}_i$ убывает с увеличением $|L_i|$.

Алгоритм 2 (декодирование по методу порядковых статистик (МПС)).

1. Выбрать информационную совокупность J кода, соответствующую наибольшим значениям $|L_i|$.
2. Привести порождающую матрицу кода к виду G_J с единичной подматрицей в столбцах J.
3. С большой вероятностью число неверных жестких решений $\hat{c}_i$, $i \in J$, мало. Перебрать все конфигурации ошибок e веса не более t на J и построить кодовые слова $c_e = (\hat{c}_J + e)G_J$, где $\hat{c}_J$ — вектор жестких решений на позициях J, а e — добавляемый вектор ошибок той же длины.
4. Выбрать наиболее правдоподобное из полученных кодовых слов.

Сложность алгоритма:

$$O\left(k^2 n + \sum_{i=0}^{t} i\binom{k}{i}\right).$$

При $t = \frac{d}{4}$ достигается вероятность ошибки, близкая к вероятности ошибки декодирования по максимуму правдоподобия.

3.3 Декодирование по решеткам

3.3.1 Мотивация

Рассмотрим двоичный (n,k,d) код C с порождающей матрицей G. Пусть $D(x,y)$ — функция расстояния Хемминга. Декодирование вектора y по максимуму правдоподобия:

$$\hat{c} = \arg\min_{c\in C} D(c,y) = \arg\min_{c\in C}\sum_{i=0}^{n-1} D(c_i, y_i).$$

Многие кодовые слова содержат одинаковые префиксы $(c_0, \ldots, c_a)$, и неразумно пересчитывать $\sum_{i=0}^{a} D(c_i, y_i)$ для них каждый раз заново. Задачу поиска ближайшего кодового слова можно сформулировать как задачу поиска кратчайшего пути на графе.

3.3.2 Решетка

Определение 7. Решеткой (trellis) называется граф, обладающий следующими свойствами:

- вершины разбиты на непересекающиеся подмножества (уровни, или ярусы);

- нулевой и последний ярусы содержат по одному узлу (терминальные узлы);
- граф направленный: допускается движение только от уровня с меньшим номером к уровню с большим номером;
- ребрам приписаны метки, соответствующие символам кодовых слов, а также метрики (веса, длины); длина пути равна сумме длин входящих в него ребер.

Естественная метрика ребра на ярусе i, помеченного символом c: $D(c, y_i)$. Сопоставим путям в решетке между терминальными узлами кодовые слова; тогда задача поиска кодового слова, минимизирующего аддитивную функцию длины, эквивалентна задаче поиска кратчайшего пути между терминальными узлами решетки.

3.3.3 Алгоритм Витерби

Обозначения: $M_{i,v}$ — метрика узла v на ярусе i; V_i — множество узлов на ярусе i; $c[i, v', v]$ — метка ребра между узлами $v' \in V_{i-1}$ и $v \in V_i$. Ребра, ведущие с яруса $i-1$ на ярус i, соответствуют $(i-1)$-му символу кодового слова (индексация символов — с нуля), поэтому в метрике ребра участвует y_{i-1}.

Алгоритм 3 (Витерби).

$$M_{0,0} = 0$$

; для $i = 1, \ldots, n$ и каждого $v \in V_i$: для каждого входящего ребра (v', v) вычислить метрику пути $M_{v',v} = M_{i-1,v'} + D(c[i, v', v], y_{i-1})$; метрика узла $M_{i,v} = \min_{v'} M_{v',v}$; узел-предшественник $B_{i,v} = \arg\min_{v'} M_{v',v}$.

Обратный проход: $\nu = 0$; для $i = n, \ldots, 1$: $\hat{c}_{i-1} = c\left[B_{i,\nu}, \nu\right]$, $\nu = B_{i,\nu}$. Возвратить $\hat{c}$ и $M_{n,0}$.

Число сложений не превосходит E — числа ребер решетки; число сравнений не превосходит $E - V$, где V — число узлов.

3.3.4 Минимальная решетка кода

Определение 8. Профиль сложности решетки: $(\xi_0, \ldots, \xi_n)$, $\xi_i = |V_i|$. Решетка называется минимальной, если профиль сложности $(\xi'_0, \ldots, \xi'_n)$ любой другой решетки того же кода удовлетворяет $\xi'_i \geq \xi_i$ для всех $0 \leq i \leq n$.

Построение минимальной решетки. Выпишем все кодовые слова $c^m = (c_0^m, \ldots, c_{n-1}^m)$ кода. Для некоторого i определим прошлое $c_m^p = (c_0^m, \ldots, c_{i-1}^m)$ и будущее $c_m^f = (c_i^m, \ldots, c_{n-1}^m)$. Ключевое наблюдение: в любой решетке пути, входящие в некоторый узел, имеют общее будущее, а пути, исходящие из одного узла, имеют общее прошлое. Поэтому

$$F_i = \left\{c^f \mid \exists c^p : \left(c^p, c^f\right) \in C\right\}$$

может быть единственным образом разбито на подмножества

$$F_i(c^p) = \left\{c^f \mid \left(c^p, c^f\right) \in C\right\}.$$

Сопоставим каждому такому подмножеству узел на ярусе i. Узел $v \in V_i$ связывается с узлом $v' \in V_{i-1}$, если для некоторого кодового слова прошлое, соответствующее v, является продолжением на один символ одной из последовательностей, ведущих в v'; этим символом помечается ребро (v', v).

Теорема 14. Всякий код имеет минимальную решетку; все минимальные решетки совпадают с точностью до нумерации узлов каждого яруса.

Доказательство. Рассмотрим произвольную решетку T' этого кода. В T' два слова $c_1 = \left(c_1^p, c_1^f\right)$, $c_2 = \left(c_2^p, c_2^f\right)$ могут иметь общую вершину на ярусе i только если $F_i(c_1^p) = F_i(c_2^p)$. По построению два пути, проходящие через общий узел в T', проходят также через общий узел в T (обратное неверно). Следовательно, число узлов на ярусе i в T' не меньше числа узлов на этом же ярусе в T. □

Признак минимальности решетки: если на каждом ярусе все пути, имеющие общее прошлое (будущее), проходят через один узел решетки, то такая решетка для данного кода минимальна.

3.3.5 Минимальная спэновая форма матрицы

Для (n, k) линейного кода C для каждого i, $0 \le i \le n$, множество прошлых и множество будущих слов также являются линейными кодами. Для построения решетки удобно найти их базисы сразу для всех i.

- **Начало** $b(x)$ вектора $(x_0, ..., x_{n-1})$ — номер первого его ненулевого элемента.
- **Конец** $e(x)$ — номер последнего его ненулевого элемента.
- Элементы на позициях $b(x), ..., e(x) - 1$ называются **активными**.

Определение 9. Порождающая матрица приведена к **минимальной спэновой форме** (МСФ), если все начала строк различны и все концы строк различны; строки упорядочены по возрастанию начал. Любая матрица может быть приведена к МСФ элементарными операциями над строками.

3.3.6 Построение решетки по порождающей матрице

Приведем порождающую матрицу к МСФ. На ярусе i узлы нумеруются значениями информационных символов, соответствующих строкам МСФ, активным в позиции i. Ребра помечаются линейными комбинациями активных элементов соответствующего столбца порождающей матрицы. Полученная решетка минимальна.

Теорема 15. Решетка, получаемая по порождающей матрице в МСФ, минимальна.

Доказательство. Докажем, что для любого l пути, определяющие кодовые слова с одинаковым будущим c^f длины $n - l$, не проходят через различные узлы на ярусе с номером l. Узел, через который проходит путь на ярусе l, определяется значениями информационных символов, соответствующих активным на этом ярусе строкам. Эти строки линейно независимы и заканчиваются на ярусах с номерами больше l, поэтому нетривиальные линейные комбинации этих строк отличаются хотя бы на одной позиции правее l. Если бы существовали два слова с одинаковым будущим, проходящие через разные узлы на ярусе l, их сумма образовала бы слово, активное на ярусе l и равное нулю на позициях правее l, — противоречие. Значит, слова с одинаковым будущим проходят через одни и те же узлы, то есть решетка минимальна. □

3.3.7 Построение решетки по проверочной матрице

Пусть дана проверочная матрица $H = (h_0, ..., h_{n-1})$ (столбцы h_i). Положим $S_0 = 0$ и определим **накопленный синдром**

$$S(x_0, ..., x_{j-1}) = \sum_{i=0}^{j-1} h_i x_i.$$

Будем нумеровать узлы v в решетке накопленными синдромами $S(v)^T$: здесь $S(v)$ — накопленный синдром пути, ведущего в узел v (сумма $\sum h_i x_i$ по всем пройденным ребрам с метками x_i). Существует ребро, помеченное символом c, из узла v' на ярусе i в узел v на ярусе $i + 1$, если

$$S(v) = S(v') + h_i c.$$

Далее оставляем единственный конечный узел, соответствующий нулевому синдрому (допустимым кодовым словам), и удаляем нетерминальные узлы, из которых не выходят ребра. Полученная решетка минимальна. Для реализации удобно (но не обязательно) привести проверочную матрицу к МСФ.

Теорема 16. Решетка, построенная по проверочной матрице, минимальна.

Доказательство. Докажем, что пути с одинаковым будущим c^f не проходят через разные узлы. Для кодового слова $c = (c^p, c^f)$ частичные синдромы, вычисленные по c^p и по c^f, совпадают (оба равны сумме всех столбцов H с весами кодового слова). Следовательно, все совпадающие c^f исходят из одного и того же узла, определяемого частичным синдромом c^p. □

3.4 Декодирование с мягким выходом

Длинные коды удобно строить, комбинируя короткие; декодеры составных кодов строятся из декодеров компонентных кодов. Взаимодействие декодеров осуществляется путем обмена апостериорными вероятностями. Пусть $y_0^{n-1} = (y_0, ..., y_{n-1})$ — результат передачи кодового слова кода C по каналу без памяти, и $C_i(a) = \{(c_0, ..., c_{n-1}) \in C \mid c_i = a\}$. Апостериорные вероятности символов:

$$P\{c_i = a \mid y_0^{n-1}\} = \frac{\sum_{c \in C_i(a)} P\{c \mid y_0^{n-1}\}}{P\{y_0^{n-1}\}},$$

а для двоичных кодов — апостериорные логарифмические отношения правдоподобия (ЛОПП):

$$L_i = \ln \frac{p\{c_i = 0 \mid y_0^{n-1}\}}{p\{c_i = 1 \mid y_0^{n-1}\}}.$$

Исходные данные — ЛОПП символов кодового слова $L_i = \ln \frac{p(y_i \mid c_i=0)}{p(y_i \mid c_i=1)}$. Декодер, принимающий на входе и выдающий на выходе ЛОПП, называется **декодером с мягким входом и мягким выходом** (soft input soft output, SISO).

3.4.1 Алгоритм Бала—Коке—Елинека—Равива (BCJR)

Запишем ЛОПП через решетку кода:

$$L_i = \ln \left(\sum_{(s',s) \in S_0} \frac{p(s_i = s', s_{i+1} = s, y_0^{n-1})}{\sum_{(s',s) \in S_1} p(s_i = s', s_{i+1} = s, y_0^{n-1})} \right),$$

где S_0 и S_1 — множества пар состояний $s' \in V_i$, $s \in V_{i+1}$, переход между которыми помечен соответственно 0 и 1; $p(y_0^{n-1})$ — совместная плотность принятых сигналов; $p(s_i = s', s_{i+1} = s, y_0^{n-1})$ — совместная плотность принятых сигналов и состояний кодера на ярусах i и $i+1$.

Поведение кодера при обработке i-го символа определяется только его состоянием s' на предыдущем шаге, а канал не имеет памяти, поэтому факторизация:

$$p(s_i = s', s_{i+1} = s, y_0^{n-1}) = \underbrace{p(s_i = s', y_0^{i-1})}_{\alpha_i(s')} \underbrace{p(s_{i+1} = s, y_i \mid s_i = s')}_{\gamma_{i+1}(s',s)} \underbrace{p(y_{i+1}^{n-1} \mid s_{i+1} = s)}_{\beta_{i+1}(s)}.$$

Лемма 3. Рекуррентные формулы (из формулы Байеса):

$$\alpha_i(s) = p(s_i = s, y_0^{i-1}) = \sum_{\tilde{s} \in V_{i-1}} \alpha_{i-1}(\tilde{s}) \gamma_i(\tilde{s}, s), \quad s \in V_i,$$

$$\beta_i(\tilde{s}) = p(y_i^{n-1} \mid s_i = \tilde{s}) = \sum_{s \in V_{i+1}} \gamma_{i+1}(\tilde{s}, s) \beta_{i+1}(s), \quad \tilde{s} \in V_i,$$

где $\beta_i(\tilde{s})$ — плотность будущих принятых сигналов $y_i, ..., y_{n-1}$ при условии, что кодер на ярусе i находится в состоянии $\tilde{s}$.

Непосредственное вычисление этих величин приводит к ошибкам округления, поэтому вводятся нормированные величины $\alpha'_i(s) = \alpha_i(s)/p(y_0^{i-1})$ и $\beta'_i(s) = \beta_i(s)/p(y_i^{n-1} \mid y_0^{i-1})$. Поделив $p(s_i = s', s_{i+1} = s, y_0^{n-1})$ на $p(y_i)/p(y_0^{i-1})$, получаем

$$p(s_i = s', s_{i+1} = s \mid y_0^{n-1})p(y_i) = \alpha'_i(s')\gamma_{i+1}(s', s)\beta'_i(s),$$

откуда

$$L_i = \ln\left(\sum_{(s',s)\in S_0} \alpha'_i(s')\gamma_{i+1}(s', s)\beta'_{i+1}\frac{s}{\sum_{(s',s)\in S_1}}\alpha'_i(s')\gamma_{i+1}(s', s)\beta'_{i+1}(s)\right).$$

Нормированные вероятности вычисляются рекурсивно. Учитывая, что $p(y_0^{i-1}) = \sum_{s\in V_i} \alpha_i(s)$, получаем прямую рекурсию

$$\alpha'_i(s) = \frac{\sum_{\tilde{s}\in V_{i-1}} \alpha'_{i-1}(\tilde{s})\gamma_i(\tilde{s}, s)}{\sum_{s'\in V_i}\sum_{\tilde{s}\in V_{i-1}} \alpha'_{i-1}(\tilde{s})\gamma_i(\tilde{s}, s')}$$

с начальными условиями

$$\alpha'_0(s) = \alpha_0(s) = \begin{cases}1 & s = 0\\ 0 & s \neq 0\end{cases}.$$

Аналогично, обратная рекурсия:

$$\beta'_i(\tilde{s}) = \frac{\sum_{s\in V_{i+1}} \gamma_{i+1}(\tilde{s}, s)\beta'_{i+1}(s)}{\sum_{s'\in V_i}\sum_{s\in V_{i+1}} \alpha'_i(s')\gamma_{i+1}(s', s)}$$

с начальными условиями

$$\beta'_n(s) = \beta_n(s) = \begin{cases}1 & s = 0\\ 0 & s \neq 0\end{cases}.$$

Переходные метрики:

$$\gamma_{i+1}(s', s) = p(s_{i+1} = s, y_i \mid s_i = s') = P\{c_i = \delta(s', s)\}p(y_i \mid c_i = \delta(s', s)),$$

где $\delta(s', s)$ — метка перехода, а $P\{c_i = \delta(s', s)\}$ — априорная вероятность того, что бит равен этой метке. Если декодер используется как часть итеративного декодера составного кода, эта вероятность берется из внешних ЛОПП L_i^e, вычисленных другим декодером: при обозначении $L_i^e = \ln\frac{P\{c_i=1\}}{P\{c_i=0\}}$ имеем $P\{c_i = 1\} = e^{L_i^e}/\left(1 + e^{L_i^e}\right)$ и

$$P\{c_i = a\} = \frac{e^{\frac{L_i^e}{2}}}{1 + e^{L_i^e}} \cdot e^{(2a-1)\frac{L_i^e}{2}}.$$

В противном случае все символы считаются равновероятными, что эквивалентно $L_i^e = 0$.

Алгоритм 4 (BCJR).

1. Найти логарифмические отношения правдоподобия отдельных символов $L(c_i)$, $i = 0, ..., n-1$.
2. Вычислить величины $\gamma_{i+1}(s', s) = P\{c_i = \delta(s', s)\}p(y_i \mid c_i = \delta(s', s))$.
3. Вычислить $\alpha'_i(s)$ прямой рекурсией, $0 < i \leq n$.
4. Вычислить $\beta'_i(s)$ обратной рекурсией, $0 \leq i < n$.
5. Вычислить апостериорные ЛОПП L_i, $0 \leq i < n$, по формуле через суммы $\alpha'\gamma'\beta'$.

6. Принять решение относительно каждого символа: $\hat{c}_i = 0$, если $L_i > 0$.

Полученная последовательность решений может не являться кодовым словом: BCJR реализует **посимвольное** декодирование по максимуму апостериорной вероятности, в отличие от алгоритма Витерби, реализующего декодирование по максимуму правдоподобия для всего кодового слова.

4 Отказоустойчивое хранение данных: локально восстанавливаемые коды

4.1 Мотивация

Рассмотрим (n, k, d) код с максимальным достижимым расстоянием: $d = n - k + 1$. Каждый проверочный символ зависит от всех k информационных символов, поэтому исправление даже одного стирания требует доступа к не менее чем k нестертым символам.

В системах хранения данных операции ввода-вывода намного более трудоемки по сравнению с вычислениями (обращение к дискам, передача данных по сети). Поэтому необходимо обеспечить восстановление одиночных стираний (или небольшого их числа) путем считывания небольшого числа нестертых символов. Для этого служат **локально восстанавливаемые коды**.

4.2 Локальность

Пусть (n, k, d) линейный блоковый код C над $\mathrm{GF}(q)$ имеет порождающую матрицу G, состоящую из столбцов $g_1, \ldots, g_n$; обозначим $\mathcal{G} = \{g_1, \ldots, g_n\}$.

Определение 10. Локальность столбца: $\mathrm{Loc}(g_i)$ — наименьшее натуральное число r такое, что существует $R \not\subset \mathcal{G}$: $|R| = r$, $i\neg \in R$ и $g_i = \sum_{g \in R} \lambda_g g$ для некоторых $\lambda_g \in \mathrm{GF}(q)$. Очевидно, $\mathrm{Loc}(g_i) \leq k$.

Определение 11. Код C имеет информационную локальность r, если существует линейно независимый набор $I \not\subset \mathcal{G}$ из k столбцов такой, что $\forall g \in I : \mathrm{Loc}(g) \leq r$.

Определение 12. Код C является $[r, d]$-кодом, если он имеет информационную локальность r и минимальное расстояние d.

4.3 Минимальное расстояние в терминах столбцов

Теорема 17. Код C имеет минимальное расстояние d тогда и только тогда, когда для всякого $S \not\subset \mathcal{G}$ с $\mathrm{rank}(S) \leq k - 1$ выполняется $|S| \leq n - d$.

Доказательство. ($\Longrightarrow$) Пусть нашлось $S \not\subset \mathcal{G}$: $\mathrm{rank}(S) = \kappa \leq k - 1$, $|S| = \nu > n - d$. Выкалывание (удаление) исходного кода на всех позициях, кроме S, привело бы к коду длины ν с минимальным расстоянием $d - n + \nu > 0$ и порождающей матрицей G' из столбцов S; однако каждое его кодовое слово лежит в κ-мерном пространстве, порождаемом столбцами S, то есть $\mathrm{rank}(G') = \kappa < k$ и среди кодовых слов были бы нулевые различия — противоречие с положительным расстоянием.

($\Longleftarrow$) Пусть для всех S с $\mathrm{rank}(S) \leq k - 1$ выполняется $|S| \leq n - d$, но минимальное расстояние C равно $\delta \neq d$. Приведем порождающую матрицу C к виду $G = \begin{bmatrix} A & B \\ 0 & c \end{bmatrix} P$, где $(0, c)^T P$ — кодовое слово веса δ, P — перестановочная матрица. Если $\delta < d$, то A — матрица размера $(k - 1) \times (n - \delta)$, и ее столбцы образуют множество S с $\mathrm{rank}(S) = k - 1$, $|S| = n - \delta > n - d$ — противоречие,

значит $\delta \geq d$. Если $\delta > d$, то каждое S с $\operatorname{rank}(S) \leq k-1$ задает кодовое слово веса $\operatorname{wt}(c) \leq d < \delta$, что невозможно; значит $\delta = d$. $\square$

4.4 Граница Гопалана–Хуанга–Симитци–Еханина

Теорема 18. Для любого (n, k, d) кода над $\mathrm{GF}(q)$ с информационной локальностью r выполняется

$$n - k \geq \left\lceil \frac{k}{r} \right\rceil + d - 2.$$

Схема доказательства. Для всякого кода C длины n множество линейных зависимостей между $r+1$ символами его кодовых слов определяет гиперграф $H(V, E)$: $V = \mathcal{G}$, а E содержит гиперребра $S \not\subset V$: $|S| \leq r+1$ и существует $\lambda_i \neq 0$: $\sum_{i \in S} \lambda_i g_i = 0$ (такие ребра существуют в силу локальности). Построим $S \not\subset \mathcal{G}$: $\operatorname{rank}(S) \leq k-1$ итеративно:

Алгоритм 5 (MakeHypergraph). $i = 1$, $S_0 = \varnothing$; пока $\operatorname{rank}(S_{i-1}) \leq k-2$: выбрать $g_i \in \mathcal{G} \setminus S_{i-1}$ такой, что существует гиперребро $T_i \in H$: $g_i \in T_i$; если $\operatorname{rank}(S_{i-1} \cup T_i) < k$, то $S_i = S_{i-1} \cup T_i$, иначе выбрать $T' \not\subset T_i$ так, что $\operatorname{rank}(S_{i-1} \cup T') = k-1$ и положить $S_i = S_{i-1} \cup T'$. Возвратить S_{i-1}.

Пусть l — число итераций, $s_i = |S_i| - |S_{i-1}| \leq r+1$, $t_i = \operatorname{rank}(S_i) - \operatorname{rank}(S_{i-1})$, тогда $\operatorname{rank}(S_l) = \sum_{i=1}^{l} t_i = k-1$. Всякий добавляемый набор из s_i векторов таков, что его нетривиальная линейная комбинация лежит в $\operatorname{Span}(S_{i-1})$, поэтому $t_i \leq s_i - 1 \leq r$.

Случай 1. $\forall i : \operatorname{rank}(S_{i-1} \cup T_i) < k$. Тогда $t_i \leq s_i - 1 \leq r$ для всех i, откуда $l \geq \left\lceil \frac{k-1}{r} \right\rceil$ и

$$|S_l| = \sum_{i=1}^{l} s_i \geq \sum_{i=1}^{l} (t_i + 1) \geq k - 1 + \left\lceil \frac{k-1}{r} \right\rceil \geq k - 2 + \left\lceil \frac{k}{r} \right\rceil.$$

Случай 2. На последней итерации $\operatorname{rank}(S_{l-1} \cup T_l) = k$. На каждой из первых $l-1$ итераций ранг увеличивается не более чем на r, поэтому $l - 1 \geq \left\lceil \frac{k}{r} \right\rceil - 1$; при этом $t_l \geq 1$, $s_l \geq t_l$, и

$$|S_l| = \sum_{i=1}^{l} s_i \geq \sum_{i=1}^{l-1} (t_i + 1) + t_l \geq k - 1 + l - 1 \geq k - 2 + \left\lceil \frac{k}{r} \right\rceil.$$

В силу минимального расстояния d (по теореме из предыдущего раздела) $|S_l| \leq n - d$, откуда

$$n - d \geq k - 2 + \left\lceil \frac{k}{r} \right\rceil.$$

4.4.1 Хорошая структура локальных проверок

Теорема 19. Пусть дан (n, k, d) код с информационной локальностью r, причем r делит k (запись $r \mid k$), $r < k$ и $n = k + \frac{k}{r} + d - 2$. Тогда гиперребра в гиперграфе $H(V, E)$ не пересекаются и имеют мощность ровно $r+1$.

Доказательство. Воспользуемся алгоритмом MakeHypergraph. При $r = 1$ имеем $t_i \leq 1$ и $n - d \geq |S_l| = \sum s_i \geq \sum (t_i + 1) \geq 2k - 2$; так как $n - d = 2k - 2$, то $l = k - 1$, $s_i = 2$, $t_i = 1$ — гиперребра мощности 1 и пересекающиеся гиперребра отсутствуют.

При $r > 1$ и $r \mid k$ имеем $\left\lceil \frac{k-1}{r} \right\rceil = \frac{k}{r}$ (так как $k - 1 = r\left(\frac{k}{r} - 1\right) + (r-1)$), поэтому оценка случая 1 дает $|S| \geq k - 1 + \frac{k}{r} > n - d = k + \frac{k}{r} - 2$ — противоречие с $|S_l| \leq n - d$; значит, реализуется случай 2. Тогда $n - d \geq |S_l| = \sum_{i=1}^{l} s_i \geq \sum_{i=1}^{l} t_i + (l-1) \geq k + \frac{k}{r} - 2$. Так как $\sum_{i=1}^{l} t_i = k - 1$ и приращения $t_i \leq r$, существует ровно одно j: $t_j = r - 1$, $t_i = r$ при $i \neq j$. Если бы $j < l$, то $\sum_{i=1}^{l-1} t_i = k - r - 1$, $t_l = r$ — это был бы случай 1; следовательно, $j = l$.

Если бы существовало гиперребро T с $|T| \le r$, то, добавляя его первым, получили бы $t_1 \le r-1$ — значит, все $|T| = r+1$. Если бы $T_1 \cap T_2 \neq \varnothing$, то $\operatorname{rank}(T_1 \cup T_2) < 2r$, то есть $t_1 + t_2 \le 2r-1$, что противоречит наличию единственного $t_j = r-1$ при $l = \frac{k}{r} \ge 3$; при $\frac{k}{r} = 2$ это был бы случай 1.□

4.4.2 Канонические коды и несуществование

Определение 13. Пусть дан (n,k,d) код с порождающей матрицей G в систематическом виде, имеющий информационную локальность r, где $r \mid k$, $r < k$, $n = k + \frac{k}{r} + d - 2$. Код называется каноническим, если множество столбцов G разбивается на:

- единичные векторы $I = \{e_0, \dots, e_{k-1}\}$;
- столбцы локальных проверок $G' = \left\{g'_0, \dots, g'_{\frac{k}{r}-1}\right\}$, $\operatorname{wt}(g'_i) = r$, с непересекающимися носителями ненулевых элементов;
- прочие столбцы $G'' = \{g''_0, \dots, g''_{d-3}\}$, $\operatorname{wt}(g''_i) = k$.

Теорема 20. При $d < r+3$, $r < k$, $r \mid k$, $n = k + \frac{k}{r} + d - 2$ всякий (n,k,d) код с информационной локальностью r является каноническим.

Теорема 21. Пусть $2 < d < r+3$, $r \mid k$, $n = k + \frac{k}{r} + d - 2$. Не существует (n,k,d) линейных кодов с локальностью r.

4.5 Пирамидальные коды

Пусть дан $(k+d-1, k, d)$ код C с максимальным достижимым расстоянием, $G = (I \mid A)$ — его порождающая матрица. Рассмотрим

$$G' = (I \mid Z \mid A),$$

где Z — матрица размера $k \times \frac{k}{r}$, содержащая столбцы веса не более r, имеющие ненулевые элементы на непересекающихся позициях, и не имеющая ненулевых строк. Тогда G' порождает

$$\left(k + d - 1 + \frac{k}{r}, \quad k, \quad \ge d\right) \quad \text{код с информационной локальностью } r.$$

Действительно, каждая группа из r информационных символов защищена своим столбцом из Z (локальная проверка), а коды, полученные выкалыванием локальных проверок, содержатся в исходном МДР-коде.

4.5.1 Коды в Windows Azure

Рассмотрим построение $(10, 6, 4)$ кода с информационной локальностью 3. Пусть $x_0, \dots, x_5$ — информационные символы, разбитые на две группы (x_0, x_1, x_2) и (x_3, x_4, x_5). Введем промежуточные переменные:

$$\begin{aligned} q_{x,0} &= \alpha_0 x_0 + \alpha_1 x_1 + \alpha_2 x_2, & q_{y,0} &= \beta_0 x_3 + \beta_1 x_4 + \beta_2 x_5, \\ q_{x,1} &= \alpha_0^2 x_0 + \alpha_1^2 x_1 + \alpha_2^2 x_2, & q_{y,1} &= \beta_0^2 x_3 + \beta_1^2 x_4 + \beta_2^2 x_5, \\ q_{x,2} &= x_0 + x_1 + x_2, & q_{y,2} &= x_3 + x_4 + x_5. \end{aligned}$$

Локальные проверочные символы: $p_x = q_{x,2}$, $p_y = q_{y,2}$. Глобальные проверочные символы:

$$p_0 = q_{x,0} + q_{y,0}, \quad p_1 = q_{x,1} + q_{y,1}.$$

Кодовое слово: $\left(x_0, \dots, x_5, p_x, p_y, p_0, p_1\right)$. Коэффициенты α_i, β_i подбираются так, чтобы обеспечить исправимость любых 3 стираний и максимально возможного числа конфигураций из 4 стираний.

4.5.2 Обеспечение исправимости конфигураций из 4 стираний

- **Не стерт ни один из проверочных символов; по 2 стирания в каждой группе** (x_i, x_j в первой, x_s, x_t во второй). Система восстановления имеет вид (уравнения для p_0 и p_1):

$$\begin{bmatrix} \alpha_i & \alpha_j & \beta_s & \beta_t \\ \alpha_i^2 & \alpha_j^2 & \beta_s^2 & \beta_t^2 \end{bmatrix} \begin{bmatrix} x_i \\ x_j \\ x_s \\ x_t \end{bmatrix} = \begin{bmatrix} p_0 \\ p_1 \end{bmatrix},$$

и требуется обратимость

$$\Delta = \begin{bmatrix} \alpha_i & \alpha_j & \beta_s & \beta_t \\ \alpha_i^2 & \alpha_j^2 & \beta_s^2 & \beta_t^2 \end{bmatrix}, \quad \det(\Delta) = (\alpha_j - \alpha_i)(\beta_t - \beta_s)(\alpha_i + \alpha_j - \beta_s - \beta_t) \neq 0.$$

- **Стерт p_x или p_y; 2 стирания в группе 1, 1 стирание в группе 2**: восстанавливаем по уравнению $q_{y,2}$ (или $q_{x,2}$) и глобальным проверкам:

$$\det(\Delta') = \beta_s(\alpha_j - \alpha_i)(\beta_s - \alpha_j - \alpha_i) \neq 0.$$

- **Стерты p_x, p_y; по одному стиранию в группах 1 и 2**:

$$\det(\Delta'') = \alpha_i \beta_s (\beta_s - \alpha_i) \neq 0.$$

Общие условия на коэффициенты:

$$\alpha_i, \alpha_j, \beta_s, \beta_t \neq 0, \quad \alpha_i, \alpha_j \neq \beta_s, \beta_t, \quad \alpha_i + \alpha_j \neq \beta_s + \beta_t.$$

Эти условия могут быть выполнены при выборе $\alpha_i, \beta_s \in \mathrm{GF}(2^4)$. Они обеспечивают исправимость всех конфигураций из 3 стираний и 86% конфигураций из 4 стираний.

4.6 Коды Тамо—Барга

Код Рида—Соломона — множество векторов $(f(x_0), \ldots, f(x_{n-1}))$ по значениям многочленов степени меньше k.

Определение 14. Многочлен $g(x)$ называется **хорошим**, если

1. $\deg g(x) = r + 1$ и $(r+1) \mid n$;
2. существует разбиение $A_1 \cup A_2 \cup \ldots \cup A_{\frac{n}{r+1}} = \{x_0, \ldots, x_{n-1}\}$ такое, что $\forall i : |A_i| = r + 1$ и $\forall \alpha, \beta \in A_i : g(\alpha) = g(\beta)$ (то есть g постоянна на каждом блоке A_i).

Определение 15. Пусть дан информационный вектор $a = \left(a_{ij}, 0 \leq i < r, 0 \leq j < \frac{k}{r}\right) \in \mathrm{GF}(q)^k$ и хороший многочлен $g(x)$. Положим

$$f_a(x) = \sum_{i=0}^{r-1} f_i(x) x^i, \quad f_i(x) = \sum_{j=0}^{\frac{k}{r}-1} a_{ij} g(x)^j.$$

Кодовое слово кода Тамо—Барга: $c = (f_a(x_i) \mid x_i \in A)$ — значения f_a во всех точках множества $A = \{x_0, \ldots, x_{n-1}\}$ (тех же, по которым строится разбиение на блоки A_j).

Локальное восстановление стертого символа, соответствующего позиции $\alpha \in A_j$, — интерполяция по уцелевшим символам блока A_j:

$$\delta(x) = \sum_{\beta \in A_j \setminus \{\alpha\}} c_\beta \prod_{\beta' \in A_j \setminus \{\alpha, \beta\}} \frac{x - \beta'}{\beta - \beta'}, \quad c_\alpha = \delta(\alpha).$$

Теорема 22. Код Тамо—Барга является $\left(n, k, d = n - k - \frac{k}{r} + 2\right)$ кодом с информационной локальностью r.

Доказательство. Многочлены $g(x)^j x^i$, $0 \le j < \frac{k}{r}$, $0 \le i < r$, имеют различные степени $(j(r+1)+i)$, а потому линейно независимы; значит, отображение $a \mapsto f_a(x)$ инъективно и $\deg f_a(x) \le \left(\frac{k}{r} - 1\right)(r+1) + r - 1 = k + \frac{k}{r} - 2 \le n - 2$. Операция кодирования инъективна, то есть размерность кода равна k.

Кодовое слово веса w соответствует многочлену степени не выше $k + \frac{k}{r} - 2$ с $n - w$ нулями, поэтому $w \ge n - \left(k + \frac{k}{r} - 2\right)$ и $d \ge n - k - \frac{k}{r} + 2$. В силу границы ГХСЕ достигается равенство: $d = n - k - \frac{k}{r} + 2$.

Каждое $f_i(x)$ постоянно на всяком блоке A_j, поэтому интерполяционный многочлен, построенный по точкам $(\beta, f_a(\beta))$, $\beta \in A_j$, имеет степень $\le r - 1$; для его построения достаточно r точек — это и есть локальное восстановление. □

4.6.1 Конструкция на основе мультипликативной подгруппы поля

Все ненулевые элементы $\mathrm{GF}(q)$ образуют циклическую группу по умножению порядка $q - 1$. Рассмотрим мультипликативную подгруппу $H \not\subset \mathrm{GF}(q) \setminus \{0\}$; ее порядок является делителем $q - 1$ в силу теоремы Лагранжа.

Аннулирующий многочлен подгруппы:

$$g_H(x) = \prod_{h \in H} (x - h).$$

Смежный класс $bH = \{bh \mid h \in H\}$. Для любых $\alpha, \beta \in bH$: $g_H(\alpha) = g_H(\beta)$. Действительно, пусть $\alpha = bh'$, $\beta = bh''$; тогда

$$g_H(bh') = \prod_{h \in H} (bh' - h) = (h')^{|H|} \prod_{h \in H} \left(b - \frac{h}{h'}\right) = \prod_{h \in H} (b - h) = g_H(b) = g_H(bh'').$$

Аннулирующий многочлен подгруппы порядка $r + 1$ является хорошим и может использоваться в конструкции кода Тамо—Барга. Возможна аналогичная конструкция на основе аддитивной подгруппы $\mathrm{GF}(q)$.

4.7 Выводы

- Коды с локальностью позволяют исправлять одиночные стирания, обращаясь к небольшому числу уцелевших символов.
- Граница ГХСЕ $n - k \ge \left\lceil \frac{k}{r} \right\rceil + d - 2$ задает фундаментальный компромисс между локальностью, расстоянием и избыточностью; коды Тамо—Барга достигают ее с равенством.
- Коды с локальностью применяются в сетевых и географически распределенных СХД (например, Windows Azure Storage).

5 Сверточные коды

5.1 Конструкция кодов

Задача кодера — сделать передаваемые символы статистически зависимыми. Блоковые коды реализуют функциональное преобразование блоков данных в кодовые слова; сверточные коды — **автоматное** отображение блоков данных в кадры кодового слова. Простейший автомат — регистр сдвига: кодер хранит m ранее поступивших блоков из k_0 символов и на каждом шаге выдает n_0 закодированных символов. Параметры:

$$R = \frac{k_0}{n_0} \quad \text{(скорость)}, \quad K = mk_0 \quad \text{(длина кодового ограничения)}.$$

Пример: $k_0 = 1, m = 2, K = 2, n_0 = 2$ с порождающими многочленами

$$g^{(1)}(x) = x^2 + 1, \quad g^{(2)}(x) = x^2 + x + 1.$$

Каждый из n_0 порождающих многочленов описывает один выходной поток: его двоичные коэффициенты $g_t^{(i)} \in \{0, 1\}$ показывают, какие выходы элементов памяти регистра (и текущий вход) суммируются по модулю 2 в i-м выходном символе; t — «глубина» элемента памяти ($t = 0$ — текущий вход, $t = m$ — самый старый). В примере первый выход равен $a_j + a_{j-2}$, второй — $a_j + a_{j-1} + a_{j-2}$.

5.1.1 Порождающая матрица сверточного кода

При $k_0 = 1$ выходная последовательность — линейная свертка информационной последовательности и порождающих многочленов кода:

$$c^{(i)}(x) = c_0^{(i)} + c_1^{(i)}x + \ldots = a(x)g^{(i)}(x) = \sum_{j \geq 0} x^j \sum_{t=0}^{m} a_{j-t} g_t^{(i)}, \quad 1 \leq i \leq n_0.$$

Теоретически кодовые слова имеют бесконечную длину. На практике длина кодового слова фиксирована: в конец информационной последовательности вводят несколько дополнительных битов, переводящих регистр сдвига в нулевое состояние (завершение хвоста). В общем случае

$$\big(c^{(1)}(x), \ldots, c^{(n_0)}(x)\big) = \big(a^{(1)}(x), \ldots, a^{(k_0)}(x)\big) G(x),$$

где $G(x)$ — многочленная порождающая матрица размера $k_0 \times n_0$. Сверточные коды являются линейными.

5.1.2 Графические представления

- **Древовидная диаграмма**: последовательности возможных переходов конечного автомата; обладает свойством самоподобия.
- **Решетчатая диаграмма**: более компактный способ задания кода. Кодовое слово — путь в решетке, начинающийся и заканчивающийся в нулевом состоянии; предполагается, что после информационной последовательности были поданы дополнительные биты, переводящие регистр в нулевое состояние.

5.1.3 Корректирующая способность

> **Определение 16**. l-м минимальным расстоянием кода d_l^* называется минимальное расстояние Хемминга между двумя кодовыми последовательностями, чьи первые l кадров различны (кадр — блок из n_0 выходных символов, выдаваемых за один шаг). Величина d_{m+1}^* называется минимальным расстоянием кода; последовательность $d_1^*, d_2^*, d_3^*, \ldots$ — дистанционным профилем кода.

Если в первых l кадрах произошло t ошибок, то первый кадр может быть исправлен при условии $2t + 1 \leq d_l^*$. **Минимальное свободное расстояние**:

$$d_{[\text{free}]} = \max_l d_l^*.$$

Свободная длина $n_{[\text{free}]}$ — длина ненулевого начального сегмента кодовой последовательности наименьшего веса.

5.1.4 Катастрофические кодеры

Катастрофический кодер отображает информационные последовательности бесконечного веса в кодовые последовательности конечного веса; они характеризуются наличием петель нулевого веса в графе переходов. Пример катастрофической порождающей матрицы: $G(x) = (x^2 + 1, x^2 + 1)$: единичные ошибки в канале могут привести к бесконечному числу ошибок декодера — при передаче нулевого

кодового слова ошибка вида ...00011000... будет продекодирована в информационную последовательность ...0001010101.... При использовании некатастрофического кодера ошибка декодирования приводит лишь к ограниченному числу ошибок на выходе.

> **Теорема 23**. Порождающая матрица не является катастрофической тогда и только тогда, когда НОД определителей всех $k_0 \times k_0$ подматриц $G(x)$ равен x^s для некоторого $s \geq 0$.

Всякий сверточный код имеет как катастрофические, так и некатастрофические порождающие матрицы.

5.1.5 Систематическое кодирование

Систематическое кодирование: информационная последовательность является подпоследовательностью кодовой. Любой сверточный код может быть преобразован к эквивалентному систематическому коду за счет введения фильтра с бесконечной импульсной характеристикой (БИХ); это преобразование информационной последовательности биективно и не влияет на корректирующие свойства кода. Порождающая матрица $G(x)$ может быть приведена к каноническому виду аналогично случаю линейных блоковых кодов — преобразованием над полем рациональных функций. Недостаток: некоторые информационные последовательности конечного веса могут породить кодовые слова бесконечного веса.

> **Пример 3**. G(x) = (x^2 + 1, x^2 + x + 1) -> ((x^2+1)/(x^2+x+1), 1). Для $a(x) = 1 + x^4 + x^5$ имеем $a(x)G(x) = (1 + x + x^2 + x^5, 1 + x^4 + x^5)$.

5.2 Декодирование

Декодирование по критерию максимума правдоподобия (эквивалентно критерию минимального расстояния) осуществляется алгоритмом Витерби: кодовые слова соответствуют путям в решетке, поэтому поиск ближайшего кодового слова сводится к поиску кратчайшего пути (см. главу об универсальных методах декодирования). Сложность декодирования растет экспоненциально с длиной кодового ограничения K и линейно с длиной кодируемой последовательности. На практике используются усечения выживших путей и мягкие метрики по корреляционной невязке.

5.3 Вероятность ошибки

5.3.1 Производящая функция

Вероятность ошибки декодирования определяется числом кодовых слов различного веса, то есть числом путей в решетке, начинающихся и заканчивающихся в нулевом состоянии. Метод подсчета:

- Пометим ребра графа переходов метками D^i, где i — вес выходной последовательности ребра; последовательность символов веса x характеризуется одночленом D^x. Совокупность кодовых слов характеризуется многочленом, например $2D^6 + 3D^8$.
- Расщепим нулевое состояние на начальное (исток a) и конечное (сток e).
- Пусть X_i — степенной ряд, коэффициенты которого равны числу путей, начинающихся в истоке и заканчивающихся в состоянии i.

Производящая функция $T(D) = \frac{X_e}{X_a}$ (в истоке $X_a = 1$) — степенной ряд, коэффициент при D^d равен числу кодовых слов веса d, выходящих из нулевого состояния и возвращающихся в него. Степень младшего ненулевого члена равна минимальному свободному расстоянию кода. Минимальное свободное расстояние растет с длиной кодового ограничения; при фиксированной длине блока сверточные коды проигрывают аналогичным блоковым.

> **Пример 4**. Для кода с $g^{(1)}(x) = x^2 + 1$, $g^{(2)}(x) = x^2 + x + 1$ состояния $a = 00$, $b = 10$, $c = 11$, $d = 01$ дают уравнения состояний:

$$X_b = D^2 X_a + D X_d,$$
$$X_c = D X_b + D X_c,$$
$$X_d = D X_c + D X_b,$$
$$X_e = D^2 X_d.$$

Решая систему, получаем производящую функцию

$$T(D) = \frac{X_e}{X_a} = \frac{D^5}{1 - 2D} = D^5 + 2D^6 + 4D^7 + 8D^8 + \ldots,$$

откуда $d_{[\text{free}]} = 5$.

5.3.2 Расширенная производящая функция

Исследуем зависимость веса кодового слова от веса информационной последовательности. Пометим ребра графа переходов метками $N^j D^i$, где i — вес выходной последовательности, а $j = 0$ для перехода по 0 и $j = 1$ для перехода по 1. **Расширенная производящая функция** $T(N, D)$ — степенной ряд от двух переменных, в котором коэффициент при $N^a D^b$ равен числу кодовых последовательностей веса b, порождаемых информационными последовательностями веса a.

Пример 5. Для того же кода уравнения состояний принимают вид:

$$X_b = N D^2 X_a + N D^0 X_d,$$
$$X_c = N D X_b + N D X_c,$$
$$X_d = D X_c + D X_b,$$
$$X_e = D^2 X_d,$$

а производящая функция

$$T(N, D) = \frac{N D^5}{1 - 2ND} = N D^5 + 2N^2 D^6 + 4N^3 D^7 + 8N^4 D^8 + \ldots.$$

5.3.3 Вероятность ошибки декодирования

Пусть $r_{ij} = (-1)^{c}_{ij} + \eta_{ij}$, $\eta_{ij} \sim N(0, \sigma^2)$, $1 \le j \le n_0$. Предположим, что передавалось нулевое кодовое слово, а декодер Витерби ищет последовательность с максимальной корреляцией $C = \sum_{i \ge 0} \sum_{j=1}^{n_0} r_{ij} (-1)^{c}_{ij}$. Оценим вероятность первого события неправильного декодирования: при слиянии нескольких путей на некотором ярусе B ошибка произойдет, если метрика ненулевого пути C_1 окажется больше метрики нулевого пути C_0:

$$P\{C_1 > C_0\} = P\left\{ \sum_{i=0}^{B} \sum_{j=1}^{n_0} r_{ij} \left((-1)^{c}_{ij} - 1 \right) > 0 \right\} = P\left\{ \sum_{i=0}^{B} \sum_{j : c_{ij} \neq 0} r_{ij} < 0 \right\}.$$

Если неправильный путь имеет вес d на ярусах $0, \ldots, B$, то $\rho = \sum_{i=0}^{B} \sum_{j : c_{ij} \neq 0} r_{ij} \sim N(-d, d\sigma^2)$, $\sigma^2 = \frac{N_0}{2}$: передавалось нулевое слово, поэтому на позициях ошибок $c_{ij} = 1$ и $r_{ij} = -1 + \eta_{ij} \sim N(-1, \sigma^2)$, и

$$P_d = P\{\rho < 0\} = Q\left(\sqrt{\frac{2dE_s}{N_0}} \right) = Q\left(\sqrt{\frac{2dRE_b}{N_0}} \right).$$

Вероятность ошибки — вероятность того, что будет выбран какой-либо неправильный путь; по объединенной границе (union bound):

$$P_e = P\{(C_1 > C_0) \vee (C_2 > C_0) \vee \ldots\} \leq \sum_i P\{C_i > C_0\} = \sum_{d>0} t_d P_d = \sum_{d=d_{[\text{free}]}}^{\infty} t_d Q\left(\sqrt{\frac{2dRE_b}{N_0}}\right),$$

где t_d — коэффициенты производящей функции $T(D) = \sum_{d \geq 0} t_d D^d$.

5.3.4 Вероятность ошибки на бит

Вероятность ошибки на бит при выборе ошибочного пути C_i, отличающегося от истинного в d кодовых и w информационных позициях, равна $\frac{w}{k_0} \cdot P_d$. Расширенная производящая функция $T(N, D) = \sum_{w,d} t_{wd} N^w D^d$ дает

$$t(D) = \frac{\partial T(N, D)}{\partial N} \,|_{N=1} = \sum_d D^d \underbrace{\sum_w t_{wd} w}_{b_d},$$

и общая вероятность ошибки декодирования на бит:

$$P_b \leq \frac{1}{k_0} \sum_{d=d_{[\text{free}]}}^{\infty} b_d Q\left(\sqrt{\frac{2RdE_b}{N_0}}\right).$$

5.3.5 Результаты моделирования

Моделирование для кодов со скоростью $\frac{1}{2}$ (коэффициенты порождающих многочленов записаны в восьмеричной системе: каждая восьмеричная цифра задает три двоичных коэффициента, младшая цифра — коэффициент при x^0):

- $K = 5$: порождающие многочлены $(023, 035)_8$;
- $K = 7$: порождающие многочлены $(0133, 0171)_8$.

Кривые вероятности ошибки на бит для декодеров Витерби и BCJR (БКЕР) совпадают с объединенной верхней границей в области больших ОСШ и лежат существенно ниже кривой без кодирования; выигрыш растет с K.

5.4 Выводы

- Сверточные коды — понятийно простой способ помехозащиты.
- Сложность декодирования алгоритмом Витерби растет экспоненциально с длиной кодового ограничения и линейно с длиной кодируемой последовательности.
- Минимальное свободное расстояние растет с длиной кодового ограничения.

6 Методы модификации и комбинирования кодов

6.1 Конструкция Плоткина

> **Теорема 24**. Пусть даны (n, k_i, d_i) коды C_i, $i = 1, 2$. Тогда
>
> $$C = \{(c_1, c_1 + c_2) \mid c_i \in C_i, i = 1, 2\}$$
>
> — $(2n, k_1 + k_2, \min(2d_1, d_2))$ код.

Доказательство. C содержит кодовые слова (c_1, c_1), $c_1 \in C_1$, и $(0, c_2)$, $c_2 \in C_2$, поэтому $d \leq 2d_1$ и $d \leq d_2$. Пусть $c_1, c_1' \in C_1 \setminus \{0\}$, $c_2, c_2' \in C_2 \setminus \{0\}$ — ненулевые кодовые слова компонентных кодов. Тогда

$$d((c_1, c_1 + c_2), (c_1', c_1' + c_2')) = d(c_1, c_1') + d(c_1 + c_2, c_1' + c_2').$$

Если $c_2 = c_2'$ и $c_1 \neq c_1'$, то это выражение равно $2d(c_1, c_1') \geq 2d_1$. Если $c_2 \neq c_2'$, то

$$d(c_1, c_1') + d(c_1 + c_2, c_1' + c_2') = \mathrm{wt}(c_1 - c_1') + \mathrm{wt}(c_1 - c_1' + c_2 - c_2') \geq \mathrm{wt}(c_1 - c_1') + \mathrm{wt}(c_2 - c_2') - \mathrm{wt}(c_1 - c_1') = \mathrm{wt}(c_2 - c_2') \geq d_2.$$

□

6.1.1 Декодирование кодов конструкции Плоткина

Декодируем $(y', y'') = (c_1, c_1 + c_2) + (e', e'')$ в метрике Хемминга. Положим

$$y''' = y'' - y' = c_1 + c_2 + e'' - c_1 - e' = c_2 + e'''.$$

1. Продекодируем y''' декодером кода C_2. Если $\mathrm{wt}((e', e'')) \leq \left\lfloor \frac{d-1}{2} \right\rfloor$, то $\mathrm{wt}(e''') \leq \mathrm{wt}(e') + \mathrm{wt}(e'') \leq \left\lfloor \frac{d-1}{2} \right\rfloor \leq \left\lfloor \frac{d_2-1}{2} \right\rfloor$, и декодирование выполняется правильно.
2. Пусть c_2 найдено правильно. Продекодируем в C_1 векторы $y' = c_1 + e'$ и $y'' - c_2 = c_1 + e''$. Если $\mathrm{wt}(e') + \mathrm{wt}(e'') \leq \left\lfloor \frac{d-1}{2} \right\rfloor \leq \left\lfloor \frac{2d_1-1}{2} \right\rfloor < d_1$, то $\mathrm{wt}(e') \leq \left\lfloor \frac{d_1-1}{2} \right\rfloor$ или $\mathrm{wt}(e'') \leq \left\lfloor \frac{d_1-1}{2} \right\rfloor$, и декодирование y' или y'' даст правильный результат.

6.1.2 Коды Рида—Маллера

Рекурсивное применение конструкции Плоткина дает коды Рида—Маллера. Обозначим $\mathrm{RM}(r, m)$ — код Рида—Маллера порядка r длины 2^m:

$$\mathrm{RM}(0, m) = (2^m, 1, 2^m), \quad \mathrm{RM}(m, m) = (2^m, 2^m, 1),$$

$$\mathrm{RM}(r+1, m+1) : \text{конструкция Плоткина к } C_1 = \mathrm{RM}(r+1, m), C_2 = \mathrm{RM}(r, m).$$

Получаются коды

$$\mathrm{RM}(r, m) = \left(2^m, \sum_{i=0}^{r} \binom{m}{i}, 2^{m-r} \right).$$

6.2 Выкалывание, укорочение и расширение

6.2.1 Укорочение

Укороченный код получается выбором кодовых слов исходного кода, содержащих нули на заданных позициях, с последующим удалением этих позиций.

Пусть дан (n, k, d) код с порождающей матрицей $G = (I \mid A)$. Удалим из порождающей матрицы m столбцов единичной подматрицы и соответствующие m строк; если появятся нулевые столбцы, удалим их тоже. Получаем порождающую матрицу

$$(\leq n - m, \quad k - m, \quad \geq d) \quad \text{кода.}$$

Пример 6. (7, 4, 3) совершенный код, эквивалентный коду Хемминга, укорачивается до $(5, 2, 3)$ кода (совершенным не является).

6.2.2 Выкалывание

Выколотый код: удалим из всех кодовых слов заданные символы (как правило, проверочных). Пусть дана проверочная матрица (n, k, d) кода в форме $H = (A \mid -I)$. Удалим из H m столбцов единичной подматрицы и соответствующие m строк; если появятся линейно зависимые строки, удалим их. Получаем

$$(n - m, \quad \leq k, \quad \geq d - m) \quad \text{код.}$$

Пример 7 (построение оптимального кода (10, 3, 5)). Граница Грайсмера: $N(3, 5) \geq 5 + N(2, 3) \geq 5 + 3 + N(1, 2) = 5 + 3 + 2 = 10$.

1. C_1: код Хемминга $(7, 4, 3)$ укорачивается до $(6, 3, 3)$ с порождающей матрицей

$$G_1 = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

1. C_2: код с повторениями $(6, 1, 6)$, $G_2 = (1 \quad 1 \quad \ldots \quad 1)$.
2. Конструкция Плоткина $(C_1, C_1 + C_2)$ дает код $(12, 4, 6)$ с порождающей матрицей

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

1. Выкалывание одного (последнего) символа дает код $(11, 4, 5)$:

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

1. Укорочение на один символ дает код $(10, 3, 5)$:

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

6.2.3 Расширение кодов

Наиболее распространенный способ — добавление проверки на четность: $(n, k, d) \to (n + 1, k, d')$. Если минимальное расстояние d исходного кода нечетно, то минимальное расстояние расширенного кода $d' = d + 1$ (вес каждого слова четен, поэтому минимальный нечетный вес возрастает на единицу).

Пример 8. (7, 4, 3) код Хемминга расширяется до $(8, 4, 4)$ расширенного кода Хемминга.

6.3 Каскадные коды

Перемежитель переставляет символы так, что последствия ошибочного декодирования одного кода могут быть легко ликвидированы декодером другого кода. Схема: внешний кодер — перемежитель — внутренний кодер — канал — внутренний декодер — деперемежитель — внешний декодер.

6.3.1 Прямое произведение кодов

Пусть даны (n_1, k_1, d_1) код (кодирование по строчкам) и (n_2, k_2, d_2) код (кодирование по столбцам) с порождающими матрицами G', G''. Информационная матрица $k_1 \times k_2$ кодируется по строкам и столбцам, кодовое слово получается выписыванием полученной матрицы (например, по столбцам). Это $(n_1 n_2, k_1 k_2, d_1 d_2)$ код с порождающей матрицей — кронекеровским произведением

$$G' \otimes G'' \quad (\text{строки } G'_i G''), \quad R = \frac{k_1 k_2}{n_1 n_2} = \frac{k_1}{n_1} \cdot \frac{k_2}{n_2},$$

где $G' \otimes G''$ — произведение Кронекера (каждая строка результата — произведение строки G' на всю G'').

Код способен исправить многие конфигурации ошибок веса больше $d_1 \frac{d_2}{2}$; допускает параллельный алгоритм декодирования (декодирование строк и столбцов независимо).

6.3.2 Пример: код Рао—Редди (48, 31, 8)

Используется РЖД в рельсовых цепях сигнализации.

- Прямое произведение расширенного $(16, 11, 4)$ кода Хемминга C_1 и $(3, 2, 2)$ кода C_2 с проверкой на четность дает $(48, 22, 8)$ код. Кодовые слова имеют вид $(c_1, c_2, c_1 + c_2)$, $c_1, c_2 \in C_1$ (три блока-строки: верхняя, нижняя и их сумма).
- Дополнение кодовыми словами кода Рида—Маллера $(16, 5, 8)$ C_3 (с дописанными в конец 32 нулями) — кодовые слова вида $(c_1 + c_3, c_2, c_1 + c_2)$, $c_1, c_2 \in C_1$, $c_3 \in C_3$:

$$\text{t}((c_1, c_2, c_1 + c_2)) \geq 8; \quad c_3 \neq 0 : \text{wt}((c_1 + c_3, c_2, c_1 + c_2)) = \text{wt}(c_1 + c_3) + \text{wt}(c_2) + \text{wt}(c_1 + c_2) \geq \text{wt}((c_1 + c_3) + c_2 + (c_1 + c_2)) = \text{w}$$

Получен код $(48, 27, 8)$.

- Добавление подходящего 4-мерного кода C_4 дает код Рао—Редди $(48, 31, 8)$ из кодовых слов вида

$$(c_1 + c_3 + c_4, \quad c_2 + c_4, \quad c_1 + c_2 + c_4), \quad c_1, c_2 \in C_1, c_3 \in C_3, c_4 \in C_4.$$

6.3.3 Лестничные коды

Данные систематически кодируются по строкам и столбцам некоторыми кодами $B_0, B_1, B_2, \ldots$ с лестничным сдвигом (проверки текущей строки кодируются в следующих строках), образуя полубесконечное кодовое слово. Это компактный способ задания бесконечных произведений кодов со сдвигом.

6.3.4 Каскадные коды Форни

1. Закодировать данные внешним (N, K, D) кодом над $\mathrm{GF}\big(q^k\big)$. Строить хорошие коды над $\mathrm{GF}\big(q^k\big)$ при $k > 1$ проще, чем над $\mathrm{GF}(q)$; например, код Рида—Соломона с $D = N - K + 1$.
2. Представить символы кодового слова как векторы длины k над $\mathrm{GF}(q)$ и закодировать каждый символ (n, k, d) внутренним кодом над $\mathrm{GF}(q)$.

Получается (Nn, Kk, Dd) код над $\mathrm{GF}(q)$. Существуют каскадные коды, достигающие предела Шеннона для двоичного симметричного канала.

6.3.5 Граница Зяблова

Выберем внутренний (n, k, d) код на границе Варшамова—Гилберта с $r = \frac{k}{n} \geq 1 - h\big(\frac{d}{n}\big) = 1 - h(\delta)$, и внешний $(N, K, D = N - K + 1)$ код Рида—Соломона с $R = 1 - \frac{D-1}{N} \sim 1 - \Delta$. Существует код со скоростью $\rho = Rr$ и относительным расстоянием

$$\delta = \frac{Dd}{Nn} = \Delta \cdot \delta \text{ внутр} \geq (1 - R)(1 - h^{-1}(r)) = \left(1 - \frac{\rho}{r}\right)(1 - h^{-1}(r)),$$

$$\delta \geq \max_{0 \leq r < h(\delta)} \left(1 - \frac{\rho}{r}\right)(1 - h^{-1}(r)).$$

Далеко не все семейства кодов при длине, стремящейся к бесконечности, одновременно обеспечивают скорость $\rho > 0$ и относительное минимальное расстояние $\delta > 0$; граница Зяблова показывает, что каскадные конструкции это обеспечивают.

6.3.6 Обобщенные каскадные коды (Блох, Зяблов, Зиновьев)

- Внешние (N, K_i, D_i) коды A_i над $\mathrm{GF}(q^{m_i})$, $1 \leq i \leq s$.
- Вложенные внутренние (n, k_i, d_i) коды B_i: $B_1 \sup B_2 \sup \ldots \sup B_s$ над $\mathrm{GF}(q)$, $k_i - k_{i+1} = m_i$; код B_i порождается последними k_i строками $k_1 \times n$ матрицы B.

Кодирование:

1. Закодировать данные внешними кодами и записать полученные кодовые слова в $s \times N$ матрицу X.
2. Заменить элементы i-й строки X на их векторное представление (столбцы длины m_i); получится $k_1 \times N$ матрица Y.
3. Умножить каждый столбец Y на B (т.е. закодировать в коде B_1).
4. Полученная $n \times N$ матрица — кодовое слово.

Это линейный

$$\left(Nn, \quad \sum_{i=1}^{s} K_i m_i, \quad \geq \min_{1 \leq i \leq s} d_i D_i \right) \quad \text{код над } \mathrm{GF}(q).$$

Некоторые ОКК (полярные коды) достигают предела Шеннона.

Декодирование ОКК. Запишем принятые символы в виде $n \times N$ матрицы. Для $i = 1, ..., s$:

1. продекодировать столбцы в коде B_i;
2. продекодировать i-ю строку в коде A_i; пусть $(c_1, ..., c_N)$ — найденное кодовое слово;
3. вычесть из j-го столбца $c_j B_i^T$.

6.4 Турбо-коды

Одновременное кодирование информационных битов несколькими сверточными кодами позволяет исключить многие конфигурации ошибок, которые могли бы возникнуть при использовании независимых сверточных кодов. Требования к компонентным кодерам:

- должны использоваться **рекурсивные систематические** сверточные коды (РССК; «рекурсивный» означает, что вход сумматоров по модулю 2 подмешивается обратно в регистр сдвига): многие информационные последовательности малого веса превращаются в кодовые слова большого веса;
- длина кодового ограничения должна быть небольшой: кодам с большим кодовым ограничением присущи длинные пакеты ошибок декодирования.

Турбо-код является линейным блоковым кодом; для повышения скорости используется выкалывание. Информационные биты u_i кодируются двумя РССК-кодерами: первый выдает (u_i, c_i^1), второй — проверочные символы c_i^2 для перемешанной перестановкой последовательности; на выходе кадры $(c_{3i}^1, c_{3i+1}^2, c_{3i+2}^2)$-типа с систематическими символами и проверками обоих кодеров.

6.4.1 Турбо-декодер

Декодеры сверточных кодов, входящих в турбо-код, обмениваются информацией, полученной в результате декодирования; как правило, достаточно 5–10 итераций. Апостериорные логарифмические отношения правдоподобия информационных символов, вычисленные одним декодером, являются априорными ЛОПП для другого декодера. Получается аппроксимация декодера максимального правдоподобия. Этот подход применим и для декодирования прямого произведения кодов.

6.4.2 BCJR для сверточных кодов

Используются формулы алгоритма Бала—Коке—Елинека—Равива (глава об универсальных методах декодирования): прямая рекурсия для $\alpha_i'(s)$, обратная для $\beta_i'(s)$, апостериорные ЛОПП

$$L_i = \ln \left(\sum_{(s',s) \in S_1} \alpha_i'(s') \gamma_{i+1}(s', s) \beta_{i+1}' \frac{s}{\sum_{(s',s) \in S_0} \alpha_i'(s') \gamma_{i+1}(s', s) \beta_{i+1}'(s)} \right).$$

Вероятности переходов (для кода со скоростью $\frac{1}{2}$):

$$\gamma_{i+1}(s', s) = p(s_{i+1} = s, y_{2i}, y_{2i+1} \mid s_i = s') = P\{u_i = \delta(s', s)\} p(y_{2i}, y_{2i+1} \mid (c_{2i}, c_{2i+1}) = \Delta(s', s)),$$

где $\delta(s', s) \in \{0, 1\}$ — информационная метка перехода, $\Delta(s', s)$ — пара символов, формируемая кодером при переходе из состояния s' в s, а $P\{u_i = \delta(s', s)\}$ — априорная вероятность информационного символа; в турбо-декодере ее вычисляет второй компонентный декодер.

6.4.3 Декодирование с использованием ЛОПП

Внешние (extrinsic) ЛОПП: при обозначении $L_i^e = \ln \frac{P\{u_i=0\}}{P\{u_i=1\}}$ имеем

$$P\{u_i = 0\} = \frac{e^{L_i^e}}{1 + e^{L_i^e}}, \quad P\{u_i = a\} = A \exp\left((-1)^a \frac{L_i^e}{2} \right), \quad A = \frac{e^{\frac{L_i^e}{2}}}{1 + e^{L_i^e}}.$$

Пусть $S_i = \ln \frac{p(y_i \mid c_i=0)}{p(y_i \mid c_i=1)}$ — канальные ЛОПП. Вероятности выходных символов:

$$p(y_{2i}, y_{2i+1} \mid c_{2i}, c_{2i+1}) = B \exp((-1)^c_{2i} S_{2i} + (-1)^c_{2i+1} S_{2i+1}).$$

Коэффициенты A, B сокращаются во всех выражениях алгоритма BCJR и могут быть отброшены.

6.4.4 Получение внешних ЛОПП

Пусть используется систематическое кодирование и $c_{2i} = u_i$. Результат работы BCJR — апостериорные

$$L_i = \ln \frac{P\{u_i = 0 \mid y_0, ..., y_{2n-1}\}}{P\{u_i = 1 \mid y_0, ..., y_{2n-1}\}}.$$

Для турбо-декодирования необходимо вычислить

$$\hat{L}_i^e = \ln \frac{P\{Y_{2i} \mid u_i = 0\}}{P\{Y_{2i} \mid u_i = 1\}}, \quad Y_{2i} = (y_0, ..., y_{2i-1}, y_{2i+1}, ..., y_{2n-1}),$$

чтобы исключить двойной учет принятых символов (по крайней мере, на первой итерации). По формуле Байеса

$$P\{u_i = a \mid Y_{2i}, y_{2i}\} = \frac{P(Y_{2i} \mid u_i = a) P(y_{2i} \mid c_{2i} = a) P\{u_i = a\}}{P(Y_{2i}, y_{2i})},$$

и логарифмирование дает разложение

$$L_i = \underbrace{\ln \frac{P(Y_{2i} \mid u_i = 0)}{P(Y_{2i} \mid u_i = 1)}}_{\hat{L}_i^e} + \underbrace{\ln \frac{p(y_{2i} \mid c_{2i} = 0)}{p(y_{2i} \mid c_{2i} = 1)}}_{S_{2i}} + \underbrace{\ln \frac{P\{u_i = 0\}}{P\{u_i = 1\}}}_{L_i^e}.$$

Различные полуитерации используют различные Y_i, при этом изначально $P\{u_i = 0\} = P\{u_i = 1\} = \frac{1}{2}$, то есть $L_i^e = 0$. Поэтому

$$\hat{L}_i^e = L_i - S_{2i} - L_i^e.$$

6.4.5 Итеративный алгоритм декодирования

1. Положить $L^e_{2\to1}(u_i) = 0$.
2. Воспользоваться декодером BCJR для сверточного кодера 1.
3. Подвергнуть перемежению полученные ЛОПП $\hat{L}_i^e$ и использовать их как $L^e_{1\to2}(u_i)$.
4. Воспользоваться декодером BCJR для сверточного кодера 2.
5. Подвергнуть деперемежению полученные апостериорные ЛОПП $\hat{L}_i^e$ и использовать их как $L^e_{2\to1}(u_i)$.
6. Перейти к шагу 2, если не превышено максимальное число итераций (5–10).

6.4.6 Построение перемежителей

Требование: близкие позиции во входной последовательности должны отображаться в максимально удаленные позиции в выходной:

$$0 < |i - j| < d \Longrightarrow |\pi(i) - \pi(j)| \geq S,$$

где π — перестановка (перемежитель), d — порог «близости» исходных позиций, S — минимально требуемое расстояние между ними после перемежения (spreading).

Важны объем памяти, требуемый для реализации перемежителя, и его задержка. Виды:

- **Псевдослучайный перемежитель**: случайная генерация с отбрасыванием перестановок с неудовлетворительными S, d.
- **Табличный перемежитель**: например, $(1, 2, ..., 16) \to (13, 9, 5, 1, 14, 10, 6, 2, 15, 11, 7, 3, 16, 12, 8, 4)$.
- **Перестановочный полином**: если $Q(x) \equiv \sum_{i=0}^{t} q_i x^i \bmod N$ — биекция в $\mathbb{Z}_N$, то перестановка может быть задана как $Q(i) \to Q(i+1)$. Пример: $Q(x) \equiv kx(x+1) \bmod N$, $k \equiv 1 \bmod 2$.

6.4.7 Результаты моделирования

Для кода $(3072, 1024)$ кривые FER после 8 и 16 итераций лежат близко к **границе Полянского—Пура—Верду** — минимальной вероятности ошибки декодирования, достижимой кодом с заданной длиной и скоростью, и существенно ниже кривой без кодирования; итерации улучшают вероятность ошибки.

6.5 Выводы

- Длинные коды можно строить из коротких; составные коды допускают простое декодирование.
- Существуют каскадные коды, у которых относительное минимальное расстояние остается положительным при всех скоростях, и каскадные коды, достигающие предела Шеннона для ДСК (1966, Форни).
- Некоторые обобщенные каскадные коды (полярные) достигают предела Шеннона.
- Турбо-коды стали первым классом корректирующих кодов, которые смогли на практике приблизиться к пределу Шеннона (1993).

7 Полярные коды

7.1 Модели каналов

Дискретный канал без памяти с входным алфавитом $\mathcal{X} = \{0, 1\}$ и дискретным выходным алфавитом $\mathcal{Y}$ задается функцией переходных вероятностей $W(y|c)$ — вероятностью наблюдения на выходе канала символа $y \in \mathcal{Y}$ при подаче на вход $c \in \mathcal{X}$.

- **Двоичный симметричный канал** (ДСК): $\mathcal{Y} = \mathcal{X}$,
$$W(y|c) = \begin{cases} p & y \neq c \\ 1-p & y = c \end{cases}.$$
- **Двоичный стирающий канал** (БСЭ): $\mathcal{Y} = \{0, 1, \varepsilon\}$,
$$W(y|c) = \begin{cases} p & y = \varepsilon \\ 1-p & y = c \end{cases}.$$
- **Двоичный симметричный канал со стираниями** (БССК): $\mathcal{Y} = \{0, 1, \varepsilon\}$,
$$W(y|c) = \begin{cases} 1-p-s & y = c \\ s & y = \varepsilon \\ p & y = 1-c \end{cases}.$$

Для непрерывного выходного алфавита $W(y|c)$ — плотность распределения выходного символа. Для **аддитивного гауссовского канала** (АБГШ) $\mathcal{Y} = \mathbb{R}$, $y = (-1)^c + \eta$, $\eta \sim N(0, \sigma^2)$:

$$W(y|c) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-(-1)^c)^2}{2\sigma^2}\right).$$

7.2 Параметр Бхаттачарьи

Пусть приемник работает по критерию максимума правдоподобия $\hat{c} = \arg\max_{c\in\{0,1\}} W(y|c)$, и передаваемые символы равновероятны. Вероятность ошибки:

$$P_e = \sum_{y: W(y|0) < W(y|1)} W(y|0) + \sum_{y: W(y|1) < W(y|0)} W(y|1) = \sum_{y\in\mathcal{Y}} \sum_{c\in\{0,1\}} W(y|c)\chi\left(\frac{W(y|1-c)}{W(y|c)}\right),$$

где $\chi(z)$ — индикаторная функция: $\chi(z) = 1$ при $z \geq 1$ и $\chi(z) = 0$ при $z < 1$. Так как $\chi(z) \leq \sqrt{z}$, получаем верхнюю оценку:

$$P_e \leq \sum_{y\in\mathcal{Y}} \sum_{c\in\{0,1\}} W(y|c)\sqrt{\frac{W(y|1-c)}{W(y|c)}} = \sum_{y\in\mathcal{Y}} \sqrt{W(y|0)W(y|1)} =: Z(W).$$

Величина $Z(W)$ называется **параметром Бхаттачарьи** канала W.

Пример 9.

$$Z(\text{БСЭ}(p)) = \sqrt{W(0|0)W(0|1)} + \sqrt{W(1|0)W(1|1)} + \sqrt{W(\varepsilon|0)W(\varepsilon|1)} = p.$$

Для АБГШ:

$$Z(\text{АБГШ}(\sigma)) = \int_{-\infty}^{\infty} \sqrt{W(y|0)W(y|1)}\,\mathrm{d}y = \frac{1}{\sqrt{2\pi}\sigma}\int_{-\infty}^{\infty} \exp\left(-\frac{(y-1)^2+(y+1)^2}{4\sigma^2}\right)\mathrm{d}y = \frac{e^{-\frac{1}{2\sigma^2}}}{\sqrt{2\pi}\sigma}\int_{-\infty}^{\infty}\exp\left(-\frac{y^2}{2\sigma^2}\right)\mathrm{d}y = e^{-\frac{1}{2\sigma^2}}.$$

7.3 Пропускная способность

$$I(W) = \max_{\{p(x)\}} \sum_{x\in\mathcal{X}}\sum_{y\in\mathcal{Y}} W(y|x)P\{x\}\log_2\frac{W(y|x)}{W(y)}.$$

Существуют методы сколь угодно надежной передачи данных со скоростью $R < I(W)$; при $R > I(W)$ вероятность ошибки ограничена снизу положительной величиной. Для многих каналов (в том числе для всех рассматриваемых ниже симметричных каналов) оптимальным входным распределением является равномерное.

Связь параметра Бхаттачарьи и пропускной способности:

$$\log_2\frac{1}{1+Z(W)} \le I(W) \le 1 - Z(W)^2.$$

7.4 Поляризация канала

Рассмотрим линейное преобразование

$$(c_0, c_1) = (u_0 + u_1, u_1).$$

Пусть оба символа передаются по двум независимым экземплярам (двум использованиям) канала БСЭ со стиранием p: $y_i = c_i$ с вероятностью $1-p$ и $y_i = \varepsilon$ с вероятностью p.

- Символ u_1 не может быть восстановлен, если стерты оба символа: вероятность $p^2 \le p$ — **улучшенный** подканал.
- Символ u_0 не может быть восстановлен, если стерт хотя бы один символ: вероятность $1-(1-p)^2 = 2p - p^2 \ge p$ — **ухудшенный** подканал.

Применяя преобразование рекурсивно к парам экземпляров обоих подканалов, при m уровнях получаем $n = 2^m$ битовых подканалов. При $p = \frac{1}{2}$ и $m = 3$ параметры Бхаттачарьи подканалов принимают значения

$$(0.996,\quad 0.879,\quad 0.81,\quad 0.32,\quad 0.68,\quad 0.19,\quad 0.121,\quad 0.039),$$

то есть часть подканалов почти полностью «шумит», а часть почти идеальна — это и есть **поляризация**. Доля недополяризованных подканалов убывает с ростом m, при этом абсолютное число таких подканалов растет.

7.4.1 Битовые подканалы

Пусть

$$A_m = ([1,0],[1,1])^{\otimes m},$$

где $\otimes\, m$ — m-кратное кронекеровское возведение матрицы в степень, и $n = 2^m$. Кодирование: $c_0^{n-1} = u_0^{n-1}A_m$. Используется краткая запись подвекторов $y_a^b = (y_a, y_{a+1}, \ldots, y_b)$ и $W_m^{n-1}(y_0^{n-1} \mid c_0^{n-1}) = \prod_{i=0}^{n-1} W(y_i \mid c_i)$.

Определение 17. Синтетические битовые подканалы задаются функциями переходных вероятностей

$$W_m^i(y_0^{n-1}, u_0^{i-1} \mid u_i) = 2^{1-n} \sum_{u_{i+1}^{n-1}} \prod_{j=0}^{n-1} W\Big(y_j \mid (u_0^{n-1} A_m)_j\Big).$$

Здесь $W_m^i(y_0^{n-1}, u_0^{i-1} \mid u_i)$ — вероятность того, что приемник наблюдает вектор y_0^{n-1}, а в переданном векторе u_0^{n-1} уже известны первые i битов u_0^{i-1}, и передавался бит u_i; суммирование ведется по всем возможным значениям еще неизвестных битов $u_{i+1}, \ldots, u_{n-1}$ (каждый набор равновероятен с весом 2^{1-n}).

7.4.2 Рекурсивное определение подканалов

Для подканалов четных и нечетных номеров ($y_{0,\text{even}}^{2\lambda-1}$ — подпоследовательность символов с четными индексами, $y_{0,\text{odd}}^{2\lambda-1}$ — с нечетными):

$$W_\lambda^{2i}(y_0^{2\lambda-1}, u_0^{2i-1} \mid u_{2i}) = \sum_{u_{2i+1}} W_{\lambda-1}^i\Big(y_{0,\text{even}}, u_{0,\text{even}}^{2i-1} + u_{0,\text{odd}}^{2i-1} \mid u_{2i} + u_{2i+1}\Big) \cdot W_{\lambda-1}^i\Big(y_{0,\text{odd}}, u_{0,\text{odd}}^{2i-1} \mid u_{2i+1}\Big),$$

$$W_\lambda^{2i+1}(y_0^{2\lambda-1}, u_0^{2i} \mid u_{2i+1}) = \frac{1}{2} \cdot W_{\lambda-1}^i\Big(y_{0,\text{even}}, u_{0,\text{even}}^{2i-1} + u_{0,\text{odd}}^{2i-1} \mid u_{2i} + u_{2i+1}\Big) \cdot W_{\lambda-1}^i\Big(y_{0,\text{odd}}, u_{0,\text{odd}}^{2i-1} \mid u_{2i+1}\Big).$$

Схема: символ u_{2i} подается на вход $W_{\lambda-1}^i$ (четные выходы) вместе с суммой $u_{2i} + u_{2i+1}$, а символ u_{2i+1} — на вход $W_{\lambda-1}^i$ (нечетные выходы).

7.4.3 Параметры подканалов

Параметры Бхаттачарьи и пропускные способности подканалов: $Z_{m,i} = Z(W_m^i)$, $I_{m,i} = I(W_m^i)$.

$$Z_{m,2i} = 2Z_{m-1,i} - Z_{m-1,i}^2, \quad Z_{m,2i+1} = Z_{m-1,i}^2,$$

$$Z_{m,2i+1} \le Z_{m,2i} \le 2Z_{m-1,i} - Z_{m-1,i}^2,$$

$$I_{m,2i} + I_{m,2i+1} = 2I_{m-1,i}, \quad I_{m,2i} \le I_{m,2i+1}.$$

Для БСЭ первое равенство превращается в равенство: $Z_{m,2i} = 2Z_{m-1,i} - Z_{m-1,i}^2$.

Теорема 25. Поляризация. Для любого $\delta \in (0,1)$ при $m \to \infty$ доля подканалов с $I(W_m^i) \in (1-\delta, 1]$ стремится к $I(W_0^0) = I(W)$, а доля подканалов с $I(W_m^i) \in [0, \delta)$ — к $1 - I(W)$; доля подканалов с промежуточными пропускными способностями стремится к нулю.

7.5 Полярный код и алгоритм последовательного исключения

7.5.1 Определение кода

Передача по плохим (замороженным) подканалам предопределенных значений (например, нулей) дает линейный блоковый код. **Полярный код**: $c_0^{n-1} = u_0^{n-1} A_m$, $u_i = 0$ для $i \in F$, где F — множество номеров замороженных подканалов; это $(2^m, 2^m - |F|)$ код.

Сложность кодирования: в силу бабочечной структуры матрицы A_m (быстрое преобразование Уолша—Адамара)

$$u_0^{n-1} A_m = \Big(\Big(u_0^{\frac{n}{2}-1} + u_{\frac{n}{2}}^{n-1}\Big) A_{m-1}, u_{\frac{n}{2}}^{n-1} A_{m-1}\Big), \quad T(n) = 2T\Big(\frac{n}{2}\Big) + \frac{n}{2} = \frac{1}{2} n \log_2 n.$$

7.5.2 Алгоритм последовательного исключения (ПИ)

В терминах апостериорных вероятностей подканалов $W_m^i(u_0^i \mid y_0^{n-1}) = \frac{W_m^i(y_0^{n-1}, u_0^{i-1} \mid u_i) P\{u_i\}}{2W(y_0^{n-1})}$:

$$W_m^i(u_0^i \mid y_0^{n-1}) = \omega \sum_{u_{i+1}^{n-1}} \prod_{j=0}^{n-1} W\Big(\big((u_0^{n-1}A_m)_j\big) \mid y_j\Big),$$

где множитель ω не зависит от u_0^i и может быть опущен. Рекуррентные переходы:

$$W_\lambda^{2i}(u_0^{2i} \mid y_0^{2\lambda-1}) = \omega \sum_{u_{2i+1}} W_{\lambda-1}^i(u^{2i} + u_{2i+1} \mid y_{0,\text{even}}) W_{\lambda-1}^i(u_{2i+1} \mid y_{0,\text{odd}}),$$

$$W_\lambda^{2i+1}(u_0^{2i+1} \mid y_0^{2\lambda-1}) = \omega W_{\lambda-1}^i(u^{2i} + u_{2i+1} \mid y_{0,\text{even}}) W_{\lambda-1}^i(u_{2i+1} \mid y_{0,\text{odd}}).$$

Алгоритм 6 (последовательного исключения). Для $i = 0, 1, \ldots, 2^m - 1$:

$$\hat{u}_i = \begin{cases} 0 & i \in F \\ \arg\max_{u_i} W_m^i(\hat{u}_0^{i-1}, u_i \mid y_0^{n-1}) & i\neg \in F \end{cases}.$$

Если предыдущие решения были правильными ($\hat{u}_0^{i-1} = u_0^{i-1}$), решение по i-му символу оптимально; однако если на каком-то шаге допущена ошибка, алгоритм не сможет ее исправить — происходит распространение ошибок. Вероятность ошибки декодирования ПИ:

$$P_{[\text{SC}]} \le \sum_{i\neg\in F} Z_{m,i} \le 2^{-n^\beta}, \quad \beta < \frac{1}{2}.$$

7.5.3 Декодер на логарифмических отношениях правдоподобия

Логарифмическое отношение правдоподобия i-го подканала:

$$L_m^i(y_0^{n-1}, u_0^{i-1}) = \log_2 \frac{W_m^i(y_0^{n-1}, u_0^{i-1} \mid 0)}{W_m^i(y_0^{n-1}, u_0^{i-1} \mid 1)}.$$

Из рекуррентных формул для подканалов:

$$L_\lambda^{2i+1}(y_0^{2\lambda-1}, u_0^{2i}) = (-1)^{u_{2i}} L_{\lambda-1}^i\big(y_{0,\text{even}}, u_{0,\text{even}}^{2i-1} + u_{0,\text{odd}}^{2i-1}\big) + L_{\lambda-1}^i\big(y_{0,\text{odd}}, u_{0,\text{odd}}^{2i-1}\big).$$

Для нечетных подканалов положим

$$p_0^0 = W_{\lambda-1}^i(0 \mid y_{0,\text{even}}, \ldots), \quad p_0^1 = W_{\lambda-1}^i(0 \mid y_{0,\text{odd}}, \ldots), \quad p_1^0, p_1^1 \quad \text{аналогично для} \quad 1,$$

тогда $p_0 = p_0^0 p_0^1 + p_1^0 p_1^1$, $p_1 = p_1^0 p_0^1 + p_0^0 p_1^1$. Используя тождество $\tanh\big(\frac{1}{2}\log\big(\frac{p_0}{p_1}\big)\big) = \frac{p_0 - p_1}{p_0 + p_1}$, получаем

$$L_\lambda^{2i}(y_0^{2\lambda-1}, u_0^{2i-1}) = 2\tanh^{-1}\left(\tanh\left(L_{\lambda-1}^i \frac{y_{0,\text{even}}, u_{0,\text{even}}^{2i-1} + u_{0,\text{odd}}^{2i-1}}{2}\right) \cdot \tanh\left(L_{\lambda-1}^i \frac{y_{0,\text{odd}}, u_{0,\text{odd}}^{2i-1}}{2}\right)\right).$$

Решения принимаются по знаку ЛОПП:

$$\hat{u}_i = \begin{cases} 0 & i \in F \text{ или } L_m^i(y_0^{n-1}, \hat{u}_0^{i-1}) > 0 \\ 1 & i\neg \in F \text{ и } L_m^i(y_0^{n-1}, \hat{u}_0^{i-1}) \le 0 \end{cases}.$$

7.6 Построение полярных кодов

Замораживанию подлежат $2^m - k$ наименее надежных подканалов, например с наибольшими $Z_{m,i}$.

- **БСЭ**: рекуррентные формулы для $Z_{m,i}$ вычисляются за $O(n)$.
- **Общий случай**: выходной алфавит канала W_m^i имеет мощность $|\mathcal{Y}|^{n2^i}$, поэтому прямое построение W_m^i нереализуемо уже при небольших m. В методе Тала—Варди канал W_m^i аппроксимируется каналом с выходным алфавитом фиксированной мощности μ, чуть лучше или чуть хуже истинного; параметры $Z_{m,i}$ вычисляются со сложностью $O(n\mu^2 \log \mu)$ (I. Tal, A. Vardy, «How to Construct Polar Codes», IEEE Trans. Inf. Theory, 2013).

- **Гауссовская аппроксимация**: полярные коды линейны, для симметричных каналов вероятность ошибки не зависит от передаваемого слова, поэтому считаем переданным нулевое слово. При передаче по АБГШ $L_0^0(y_i) = 2\frac{y_i}{\sigma^2}$, откуда $M(L_0^0(y_i)) = \frac{2}{\sigma^2}$ и $D(L_0^0(y_i)) = \frac{4}{\sigma^2} = 2M(L_0^0(y_i))$. Предположим, что все ЛОПП нормальны:

$$L_\lambda^i \sim N\big(\mu_{\lambda,i}, 2\mu_{\lambda,i}\big), \quad 0 \le i < 2^\lambda, 0 \le \lambda \le m,$$

с рекуррентными средними

$$\mu_{\lambda,2i} = \Theta\big(\mu_{\lambda-1,i}\big) = \varphi^{-1}\Big(1 - \big(1 - \varphi\big(\mu_{\lambda-1,i}\big)\big)^2\Big), \quad \mu_{\lambda,2i+1} = 2\mu_{\lambda-1,i},$$

где

$$\varphi(x) = 1 - \frac{1}{\sqrt{4\pi x}} \int_{-\infty}^{\infty} \tanh\Big(\frac{u}{2}\Big) \exp\left(-\frac{(u-x)^2}{4x}\right) \mathrm{d}u.$$

Замораживаются символы с наименьшими $\mu_{m,i}$. Функция $\Theta(x) = \varphi^{-1}(x)$ хорошо приближается кусочно-квадратичной функцией (хорошо известная аппроксимация, удобная для программной реализации).

7.7 Минимальное расстояние

> **Теорема 26**. Минимальное расстояние d полярного кода длины $n = 2^m$ с замороженным множеством F равно

$$d = \min_{i^\neg \in F} 2^{\mathrm{wt}(i)} = \min_{i^\neg \in F} \mathrm{wt}\big(A_m^i\big),$$

где $\mathrm{wt}(i)$ — вес целого числа i (число ненулевых битов двоичной записи), A_m^i — i-я строка матрицы A_m (ее вес равен $2^{\mathrm{wt}(i)}$).

Следствие: минимальное расстояние полярных кодов мало (у хороших кодов растет лишь логарифмически или медленнее при фиксированной скорости), что портит их характеристики при больших длинах и низких вероятностях ошибки.

Коды Рида—Маллера как полярные коды. $\mathrm{RM}(r, m)$ — полярный код с замороженным множеством

$$F = \{i \mid 0 \le i < 2^m, \mathrm{wt}(i) < m - r\},$$

размерность $k = \sum_{i=m-r}^{m} \binom{m}{i} = \sum_{i=0}^{r} \binom{m}{i}$ и минимальным расстоянием $d = 2^{m-r}$ (в силу теоремы: наименьший $i^\neg \in F$ имеет вес $m - r$).

> **Пример 10**. Сравнение на длине 64 при скорости около $\frac{1}{2}$: расширенные коды БЧХ имеют заметно большее минимальное расстояние, чем полярные коды той же длины и размерности; коды Рида—Маллера занимают промежуточное положение. Это отражается на характеристиках: при декодировании по максимуму правдоподобия коды БЧХ существенно превосходят полярные.

7.8 Субоптимальность алгоритма ПИ

- При принятии решения о незамороженном символе не учитываются ограничения замораживания на будущие символы u_j, $j \in F$, $j > i$.
- Если принято неправильное решение $\hat{u}_i$, алгоритм ПИ не способен его исправить (распространение ошибок).
- Минимальное расстояние полярных кодов слишком мало.

Поэтому при умеренных длинах алгоритм ПИ уступает декодированию LDPC-кодов, и требуются улучшенные алгоритмы.

7.9 Списочное декодирование

7.9.1 Идея

Не будем принимать окончательное решение относительно u_i на фазе i. На каждой фазе будем рассматривать L векторов u_0^{i-1} (кандидатов), строить их возможные продолжения u_0^i и выбирать из них L наиболее вероятных. Векторы u_0^i задают пути в кодовом дереве.

7.9.2 Приближенные вероятности путей

$$\widehat{W}_\lambda^i(u_0^i \mid y_0^{2^\lambda-1}) = \max_{u_{2^\lambda-1}} \widehat{W}_\lambda^{2^\lambda-1}(u_0^{2^\lambda-1} \mid y_0^{2^\lambda-1}),$$

$$\widehat{W}_\lambda^{2i}(u_0^{2i} \mid y_0^{2^\lambda-1}) = \max_{u_{2i+1}} \widehat{W}_{\lambda-1}^i\left(u_{0,\text{even}}^{2i+1} + u_{0,\text{odd}}^{2i+1} \mid y_{0,\text{even}}\right)\widehat{W}_{\lambda-1}^i\left(u_{0,\text{odd}}^{2i+1} \mid y_{0,\text{odd}}\right),$$

$$\widehat{W}_\lambda^{2i+1}(u_0^{2i+1} \mid y_0^{2^\lambda-1}) = \widehat{W}_{\lambda-1}^i\left(u_{0,\text{even}}^{2i+1} + u_{0,\text{odd}}^{2i+1} \mid y_{0,\text{even}}\right)\widehat{W}_{\lambda-1}^i\left(u_{0,\text{odd}}^{2i+1} \mid y_{0,\text{odd}}\right),$$

то есть наибольшее вероятное продолжение пути ищется без учета замороженных символов $u_j = 0$, $j \in F$, $j > i$.

7.9.3 Вес пути и модифицированные ЛОПП

Вес пути:

$$R_\lambda^i(u_0^i, y_0^{2^\lambda-1}) = \log \widehat{W}_\lambda^i(u_0^i \mid y_0^{2^\lambda-1}).$$

Модифицированное ЛОПП:

$$S_\lambda^i(u_0^{i-1}, y_0^{2^\lambda-1}) = R_\lambda^i(u_0^{i-1}.0, y_0^{2^\lambda-1}) - R_\lambda^i(u_0^{i-1}.1, y_0^{2^\lambda-1}),$$

где точка обозначает конкатенацию. Если значение u_i совпадает с наиболее вероятным продолжением пути u_0^{i-1}, то $R_m^i(u_0^i, y_0^{2^m-1}) = R_m^{i-1}(u_0^{i-1}, y_0^{2^m-1})$; в противном случае вес уменьшается на $|S|$. Это записывается единообразно:

$$R_m^i(u_0^i, y_0^{n-1}) = R_m^{i-1}(u_0^{i-1}, y_0^{n-1}) + \tau(S_m^i(u_0^{i-1}, y_0^{n-1}), u_i),$$

$$\tau(S, v) = \begin{cases} 0 & \operatorname{sgn}(S) = (-1)^v \\ -|S| & \text{иначе} \end{cases}.$$

Рекуррентные формулы для модифицированных ЛОПП: пусть $J(c) = R_{\lambda-1}^i\left(\left(v_{0,\text{even}}^{2i-1} \oplus v_{0,\text{odd}}^{2i-1}\right).c \mid y_{0,\text{even}}\right)$, $K(c) = R_{\lambda-1}^i\left(v_{0,\text{odd}}^{2i-1}.c \mid y_{0,\text{odd}}\right)$, $a = J(0) - J(1)$, $b = K(0) - K(1)$. Тогда

$$S_\lambda^{2i}(u_0^{2i-1}, y_0^{2^\lambda-1}) = \max(J(0)+K(0), J(1)+K(1)) - \max(J(1)+K(0), J(0)+K(1)) = \max(a+b, 0) - \max(a, b),$$

$$S_\lambda^{2i+1}(u_0^{2i}, y_0^{2^\lambda-1}) = J(u_{2i}) + K(0) - J(u_{2i}+1) - K(1) = (-1)^{u_{2i}}a + b.$$

7.9.4 Декодер min-sum

Замена точной формулы приближением дает **декодер минимум-суммы**:

$$S_\lambda^{2i}(u_0^{2i-1}, y_0^{2^\lambda-1}) = Q(a, b) = \operatorname{sgn}(a)\operatorname{sgn}(b)\min(|a|, |b|), \quad S_\lambda^{2i+1}(u_0^{2i}, y_0^{2^\lambda-1}) = P(u_{2i}, a, b) = (-1)^{u_{2i}}a + b.$$

Алгоритм ПИ на модифицированных ЛОПП принимает решения по знаку S_m^i (с учетом замороженных символов).

7.9.5 Списочный алгоритм Тала—Варди

Пусть $V[l]$ — l-й рассматриваемый вектор u_0^{i-1}, $0 \le l < L$, и $R[l] = R_m^{i-1}(V[l], y_0^{n-1})$.

> **Алгоритм 7** (списочного декодирования Тала—Варди). Для $i = 0, 1, ..., 2^m - 1$:
>
> 1. Вычислить $s_l = S_m^i(V[l], y_0^{n-1})$, $0 \le l < L$.

2. Если $i \in F$: дописать к $V[l]$ значение замороженного символа и пересчитать $R[l] \downarrow R[l] + \tau(s_l, (V[l])_i)$.
3. Если $i\neg \in F$: выбрать L пар (l, v) с наибольшими значениями $r_{lv} = R[l] + \tau(s_l, v)$, $v \in \{0, 1\}$, $0 \le l < L$; для выбранных пар построить продолженные пути $V'[l] \downarrow V[l].v$, $R'[l] \downarrow r_{lv}$ и положить $V \downarrow V'$, $R \downarrow R'$.

Из полученного списка выбрать наилучший путь: с наибольшим $R[l]$ или удовлетворяющий дополнительным условиям (например, проверке CRC).

Реализация. Полярный код получается рекурсивным применением конструкции Плоткина: $c = (c_1 + c_0, c_1)$. Используются массивы частичных сумм C_λ размерности $2^{m-\lambda}$: в C_0 формируется кодовое слово c, в C_1 — слово c_0 компонентного кода, и т.д.; по готовности кодовых слов применяется преобразование Плоткина, и массивы высвобождаются. Массив ЛОПП S_λ размерности $2^{m-\lambda}$: S_0 содержит ЛОПП принятого вектора. На i-й итерации алгоритма ПИ обновляются массивы $S_{m-t}, \ldots, S_m$, где t — максимальная степень двойки, делящая i ($t < m$):

$$S_{m-t}[j] = P(C_{m-t-1}[j], S_{m-t-1}[j], S_{m-t-1}[j + 2^t]), \quad 0 \le j < 2^{m-t},$$

$$S_{\lambda[j]} = Q\big(S_{\lambda-1}[j], S_{\lambda-1}[j + 2^{m-\lambda}]\big), \quad m - t < \lambda \le m.$$

При $i = 0$ считается $t = m$, первая формула не используется. Многие массивы частичных сумм и ЛОПП совпадают у различных путей списка, поэтому копирование данных может быть полностью исключено (Tal, Vardy, «List decoding of polar codes», 2015).

Сложность списочного декодера — $O(Ln \log n)$. Моделирование кода $(2048, 1024)$: списочное декодирование ($L = 2, 4, 8$) последовательно улучшает FER по сравнению с алгоритмом ПИ, но из-за низкого минимального расстояния полярные коды все еще уступают LDPC с сопоставимой сложностью.

7.10 Полярные коды с CRC

Циклический код с проверкой избыточности (CRC) — циклический код, обнаруживающий ошибки. Систематическое кодирование циклического кода длины n с порождающим многочленом $g(x)$:

$$c(x) = a(x)x^{n-k} + b(x), \quad b(x) \equiv a(x)x^{n-k} \pmod{g(x)}.$$

К данным добавляются проверочные символы $b(x)$ **до** кодирования полярным кодом, а из списка, формируемого списочным декодером Тала—Варди, удаляются кодовые слова с неправильным значением контрольной суммы. Полярные коды с CRC ($L = 32$ и даже $L = 512$) при длинах порядка 2048 превосходят LDPC-декодеры с 20–200 итерациями в области высоких ОСШ.

7.11 Полярные подкоды

7.11.1 Динамически замороженные символы

Классические полярные коды: замороженные символы $u_i = 0$, $i \in F$. Обобщение — **динамическое замораживание**:

$$u_i = \sum_{j=0}^{i-1} V_{s_i, j} u_j \quad (i \in F), \quad \text{в матричной форме} \quad uV^T = 0,$$

где V — матрица ограничений (аналог проверочной матрицы), а s_i — номер ее строки, отвечающей замороженному символу i (для классических полярных кодов V — единичная матрица и $u_i = 0$).

Алгоритм ПИ обобщается: при $i \in F$ значение $\hat{u}_i$ не выбирается, а вычисляется как $\sum_{j<i} V_{s_i, j} \hat{u}_j$. Вероятность ошибки $P_{[\mathrm{SC}]} \le \sum_{i\neg \in F} Z_{m,i}$ совпадает с таковой для классического полярного кода с тем же F; списочный алгоритм Тала—Варди обобщается непосредственно. Возникает вопрос: как выбрать матрицу ограничений V?

7.11.2 Представление произвольного линейного кода

Рассмотрим $(n = 2^m, k)$ линейный блоковый код с проверочной матрицей H. Положим $V' = HA_m^T$ и применим к V' элементарные операции над строками (их композиция — некоторая обратимая матрица Q) так, чтобы последние ненулевые элементы строк расположились в различных столбцах j_i; полученная матрица $V = QHA_m^T$ задает ограничения динамического замораживания. Таким образом, алгоритм ПИ (и его списочные аналоги) применим к декодированию **произвольных** линейных блоковых кодов.

- В общем случае вероятность ошибки декодирования методом ПИ много больше вероятности ошибки декодирования по максимуму правдоподобия.
- Перестановка столбцов проверочной матрицы (переход к эквивалентному коду) может радикально изменить множество F и $P_{[\mathrm{SC}]}$.
- Для расширенных примитивных кодов БЧХ в узком смысле $P_{[\mathrm{SC}]}$ достаточно мала, поэтому списочный алгоритм с небольшим списком обеспечивает хорошую корректирующую способность.

Пример 11. Коды длины 64, размерности 30 и 36: при $L = 1$ (чистый ПИ) полярный код Арикана минимизирует вероятность ошибки $P_{[\mathrm{SC}]}$ и выигрывает у кода БЧХ; при $L = 4$ достигается декодирование полярных кодов по максимуму правдоподобия; для кодов БЧХ почти-МП достигается лишь при $L = 256$, но благодаря большему минимальному расстоянию коды БЧХ обеспечивают лучшую корректирующую способность.

7.11.3 Полярные подкоды в узком смысле

Определение 18. Рассмотрим канал $W(y|c)$ и $(n = 2^m, k', d)$ код C' над $\mathrm{GF}(2)$, называемый **протокодом**, и множество номеров его замороженных символов F'. $(n, k, \geq d)$ **полярным подкодом в узком смысле** кода C' называется множество векторов $c_0^{n-1} = u_0^{n-1}A_m$, где u_0^{n-1} одновременно удовлетворяет ограничениям замораживания кода C' и дополнительным ограничениям $u_s = 0$ для $k' - k$ номеров $s^{\neg} \in F'$ с наибольшими вероятностями ошибки подканалов $P_{m,s}$ (вероятность ошибки символа s при декодировании методом ПИ).

Схема построения: выбрать протокод с достаточно большим минимальным расстоянием (хороший выбор — расширенные примитивные коды БЧХ в узком смысле) и удалить из него кодовые слова, препятствующие эффективному декодированию методом ПИ.

7.11.4 Полярные подкоды в широком смысле

Определение 19. Полярным подкодом в широком смысле называется множество векторов $c = xWA_m$, $x \in \mathrm{GF}(2)^k$, где матрица W (матрица прекодирования) имеет нулевые столбцы в позициях, соответствующих r наименее надежным подканалам W_m^j.

Схема построения: взять полярный код $(n = 2^m, n - r)$, эффективно декодируемый методом ПИ, и удалить из него кодовые слова, ответственные за высокую вероятность ошибки декодирования по максимуму правдоподобия. Связь матриц: $c_0^{n-1} = u_0^{n-1}A_m$, $u_0^{n-1}V^T = 0$ эквивалентно $u_0^{n-1} = xW$, $WV^T = 0$; здесь V — матрица ограничений (аналог проверочной матрицы), W — матрица прекодирования (аналог порождающей матрицы).

Моделирование: полярные подкоды (в узком и широком смысле) при $L = 32$ и $L = 512$ превосходят полярные коды с CRC и CCSDS LDPC (20–200 итераций) по FER, при этом среднее число операций декодирования сопоставимо или меньше.

7.12 Последовательное декодирование

7.12.1 Недостатки списочного декодера

- Сложность $O(Ln\log n)$, где L — размер списка; из L найденных кодовых слов не более чем одно может быть правильным, поэтому большая часть вычислений бесполезна.
- Размер списка L, требуемый для декодирования почти по максимуму правдоподобия, растет экспоненциально с длиной кода.

7.12.2 Последовательный (стековый) алгоритм

Будем одновременно рассматривать пути v_0^i **различной длины**, храня их в приоритетной очереди вместе с весами $Q(v_0^i, y_0^{n-1})$.

Алгоритм 8 (последовательного декодирования).

1. Поместить в приоритетную очередь путь нулевой длины с нулевым весом; счетчики $\varphi_i = 0$, $0 \le i < n$.
2. Извлечь из очереди путь v_0^{i-1} с наибольшим весом.
3. Если $i = n$: возвратить v_0^{n-1} и завершить работу.
4. $\varphi_i \downarrow \varphi_i + 1$. Если $\varphi_i \ge L$: удалить из очереди все пути длины $\le i$.
5. Построить все допустимые продолжения v_0^i, вычислить их веса $Q(v_0^i, y_0^{n-1})$ и поместить в очередь.
6. Перейти к шагу 2.

Весовая функция должна обеспечивать корректное сравнение путей различной длины.

7.12.3 Весовая функция

Используются модифицированные ЛОПП декодера минимум-суммы ($Q(a,b)$ и P из предыдущего раздела). Если передавалось нулевое слово и декодер идет по правильному пути ($v_0^{i-1} = 0$), то величины $S_\lambda^i\left(0, y_0^{2\lambda-1}\right)$ имеют функции распределения $F_\lambda^i(x)$ с рекуррентными формулами

$$F_\lambda^{2i}(x) = \begin{cases} 2F_{\lambda-1}^i(x)\left(1 - F_{\lambda-1}^i(-x)\right) & x < 0 \\ 2F_{\lambda-1}^i(x) - \left(F_{\lambda-1}^i(-x)\right)^2 - \left(F_{\lambda-1}^i(x)\right)^2 & x \ge 0 \end{cases},$$

$$F_\lambda^{2i+1}(x) = \int_{-\infty}^{\infty} F_{\lambda-1}^i(x-y)\,\mathrm{d}F_{\lambda-1}^i(y).$$

Логарифм вероятности наиболее вероятного продолжения пути (ЛВНВП):

$$R_m^i(v_0^i, y_0^{n-1}) = R_m^{i-1}(v_0^{i-1}, y_0^{n-1}) + \tau(S_m^i(v_0^{i-1}, y_0^{n-1}), v_i).$$

Весовая функция последовательного декодирования:

$$Q(v_0^i, y_0^{n-1}) = R_m^i(v_0^i, y_0^{n-1}) - M \cdot M_{Y^{n-1}}(R_m^i(0, Y^{n-1})),$$

где $M > 0$ — подбираемый множитель (параметр алгоритма), а вычитаемое корректирует вес с учетом того, сколько в среднем «набирает» правильный путь такой же длины.

где $M_{Y^{n-1}}(R_m^i(0, Y^{n-1}))$ — математическое ожидание ЛВНВП при условии, что декодер идет по пути v_0^i, соответствующему информационному вектору u_0^i передатчика. Оно вычисляется по функциям распределения:

$$_{-1}(R_m^i(0, y^{n-1})) = \sum_{j=0}^{i} M(\tau(S_m^j(0, y^{n-1}), 0)) = \sum_{j=0}^{i} \int_{-\infty}^{\infty} \tau(x,0) f_m^j(x)\,\mathrm{d}x = \sum_{j=0}^{i} \left(xF_m^j(x)\ |_0^{-\infty} - \int_{-\infty}^{0} F_m^j(x)\,\mathrm{d}x \right) = -\sum_{j=0}^{i} \int_{-\infty}^{0} F_m^j(x$$

Пример 12. Сравнение весовых функций для полярного подкода $(1024, 512)$: использование истинной вероятности пути W_m^i (метрика M_1), веса R_m^i (метрика M_2) и скорректированного веса Q (метрика M_3) показывает, что нормировка на средний ЛВНВП (M_3) обеспечивает наилучшее качество декодирования и наименьшее среднее число итераций при обоих размерах списка.

7.13 Выводы

- Полярные коды — первые коды с доказанным достижением предела Шеннона при простой конструкции, кодировании ($O(n \log n)$) и декодировании ($O(n \log n)$ для алгоритма ПИ).
- Корректирующая способность полярных кодов при декодировании методом ПИ неудовлетворительна из-за малого минимального расстояния; ее улучшают списочный и последовательный алгоритмы и метод распространения доверия.
- Полярные подкоды и полярные коды с CRC на длинах до нескольких тысяч имеют лучшую корректирующую способность и меньшую сложность декодирования (при последовательном декодировании) по сравнению с LDPC- и турбо-кодами.
- Алгоритм ПИ и его аналоги плохо распараллеливаются и имеют большую задержку.

8 Циклические коды

8.1 Алгебраические сведения

8.1.1 Группы

Группа $G = (G, \cdot)$ — множество G с бинарной операцией $\cdot$, которая ассоциативна, имеет нейтральный элемент $1 \in G$ ($1 \cdot x = x \cdot 1 = x$) и для каждого $x \in G$ обратный элемент y: $x \cdot y = y \cdot x = 1$. Если операция коммутативна ($a \cdot b = b \cdot a$), группа называется **абелевой**; далее все группы коммутативны. В аддитивной записи операция обозначается $+$, нейтральный элемент 0. **Подгруппа** $H = (H, \cdot)$ — подмножество $H \subset G$, замкнутое относительно операции, само являющееся группой. Группа называется **циклической**, если в ней есть образующий элемент a: всякий $x \in G$ есть степень a^n для некоторого целого n. **Порядок** конечной группы — число ее элементов $|G|$; **порядок элемента** a — наименьшее натуральное n, для которого $a^n = 1$. Порядок образующего конечной циклической группы равен порядку самой группы.

Теорема 27. Пусть g и h — элементы конечной группы порядков r и s соответственно, причем $\gcd(r, s) = 1$ (взаимно просты). Тогда элемент gh имеет порядок rs.

Доказательство. $(gh)^{rs} = 1$, поэтому порядок p элемента gh делит rs. Пусть $p \mid (rs)$ и $(gh)^p = 1$. Тогда $(gh)^{pr} = h^{pr} = 1$, следовательно $s \mid (pr)$, откуда $s \mid p$ (так как r и s взаимно просты). Аналогично $r \mid p$. Значит, rs делит p, то есть $p = rs$. □

8.1.2 Смежные классы и теорема Лагранжа

Смежный класс подгруппы H по элементу $a \in G$: $aH = \{a \cdot h \mid h \in H\}$. Отношение $a \sim b \Leftrightarrow aH = bH$ является отношением эквивалентности; его классы эквивалентности — смежные классы.

Лемма 4. Всякий смежный класс aH равномощен H.

Доказательство. Отображение $\varphi_a : H \to aH$, $\varphi_a(h) = ah$, сюръективно по определению aH и инъективно: из $ah_1 = ah_2$ в группе следует $h_1 = h_2$. Значит, φ_a — биекция и $|aH| = |H|$. □

Теорема 28. Теорема Лагранжа. Порядок конечной группы G делится на порядок любой ее подгруппы H.

Доказательство. Смежные классы aH образуют разбиение G на $|G|/|H|$ равномощных подмножеств по $|H|$ элементов. □

8.1.3 Кольца и идеалы

Поле — множество F с операциями сложения, вычитания, умножения и деления, удовлетворяющее аксиомам ассоциативности и коммутативности обеих операций, дистрибутивности, существования нуля 0, единицы $1 \neq 0$, противоположного $-a$ для каждого a и обратного $a^{-1} = 1/a$ для каждого $a \neq 0$. Конечное поле с q элементами называется **полем Галуа** и обозначается $\mathrm{GF}(q)$. Поле — область целостности: из $ab = 0$ при $a, b \neq 0$ следовало бы $0 = abb^{-1}a^{-1} = 1$.

Определение 20. Подмножество I кольца R (кольцо — множество с ассоциативными $+$ и $\cdot$, дистрибутивностью, нулем и единицей; далее все кольца коммутативны) называется **идеалом**, если $(I, \{+\})$ — подгруппа $(R, \{+\})$ и для всех $r \in R$, $x \in I$: $xr \in I$ (замкнутость относительно умножения на любые элементы кольца). **Идеалом, порождаемым** конечным множеством $A \not\subset R$, называется наименьший идеал, содержащий A:

$$\langle A \rangle = \left\{ r_1 a_1 + \ldots + r_{|A|} a_{|A|} \mid a_i \in A, r_i \in R \right\}.$$

Идеал **главный**, если он порождается одним элементом: $I = \langle a \rangle$. **Максимальный** идеал: $I \neq R$, и всякий идеал J, содержащий I, равен I или R.

Пример 13. Четные числа — главный идеал $\langle 2 \rangle$ в кольце целых чисел $\mathbb{Z}$; многочлены с вещественными коэффициентами, делящиеся на $x^2 + 1$, — идеал в кольце многочленов $\mathbb{R}[x]$; $\langle 7 \rangle$ — максимальный идеал в $\mathbb{Z}$ (кратные 7 нельзя получить иначе, чем умножением 7 на целые числа); $\{0\}$ и R — идеалы любого кольца.

Область целостности, в которой все идеалы главные, называется **кольцом главных идеалов** (КГИ). Если $\langle f_0, f_1, \ldots, f_n \rangle$ — максимальный идеал, то добавление в систему образующих любого f_0 вне идеала дает все кольцо: $\langle f_0, f_1, \ldots, f_n \rangle = R$.

8.1.4 Факторкольца

Пусть R — кольцо, $I \subset R$ — идеал. Отношение $a \sim b \Leftrightarrow b - a \in I$ — конгруэнтность (эквивалентность, согласованная с операциями). Класс эквивалентности элемента a: $[a] = a + I = \{a + r \mid r \in I\}$. Множество классов — **факторкольцо** (кольцо вычетов) R/I с операциями

$$(a + I) + (b + I) = (a + b) + I, \quad (a + I)(b + I) = ab + I;$$

его нуль — $I = 0 + I$, единица — $1 + I$. Если R — КГИ и $I = aR$, факторкольцо обозначается R/aR.

Теорема 29. Если I — максимальный идеал кольца R, то факторкольцо R/I — поле.

Доказательство. Надо показать, что всякий ненулевой $a + I \in R/I$ (то есть $a\neg \in I$) обратим. Рассмотрим $J = \{ax + m \mid x \in R, m \in I\}$: это идеал, так как суммы и произведения на элементы кольца сохраняют вид $ax + m$; при $x = 0$ получаются все элементы I, поэтому $I \subset J$; кроме того $a \in J$, $a\neg \in I$. В силу максимальности $J = R$, поэтому $1 \in J$: существуют $b \in R$, $m \in I$ с $1 = ab + m$, откуда $ab - 1 \in I$ и $(a + I)(b + I) = 1 + I$ — обратный элемент найден. □

Теорема 30. Пусть R — КГИ и p — его неприводимый элемент (неразложимый в произведение необратимых). Тогда R/pR — поле.

Доказательство. Идеал $I = \langle p \rangle$ максимален: пусть $I \subset J = \langle q \rangle \not\subset R$. Тогда $p = qs$; так как p неприводим, s обратим, откуда $q = s^{-1}p$ и $J \subset I$, то есть $I = J$. □

Пример 14.

- $\mathbb{Z}$ — КГИ, поэтому $\mathbb{Z}/p = \mathrm{GF}(p)$ — поле для простого p (арифметика по модулю p).
- $x^2 + 1$ неприводим над $\mathbb{R}$, поэтому $\mathbb{R}[x]/(x^2+1)$ — поле, изоморфное полю комплексных чисел $\mathbb{C}$: произведение классов $(a+bx)(c+dx) \equiv (ac-bd) + (ad+bc)x \bmod (x^2+1)$ совпадает с произведением комплексных чисел $a+bi$ и $c+di$, где класс $[x]$ играет роль мнимой единицы i, и $i \cdot i = x^2 \equiv -1$.
- $x^2 + x + 1$ неприводим над $\mathrm{GF}(2)$, поэтому $\mathrm{GF}(2)[x]/(x^2+x+1) = \mathrm{GF}(2^2)$ — поле из 4 элементов: классы вычетов $0, 1, x, x+1$; все операции — по модулю x^2+x+1, например $x(x+1) \equiv 1$.

8.1.5 Конечные поля

Характеристика конечного поля — наименьшее натуральное p, для которого сумма p единиц равна нулю: $\underbrace{1 + \ldots + 1}_{p \text{ раз}} = 0$. Характеристика — простое число: иначе $p = st$, и $0 = \left(\underbrace{1 + \ldots + 1}_{s \text{ раз}}\right) \cdot \left(\underbrace{1 + \ldots + 1}_{t \text{ раз}}\right)$ — противоречие с отсутствием делителей нуля.

Теорема 31. Поле F из q элементов имеет $q = p^m$ элементов, где p — простое (характеристика), $m \in \mathbb{N}$.

Доказательство. Элемент 1 порождает аддитивную циклическую подгруппу порядка p, поэтому p делит q и внутри F лежит копия $\mathrm{GF}(p)$. Выберем максимальное по включению множество линейно независимых над $\mathrm{GF}(p)$ элементов $\alpha_1, \ldots, \alpha_m$; тогда всякий $\alpha_0 \in F$ представим в виде $\sum_i x_i \alpha_i$ с коэффициентами $x_i \in \mathrm{GF}(p)$ (иначе множество расширяется), а различные наборы коэффициентов дают различные элементы. Значит, $|F| = p^m$, и $\mathrm{GF}(p^m)$ — m-мерное линейное пространство над $\mathrm{GF}(p)$. □

При $m > 1$ поле $\mathrm{GF}(p^m)$ называется **расширенным**, m — степень расширения. Поле $\mathrm{GF}(p^m)$ не имеет ничего общего с кольцом вычетов $\mathbb{Z}_{p^m}$, которое полем не является.

Теорема 32. Ненулевые элементы $\mathrm{GF}(q)$ образуют циклическую группу по умножению порядка $q-1$.

Доказательство. $F \setminus \{0\}$ — конечная группа по умножению. Выберем в ней элемент α наибольшего порядка r. Пусть l — порядок другого элемента β, и π — простое число, делящее l: запишем $r = \pi^a r'$, $l = \pi^b l'$ с $\gcd(r', \pi) = \gcd(l', \pi) = 1$. Тогда α^{π^a} имеет порядок r', $\beta^{l'}$ — порядок π^b, а по предыдущей теореме элемент $\gamma = \alpha^{\pi^a}\beta^{l'}$ имеет порядок $\pi^b r'$. В силу максимальности r: $\pi^b r' \le \pi^a r'$, то есть $b \le a$ для всех простых π, откуда l делит r. Значит, все $q-1$ ненулевых элементов — корни многочлена $x^r - 1$, поэтому $r \ge q-1$; но порядок элемента не превосходит порядка группы, значит $r = q-1$. □

Следствия:

- для всякого ненулевого β: $\beta^{q-1} = 1$; все элементы поля — корни уравнения $x^q - x = 0$;
- порядок любого ненулевого элемента делит $q-1$;
- в поле характеристики p справедливо $(x+y)^p = x^p + y^p$: в биномиальном разложении все промежуточные коэффициенты $\binom{p}{i}$ кратны p и равны нулю.

8.1.6 Минимальные многочлены

Определение 21. **Минимальный многочлен** элемента $\beta \in \mathrm{GF}(p^m)$ над $\mathrm{GF}(p)$ — приведенный (со старшим коэффициентом 1) многочлен $M_\beta(x) \in \mathrm{GF}(p)[x]$ наименьшей степени, для которого $M_\beta(\beta) = 0$.

Теорема 33. Свойства минимальных многочленов:
- $M_\beta(x)$ неприводим над $\mathrm{GF}(p)$;
- если $f(x) \in \mathrm{GF}(p)[x]$ и $f(\beta) = 0$, то $M_\beta(x)$ делит $f(x)$;
- $M_\beta(x)$ делит $x^{p^m} - x$ для всех $\beta \in \mathrm{GF}(p^m)$;
- $\deg M_\beta(x) \leq m$.

Доказательство. Неприводимость: если $M_\beta = M_1 M_2$ с меньшими степенями, то $M_1(\beta) = 0$ или $M_2(\beta) = 0$ — противоречие с минимальностью. Делимость f: разделим с остатком $f = qM_\beta + r$, $\deg r < \deg M_\beta$; подстановка β дает $r(\beta) = 0$, значит $r = 0$. Третий пункт следует из $\beta^{p^m} = \beta$. Оценка степени: $\mathrm{GF}(p^m)$ — m-мерное пространство над $\mathrm{GF}(p)$, поэтому элементы $1, \beta, \ldots, \beta^m$ линейно зависимы, и существует ненулевой многочлен степени не выше m с корнем β. □

Теорема 34. Минимальный многочлен не меняется при возведении элемента в степень p: $M_\beta(x) = M_{\beta^p}(x)$.

Доказательство. Если $0 = M_\beta(\beta) = \sum_{i=0}^{d} M_{\beta,i}\beta^i$ с коэффициентами $M_{\beta,i} \in \mathrm{GF}(p)$, то возведение в p-ю степень дает $0 = \sum_i M_{\beta,i}^p \beta^{pi} = \sum_i M_{\beta,i}\beta^{pi} = M_\beta(\beta^p)$, так как $M_{\beta,i}^p = M_{\beta,i}$ для $M_{\beta,i} \in \mathrm{GF}(p)$. Значит, M_{β^p} делит M_β; в силу неприводимости и единичности старших коэффициентов они равны. □

Сопряженные элементы $\beta, \beta^p, \beta^{p^2}, \ldots$ — корни одного минимального многочлена. Число сопряженных элементов:

Теорема 35. $M_\beta(x) = \prod_{i=0}^{m_\beta - 1}\left(x - \beta^{p^i}\right)$, где m_β — наименьшее положительное число, для которого $\beta^{p^{m_\beta}} = \beta$ (степень M_β).

Доказательство. Многочлен $\prod_i \left(x - \beta^{p^i}\right) = \sum_i a_i x^i$ имеет коэффициенты в $\mathrm{GF}(p)$: возведение его в p-ю степень переставляет сомножители циклически ($x \to x^p$ переводит корни β^{p^i} в $\beta^{p^{i+1}}$), поэтому $\sum_i a_i^p x^{pi} = \left(\sum_i a_i x^i\right)^p = \sum_i a_i x^{pi}$, то есть $a_i^p = a_i$. Многочлен имеет корнем β (как и всякий минимальный), поэтому его степень равна степени M_β — минимально возможной. □

Теорема 36. Если α — примитивный элемент $\mathrm{GF}(p^m)$ (образующий мультипликативной группы), то $\deg M_\alpha(x) = m$.

Доказательство. Из $M_\alpha(x) = \sum_{i=0}^{d} \pi_i x^i$, $d \leq m$, следует $\alpha^d = -\sum_{i=0}^{d-1} \pi_i \alpha^i$; домножая на степени α, получаем, что всякий α^j, а значит, и всякий ненулевой элемент поля выражается линейно через $1, \alpha, \ldots, \alpha^{d-1}$. Так как ненулевых элементов $p^m - 1$, то $p^d \geq p^m$, то есть $d \geq m$. □

Минимальный многочлен примитивного элемента называется **примитивным многочленом**; не всякий неприводимый многочлен примитивен. Всякий элемент поля представим как $\beta = \sum_{i=0}^{m-1} b_{\beta,i}\alpha^i$ с $b_{\beta,i} \in \mathrm{GF}(p)$ (координаты в базисе степеней примитивного элемента).

Теорема 37. Все конечные поля из p^m элементов изоморфны.

Доказательство. Пусть F и G — поля из p^m элементов, α — примитивный элемент F с минимальным многочленом $\pi(x)$. Так как $\pi(x)$ делит $x^{p^m} - x$, а все элементы G — корни $x^{p^m} - x$, найдется $\beta \in G$ с $\pi(\beta) = 0$; соответствие между многочленами от α и от β степени не более $m - 1$ задает изоморфизм полей. □

Пример 15. Поле $\mathrm{GF}(2^3)$ можно задать многочленом $x^3 + x + 1$ (примитивный элемент α, $\alpha^3 + \alpha + 1 = 0$) или многочленом $x^3 + x^2 + 1$ (примитивный элемент γ). Элементы поля — двоичные тройки: $\alpha^3 = \alpha + 1$, $\alpha^4 = \alpha^2 + \alpha$, $\alpha^5 = \alpha^2 + \alpha + 1$, $\alpha^6 = \alpha^2 + 1$ и аналогично для γ. Подстановка показывает $\left(\gamma^3\right)^3 + \gamma^3 + 1 = 0$, то есть γ^3 — корень $x^3 + x + 1$; соответствие $\alpha \downarrow \gamma^3$ задает изоморфизм двух представлений поля.

8.2 Циклические коды

8.2.1 Определение

Определение 22. Линейный блоковый код C длины n над полем F называется **циклическим**, если всякий циклический сдвиг кодового слова является кодовым словом:

$$(c_0, c_1, ..., c_{n-1}) \in C \Longrightarrow (c_{n-1}, c_0, c_1, ..., c_{n-2}) \in C.$$

Многочленное представление: вектору $(c_0, ..., c_{n-1})$ сопоставляется многочлен $c(x) = \sum_{i=0}^{n-1} c_i x^i$ (коэффициенты — символы кодового слова). Циклический сдвиг на одну позицию эквивалентен умножению на x по модулю $x^n - 1$:

$$xc(x) = c_0 x + c_1 x^2 + ... + c_{n-2} x^{n-1} + c_{n-1} x^n \equiv c_{n-1} + c_0 x + ... + c_{n-2} x^{n-1} \pmod{x^n - 1},$$

так как $x^n \equiv 1$. Далее вектор и соответствующий многочлен отождествляются.

Теорема 38. Подмножество C кольца вычетов $F[x]/(x^n - 1)$ образует циклический код тогда и только тогда, когда

1. C — группа по сложению;
2. для любых $c(x) \in C$ и $a(x) \in F[x]/(x^n - 1)$ произведение $a(x)c(x) \bmod (x^n - 1)$ лежит в C, то есть C — идеал кольца $F[x]/(x^n - 1)$.

Доказательство. Пусть свойства выполнены: замкнутость относительно умножения на скаляр делает C линейным пространством; умножение на x^i (то есть i циклических сдвигов) не выводит за пределы C — код циклический. Обратно: циклический линейный код — группа по сложению, а умножение на произвольный многочлен — взвешенная сумма его циклических сдвигов. □

8.2.2 Порождающий и проверочный многочлены

Порождающий многочлен (ПМ) циклического кода — ненулевой кодовый многочлен $g(x)$ наименьшей степени со старшим коэффициентом 1.

Теорема 39.

- Всякое кодовое слово $c(x)$ делится на $g(x)$: разделив с остатком $c = ag + r$, $\deg r < \deg g$, получаем $r \equiv c - ag \in C$, что противоречит минимальности степени g.
- Порождающий многочлен единственен.

- Циклический код длины n с ПМ $g(x)$ существует тогда и только тогда, когда $g(x)$ делит $x^n - 1$.
- Размерность кода $k = n - \deg g(x)$.

Доказательство. Существование $\Longrightarrow$: если код с ПМ $g(x)$ существует, запишем $x^n - 1 = a(x)g(x) + r(x)$, $\deg r < \deg g$. Многочлен $b(x) \equiv a(x)g(x) \equiv -r(x) \bmod (x^n - 1)$ лежит в C (произведение кодового многочлена на произвольный по свойству идеала), поэтому и $r(x) \in C$, и $r(x) = 0$, то есть $g \mid x^n - 1$. $\Longleftarrow$: множество кратных $g(x)$ по модулю $x^n - 1$ — идеал, то есть циклический код. Единственность: два ПМ делят друг друга. Пусть $x^n - 1 = h(x)g(x)$; многочлен $h(x)$ — **проверочный**: для всякого кодового $c(x) = a(x)g(x)$ имеем $c(x)h(x) = a(x)(x^n - 1) \equiv 0$. Размерность: кодовые слова — все кратные $g(x)$ степени меньше n, то есть $a(x)g(x)$ с $\deg a < n - \deg g$; их $n - \deg g = k$, и $\deg h = k$. □

8.2.3 Кодирование

- **Несистематическое**: $c(x) = a(x)g(x)$; порождающая матрица — сдвиги строки коэффициентов $(g_0, g_1, \ldots, g_{n-k})$.
- **Систематическое** (информационные символы $a_0, \ldots, a_{k-1}$ занимают позиции $x^{n-k}, \ldots, x^{n-1}$):

$$c(x) = x^{n-k}a(x) - r(x), \quad \text{где остаток } r(x) \equiv x^{n-k}a(x) \pmod{g(x)}, \quad \deg r(x) < \deg g(x).$$

Каждому методу соответствует своя порождающая матрица; все они выражаются друг через друга как $G' = QG$ с обратимой матрицей Q, поэтому метод кодирования не влияет на корректирующую способность кода.

8.2.4 Корни порождающего многочлена

Разложим $x^n - 1 = \prod_{i=0}^{l-1} f_i(x)$ на неприводимые над $\mathrm{GF}(q)$ многочлены (однозначно с точностью до порядка сомножителей). ПМ любого циклического кода есть произведение части сомножителей: $g(x) = \prod_{i \in J} f_i(x)$, $J \subset \{0, \ldots, l-1\}$; при различных f_i существует $2^l - 2$ нетривиальных циклических кодов длины n. Циклические коды над $\mathrm{GF}(q)$ длины $n = q^m - 1$ называются **примитивными**.

Теорема 40. Пусть $\beta_1, \ldots, \beta_r \in \mathrm{GF}(q^m)$ — корни порождающего многочлена $g(x)$ циклического кода C над $\mathrm{GF}(q)$. Многочлен $c(x) \in \mathrm{GF}(q)[x]$ является кодовым тогда и только тогда, когда $c(\beta_1) = c(\beta_2) = \ldots = c(\beta_r) = 0$.

Доказательство. Если $c(x) = a(x)g(x)$, то все корни g — корни c. Обратно: $c(\beta_i) = 0$ влечет $M_{\beta_i}(x) \mid c(x)$ (свойство минимального многочлена β_i); если это выполнено для всех i, то $g(x)$ — произведение минимальных многочленов своих корней — делит $c(x)$, то есть $c(x) \in C$. □

8.2.5 Проверочная матрица над расширенным полем

Равенства $c(\beta_i) = \sum_{j=0}^{n-1} c_j \beta_i^j = 0$, $i = 1, \ldots, r$, — это система линейных уравнений на кодовые символы, то есть проверочная матрица над $\mathrm{GF}(q^m)$:

$$H = \begin{bmatrix} \beta_1^0 & \beta_1^1 & \cdots & \beta_1^{n-1} \\ \beta_2^0 & \beta_2^1 & \cdots & \beta_2^{n-1} \\ \vdots & \vdots & \vdots & \vdots \\ \beta_r^0 & \beta_r^1 & \cdots & \beta_r^{n-1} \end{bmatrix}, \quad Hc^T = 0.$$

Проверочная матрица H' над $\mathrm{GF}(q)$ получается заменой каждого элемента $\beta_i^j \in \mathrm{GF}(q^m)$ на столбец длины m из $\mathrm{GF}(q)$ — его координаты в фиксированном базисе $\mathrm{GF}(q^m)$ над $\mathrm{GF}(q)$.

Пример 16. (7, 4, 3) код Хемминга: $q = 2$, $n = 7$, $g(x) = x^3 + x + 1$ с корнями $\beta_1 = \alpha$, $\beta_2 = \alpha^2$, $\beta_3 = \alpha^4$, где α — примитивный элемент $\mathrm{GF}(2^3)$. Строки H над $\mathrm{GF}(2^3)$ — степени α, α^2, α^4; при переходе к $\mathrm{GF}(2)$ каждый элемент заменяется двоичным столбцом, и столбцами матрицы H' оказываются двоичные записи $1, \alpha, \alpha^2, \ldots, \alpha^6$ — все ненулевые векторы длины 3, то есть знакомая проверочная матрица кода Хемминга.

8.3 Коды БЧХ и Рида—Соломона

8.3.1 Определение

Определение 23. **Код БЧХ** (Боуза—Чоудхури—Хоквингема) над $\mathrm{GF}(q)$ длины n с **конструктивным расстоянием** δ — циклический код наибольшей возможной размерности, порождающий многочлен которого имеет корни $\alpha^b, \alpha^{b+1}, \ldots, \alpha^{b+\delta-2}$, где $\alpha \in \mathrm{GF}(q^m)$ — примитивный корень степени n из 1 (то есть $\alpha^n = 1$, $\alpha^j \neq 1$ при $0 < j < n$).

В силу теоремы Лагранжа n должно делить $q^m - 1$; если такое m подобрать нельзя, кода БЧХ с данными параметрами не существует. Частные случаи:

- $n = q^m - 1$ — **примитивный** код БЧХ;
- $b = 1$ — код БЧХ **в узком смысле**;
- $m = 1$ — **код Рида—Соломона**.

8.3.2 Граница БЧХ

Теорема 41. Если порождающий многочлен циклического кода длины n над $\mathrm{GF}(q)$ имеет корни $\alpha^b, \ldots, \alpha^{b+\delta-2}$, где α — примитивный корень степени n из 1, то минимальное расстояние кода $d \geq \delta$.

Доказательство. Линейный код имеет минимальное расстояние d тогда и только тогда, когда любые $d - 1$ столбцов его проверочной матрицы линейно независимы. Возьмем любые $t \leq \delta - 1$ столбцов $j_1, \ldots, j_t$ матрицы H над $\mathrm{GF}(q^m)$: первые t строк этих столбцов образуют матрицу

$$\begin{bmatrix} \alpha^{bj_1} & \alpha^{bj_2} & \cdots & \alpha^{bj_t} \\ \alpha^{(b+1)j_1} & \alpha^{(b+1)j_2} & \cdots & \alpha^{(b+1)j_t} \\ \vdots & \vdots & \vdots & \vdots \\ \alpha^{(b+t-1)j_1} & \alpha^{(b+t-1)j_2} & \cdots & \alpha^{(b+t-1)j_t} \end{bmatrix} = \begin{bmatrix} 1 & \cdots & 1 \\ \alpha^{j_1} & \cdots & \alpha^{j_t} \\ \vdots & \vdots & \vdots \\ \alpha^{(t-1)j_1} & \cdots & \alpha^{(t-1)j_t} \end{bmatrix} \operatorname{diag}\left(\alpha^{bj_1}, \ldots, \alpha^{bj_t}\right).$$

Левый множитель W — матрица Вандермонда от чисел $\alpha^{j_1}, \ldots, \alpha^{j_t}$, которые различны (различны показатели $j \bmod n$) и ненулевые; правый — диагональная матрица с ненулевой диагональю. Обе обратимы, поэтому столбцы линейно независимы над $\mathrm{GF}(q^m)$ и, следовательно, над $\mathrm{GF}(q)$. $\square$

8.3.3 Порождающий многочлен кода БЧХ

По теореме о корнях порождающего многочлена

$$g(x) = \operatorname{lcm}(M_{\alpha^b}(x), M_{\alpha^{b+1}}(x), \ldots, M_{\alpha^{b+\delta-2}}(x)),$$

то есть НОК (произведение **различных**) минимальных многочленов элементов $\alpha^b, \ldots, \alpha^{b+\delta-2}$. Проверочная матрица над $\mathrm{GF}(q^m)$ содержит $\delta - 1$ строк, над $\mathrm{GF}(q)$ — $m(\delta - 1)$ строк (часть может оказаться линейно зависимой), поэтому размерность

$$k \geq n - m(\delta - 1).$$

Для **двоичных** кодов БЧХ в узком смысле $M_\beta(x) = M_{\beta^2}(x)$ (свойство минимальных многочленов), поэтому достаточно корней $\alpha, \alpha^3, \alpha^5, \ldots$:

$$g(x) = \operatorname{lcm}(M_\alpha(x), M_{\alpha^3}(x), ..., M_{\alpha^{\delta-2}}(x)), \quad k \geq n - m\left\lfloor \frac{\delta-1}{2} \right\rfloor,$$

и в проверочной матрице над $\mathrm{GF}(2^m)$ достаточно оставить $\left\lfloor \frac{\delta-1}{2} \right\rfloor$ строк, соответствующих α^{2i+1}.

> **Пример 17.** Примитивный $(15, 7, 5)$ код БЧХ в узком смысле над $\mathrm{GF}(2)$. Пусть α — примитивный элемент $\mathrm{GF}(2^4)$, $\alpha^4 + \alpha + 1 = 0$; тогда $M_\alpha(x) = M_{\alpha^2}(x) = M_{\alpha^4}(x) = x^4 + x + 1$ (сопряженные элементы), $M_{\alpha^3}(x) = x^4 + x^3 + x^2 + x + 1$. Для $\delta = 5$ нужны корни $\alpha, \alpha^2, \alpha^3, \alpha^4$:
>
> $$g(x) = \operatorname{lcm}(M_\alpha(x), M_{\alpha^2}(x), M_{\alpha^3}(x), M_{\alpha^4}(x)) = (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1) = x^8 + x^7 + x^6 + x^4 + 1,$$
>
> откуда $k = 15 - 8 = 7$ и $d \geq \delta = 5$.

8.3.4 Коды Рида—Соломона

Код РС — код БЧХ длины $n \mid (q-1)$ над $\mathrm{GF}(q)$ (случай $m = 1$, все корни лежат в самом поле). Минимальный многочлен элемента $\beta \in \mathrm{GF}(q)$ над $\mathrm{GF}(q)$ линеен: $M_\beta(x) = x - \beta$, поэтому

$$g(x) = \prod_{i=0}^{\delta-2} (x - \alpha^{b+i}), \quad k = n - \delta + 1, \quad d \geq \delta.$$

Граница Синглтона $d \leq n - k + 1 = \delta$ дает $d = n - k + 1$: код Рида—Соломона — код с максимальным достижимым расстоянием (МДР-код).

8.4 Декодирование кодов БЧХ

Всюду ниже исправляется вектор ошибок веса $t \leq \left\lfloor \frac{\delta-1}{2} \right\rfloor$: позиции ошибок $j_1, ..., j_t$.

8.4.1 Синдромы, локаторы и значения ошибок

Пусть принят $y = c + e$, где e — вектор ошибок; в многочленной форме $y(x) = a(x)g(x) + e(x)$. Так как $g\big(\alpha^{b+i}\big) = 0$, **синдромы** (значения принятого многочлена в корнях ПМ) зависят только от ошибок:

$$S_i = y\big(\alpha^{b+i}\big) = e\big(\alpha^{b+i}\big) = \sum_{r=0}^{n-1} e_r \alpha^{(b+i)r}, \quad 0 \leq i < \delta - 1.$$

Введем стандартные обозначения: **значения ошибок** $E_l = e_{j_l}$ — символы ошибки в позиции j_l; **локаторы ошибок** $X_l = \alpha^{j_l}$ — степени примитивного корня, нумерующие позиции. Тогда

$$S_i = \sum_{l=1}^{t} E_l X_l^{b+i}, \quad 0 \leq i < \delta - 1: \quad t \text{ неизвестных пар } (E_l, X_l) \text{ и } \delta - 1 \text{ уравнений.}$$

8.4.2 Алгоритм Питерсона—Горенстайна—Цирлера

Многочлен локаторов ошибок — многочлен, корни которого суть обратные локаторы:

$$\Lambda(x) = \prod_{l=1}^{t} (1 - X_l x) = \sum_{l=0}^{t} \Lambda_l x^l, \quad \Lambda_0 = 1.$$

Для каждого локатора $0 = \Lambda(X_i^{-1}) = \sum_{l=0}^{t} \Lambda_l X_i^{-l}$, $1 \leq i \leq t$. Умножим на $E_i X_i^{b+j+t}$ и просуммируем по i; по определению синдромов $\sum_i E_i X_i^{b+j+t-l} = S_{j+t-l}$, поэтому

$$S_{j+t} + \Lambda_1 S_{j+t-1} + ... + \Lambda_t S_j = 0, \quad 0 \leq j < t: \quad \text{система из } t \text{ линейных уравнений на } \Lambda_1, ..., \Lambda_t.$$

В матричной форме:

$$\begin{bmatrix} S_0 & S_1 & \cdots & S_{t-1} \\ S_1 & S_2 & \cdots & S_t \\ \vdots & \vdots & \vdots & \vdots \\ S_{t-1} & S_t & \cdots & S_{2t-2} \end{bmatrix} \begin{bmatrix} \Lambda_t \\ \Lambda_{t-1} \\ \vdots \\ \Lambda_1 \end{bmatrix} = \begin{bmatrix} -S_t \\ -S_{t+1} \\ \vdots \\ -S_{2t-1} \end{bmatrix}.$$

Теорема 42. Ганкелева матрица синдромов S_z размера $z \times z$, $(S_z)_{ij} = S_{i+j}$, обратима при z, равном числу ошибок t, и вырождена при $z > t$.

Доказательство. $S_z = WDW^T$, где W — матрица Вандермонда от локаторов $X_1, \ldots, X_z$ (строки — степени X_l^i), а $D = \operatorname{diag}\left(E_1 X_1^b, \ldots, E_z X_z^b\right)$ — диагональная матрица (при $z \le t$; при $z > t$ считаем $E_z = X_z = 0$, что дает нулевые столбцы). Вандермонд обратим при различных ненулевых X_l, то есть при $z = t$; D обратима при $z \le t$ и вырождена при $z > t$. □

Алгоритм 9 (Питерсона—Горенстайна—Цирлера).

1. Вычислить синдромы $S_i = y\left(\alpha^{b+i}\right)$, $0 \le i < \delta - 1$ (схема Горнера, сложность $O((\delta-1)n)$).
2. Уменьшая предполагаемое число ошибок $t \le \tau = \left\lfloor \frac{\delta-1}{2} \right\rfloor$, добиться обратимости матрицы S_t (проверка обратимости — $O(t^3)$ операций).
3. Решить систему: коэффициенты $\Lambda_1, \ldots, \Lambda_t$ многочлена локаторов $\Lambda(x) = 1 + \sum_{i=1}^{t} \Lambda_i x^i$.
4. Найти локаторы: корни X_i^{-1} многочлена $\Lambda(x)$ (процедура Ченя — подстановка всех α^i, $0 \le i < n$, сложность $O(nt)$).
5. Найти значения ошибок E_l из системы $S_i = \sum_l E_l X_l^i$, $0 \le i < t$ (сложность $O(t^3)$).

Общая сложность непосредственной реализации — $O(\tau^4)$ из-за перебора t с решением СЛАУ.

8.4.3 Ключевое уравнение

Более эффективный путь — свести декодирование к одной полиномиальной задаче. Используя тождество геометрической прогрессии $(1 - X_l x)\left(\sum_{i=0}^{\delta-2} (X_l x)^i\right) \equiv 1 \bmod x^{\delta-1}$, перепишем многочлен синдромов $S(x) = \sum_{i=0}^{\delta-2} S_i x^i$:

$$S(x) = \sum_{l=1}^{t} E_l X_l^b \sum_{i=0}^{\delta-2} (X_l x)^i \equiv \sum_{l=1}^{t} \frac{E_l X_l^b}{1 - X_l x} \pmod{x^{\delta-1}}.$$

Определим **многочлен значений ошибок** $\Gamma(x) = \sum_{l=1}^{t} E_l X_l^b \prod_{j \ne l} (1 - X_j x)$ (числитель приведенной суммы дробей); умножение равенства на $\Lambda(x) = \prod_l (1 - X_l x)$ дает **ключевое уравнение**:

$$\Gamma(x) \equiv \Lambda(x) S(x) \pmod{x^{\delta-1}}, \quad \deg \Lambda(x) \le \left\lfloor \frac{\delta-1}{2} \right\rfloor, \quad \deg \Gamma(x) < \left\lfloor \frac{\delta-1}{2} \right\rfloor.$$

Теорема 43. Алгоритм Форни. После нахождения Λ и Γ значения ошибок вычисляются за $O(t^2)$:

$$E_i = \frac{X_i^{-b} \Gamma(X_i^{-1})}{\prod_{j \ne i} (1 - X_j X_i^{-1})}, \quad 0 \le i < t.$$

Доказательство. Подставим определение Γ; при $x = X_i^{-1}$ все сомножители с $j \ne i$ сокращаются:

$$\frac{X_i^{-b} \Gamma(X_i^{-1})}{\prod_{j \ne i} (1 - X_j X_i^{-1})} = \frac{X_i^{-b} \sum_{l=1}^{t} E_l X_l^b \prod_{j \ne l} (1 - X_j X_i^{-1})}{\prod_{j \ne i} (1 - X_j X_i^{-1})} = \frac{E_i \prod_{j \ne i} (1 - X_j X_i^{-1})}{\prod_{j \ne i} (1 - X_j X_i^{-1})} = E_i.$$

□

8.4.4 Расширенный алгоритм Евклида

Поиск НОД многочленов $a(x)$, $b(x)$ (полагаем $\deg a \ge \deg b$): последовательность делений с остатком

$$r_{i-1}(x) = q_i(x) r_i(x) + r_{i+1}(x), \quad \deg r_{i+1}(x) < \deg r_i(x),$$

начиная с $r_{-1} = a$, $r_0 = b$; НОД — последний ненулевой остаток. Каждое деление в матричной форме:

$$\begin{bmatrix} r_{i+1} \\ r_i \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q_i \end{bmatrix} \begin{bmatrix} r_i \\ r_{i-1} \end{bmatrix}, \quad \text{и накопленное произведение } \begin{bmatrix} r_{i+1} \\ r_i \end{bmatrix} = \begin{bmatrix} u_{i,0} & u_{i-1,0} \\ u_{i,1} & u_{i-1,1} \end{bmatrix} \begin{bmatrix} r_0 \\ r_{-1} \end{bmatrix}.$$

Теорема Безу: существуют многочлены $u(x)$, $v(x)$: $b(x)u(x) + a(x)v(x) = \gcd(a(x), b(x))$.

Свойства (индексы $u_{j,0}$, $u_{j,1}$ — элементы первой и второй строк накопленной матрицы после j шагов):

- $\deg u_{j,0}(x) = \sum_{i=0}^{j} \deg q_i(x) = \sum_{i=0}^{j} (\deg r_{i-1} - \deg r_i) = \deg r_{-1} - \deg r_j$;
- определитель $\det U_j = (-1)^{j+1} \neq 0$, поэтому всякий общий делитель $u_{j,0}$ и $u_{j,1}$ делит определитель, то есть $\gcd(u_{j,0}, u_{j,1}) = 1$;
- $r_{j+1}(x) = r_0(x)u_{j,0}(x) + r_{-1}(x)u_{j,1}(x)$, откуда $r_{j+1}(x) \equiv r_0(x)u_{j,0}(x) \bmod r_{-1}(x)$ — соотношение вида ключевого уравнения;
- $\gcd(r_{j+1}, u_{j,0}) = \gcd(r_{-1}, u_{j,0})$.

8.4.5 Алгоритм Сугиямы

Решает ключевое уравнение через расширенный алгоритм Евклида. Пусть $\delta = 2\tau + 1$ (τ — число исправляемых ошибок). Положим $r_{-1}(x) = x^{2\tau}$ (аналог a) и $r_0(x) = S(x)$ (аналог b), выполнять деления алгоритма Евклида, пока не выполнится условие останова

$$\deg r_i(x) \geq \tau \quad \text{и} \quad \deg r_{i+1}(x) < \tau,$$

и положить $\Gamma(x) = r_{i+1}(x)$, $\Lambda(x) = u_{i,0}(x)$. Сложность — $O(\tau^2)$ операций.

Корректность. Степени r_i монотонно убывают, поэтому останов достижим. Из свойства накопленной матрицы:

$$\Gamma(x) = r_{i+1}(x) = S(x)\Lambda(x) + x^{2\tau}u_{i,1}(x) \equiv S(x)\Lambda(x) \pmod{x^{2\tau}}, \quad \deg\Lambda = \deg r_{-1} - \deg r_i \leq 2\tau - \tau = \tau.$$

Пусть теперь $\Lambda'(x)$, $\Gamma'(x)$ — истинные многочлены локаторов и значений ($\Gamma' \equiv S\Lambda' \bmod x^{2\tau}$, $\deg\Lambda' \leq \tau$, $\deg\Gamma' < \tau$). Истинные многочлены взаимно просты: иначе у Λ' и Γ' был бы общий корень X_l^{-1} (все корни Λ' простые), что исключено определением Γ'. Тогда

$$\Gamma'\Lambda \equiv S\Lambda'\Lambda \equiv \Gamma\Lambda' \pmod{x^{2\tau}},$$

и поскольку обе части имеют степень меньше 2τ, равенство точное: $\Gamma'\Lambda = \Gamma\Lambda'$. Значит, $\mu = \Lambda/\Lambda' = \Gamma/\Gamma'$ — многочлен; при этом μ делит $u_{i,1}$ (из представления $\Gamma = S\Lambda + x^{2\tau}u_{i,1}$), но $\Lambda = u_{i,0}$ и $u_{i,1}$ взаимно просты, поэтому μ — константа: алгоритм дает истинное решение с точностью до умножения на константу.

8.4.6 Алгоритм Берлекэмпа—Месси

Третий способ решения ключевого уравнения — синтез **регистра сдвига с линейной обратной связью** (РСЛОС) минимальной длины, порождающего последовательность синдромов: рекуррентность $S_{j+t} + \Lambda_1 S_{j+t-1} + \ldots + \Lambda_t S_j = 0$ — это в точности уравнение РСЛОС. Фильтр (РСЛОС) длины L с многочленом связей $\Lambda(x) = 1 + \sum_{i=1}^{L} \Lambda_i x^i$ порождает первые n элементов последовательности $S_0, S_1, \ldots$, если

$$S_k = -\sum_{i=1}^{L} \Lambda_i S_{k-i} \quad \text{для всех } L \leq k \leq n-1.$$

Лемма 5. Пусть фильтры минимальных длин $\left(L_{n-1}, \Lambda^{(n-1)}(x)\right)$ и $\left(L_n, \Lambda^{(n)}(x)\right)$ порождают соответственно первые $n-1$ и первые n элементов последовательности, причем первый не порождает n-й элемент. Тогда $L_n \geq \max(L_{n-1}, n - L_{n-1})$.

Доказательство. Фильтр, порождающий более длинную последовательность, порождает и короткую, поэтому $L_n \geq L_{n-1}$. Пусть, вопреки утверждению, $L_n < n - L_{n-1}$. Тогда $S_{n-1} \neq$

$-\sum_{i=1}^{L_{n-1}} \Lambda_i^{(n-1)} S_{n-1-i}$. Но каждый элемент S_{n-1-i} с $i \le L_{n-1}$ имеет номер $n-1-i \ge n-L_{n-1}-1 > L_n - 1 \ge L_n$? Нет: $n-1-i \le n-2$, и все эти элементы порождаются фильтром $\left(L_n, \Lambda^{(n)}\right)$, так как $L_n \le n-i-1 \le n-2$ для всех $i \ge 1$. Подставляя их разложения и меняя порядок суммирования:

$$S_{n-1} \neq \sum_{k=1}^{L_n} \Lambda_k^{(n)} \sum_{i=1}^{L_{n-1}} \Lambda_i^{(n-1)} S_{n-1-k-i} = -\sum_{k=1}^{L_n} \Lambda_k^{(n)} S_{n-1-k} = S_{n-1}$$

— противоречие. □

Теорема 44. Пусть $\left(L_i, \Lambda^{(i)}(x)\right)$ — фильтры наименьшей длины, порождающие первые $i+1$ элементов $S_0, \ldots, S_i$, $0 \le i \le r-1$. Тогда фильтр с многочленом связей

$$\Lambda^{(r)}(x) = \begin{cases} \Lambda^{(r-1)}(x) & \text{если } \Delta_r^{(r-1)} = 0 \\ \Lambda^{(r-1)}(x) - \frac{\Delta_r^{(r-1)}}{\Delta_m^{(m-1)}} x^{r-m} \Lambda^{(m-1)}(x) & \text{если } \Delta_r^{(r-1)} \neq 0 \end{cases}$$

порождает первые $r+1$ элементов и имеет наименьшую длину $L_r = \max(L_{r-1}, r - L_{r-1})$. Здесь $\Delta_r^{(v)} = \sum_{j=0}^{L_v} \Lambda_j^{(v)} S_{r-1-j}$ — **невязка** (разность истинного значения S_{r-1} и вычисленного фильтром v), m — наибольшее число, меньшее r, при котором менялась длина фильтра ($L_{m-1} < L_m$), начальные условия $\Lambda^{(0)}(x) = 1$, $\Delta_0^{(0)} = 1$.

Доказательство. При нулевой невязке менять фильтр не нужно. При ненулевой: длина в последний раз менялась на шаге m, причем $L_{r-1} = L_m = m - L_{m-1}$ и $L_{m-1} < L_{r-1}$. Степень модифицированного многочлена не превосходит

$$\max\left(\deg \Lambda^{(r-1)}, r - m + \deg \Lambda^{(m-1)}\right) \le \max(L_{r-1}, r - m + L_{m-1}) = \max(L_{r-1}, r - L_{r-1}),$$

то есть соответствует длине L_r. Невязка модифицированного фильтра на элементе S_{r-1}:

$$\Delta_r^{(r)} = \Delta_r^{(r-1)} - \frac{\Delta_r^{(r-1)}}{\Delta_m^{(m-1)}} \cdot \sum_{j=0}^{L_{m-1}} \Lambda_j^{(m-1)} S_{r-1-j-(r-m)} = \Delta_r^{(r-1)} - \frac{\Delta_r^{(r-1)}}{\Delta_m^{(m-1)}} \cdot \Delta_m^{(m-1)} = 0,$$

так как слагаемая сумма — невязка фильтра $m-1$ на элементе $S_{r-1-(r-m)} = S_{m-1}$, равная $\Delta_m^{(m-1)}$. Для предшествующих элементов S_k, $L_{r-1} + 1 \le k \le r-1$, невязка

$$\Delta_k^{(r)} = \Delta_k^{(r-1)} - \frac{\Delta_r^{(r-1)}}{\Delta_m^{(m-1)}} \cdot \Delta_{k-r+m}^{(m-1)} = 0 :$$

первое слагаемое равно нулю (фильтр $r-1$ порождает элементы до номера $r-2$), а второе — невязка фильтра $m-1$ на элементе $S_{k-r+m-1}$ с номером $\le m-2$, также нулевая. □

Алгоритм 10 (Берлекэмпа—Месси).

```
BerlekampMassey(S_0, ..., S_(δ-2)):
  Λ(x) := 1,  r := 1,  m := 0,  L := 0,  B(x) := 1
  while r ≤ δ-1:
    Δ_r := Σ_{j=0..L} Λ_j S_(r-1-j)
    if Δ_r ≠ 0:
      T(x) := Λ(x) - Δ_r x^(r-m) B(x)
      if 2L ≤ r-1:
        B(x) := Δ_r^(-1) Λ(x)
        Λ(x) := T(x)
        L := r - L
```

```
      m := r
    else:
      Λ(x) := T(x)
  r := r + 1
return (L, Λ(x))
```

Здесь $B(x)$ хранит $\Delta_m^{((m-1))^{-1}}\Lambda^{(m-1)}(x)$ — масштабированный старый фильтр; условие $2L \le r-1$ фиксирует моменты смены длины. Если в результате $L \neq \deg\Lambda(x)$, число ошибок превышает $\frac{\delta-1}{2}$ (кроме расширенных кодов БЧХ). Сложность — $O((\delta-1)^2)$; для двоичных кодов БЧХ в узком смысле каждая вторая невязка $\Delta_{2i}=0$, поэтому половина итераций пропускается.

8.4.7 Сложность этапов декодирования

- **Синдромы**: схема Горнера $S_i = y_0 + \alpha^{b+i}\big(y_1 + \alpha^{b+i}(y_2 + \ldots)\big) - O(n\delta)$; метод Герцеля: вычислить остатки $r_i(x) \equiv y(x) \bmod M_{\alpha^{b+i}}(x)$ (минимальные многочлены, деление на которые в $\mathrm{GF}(p)[x]$ требует только сложений), затем $S_i = r_i\big(\alpha^{b+i}\big)$; многие минимальные многочлены совпадают, что экономит деления.
- **Решение ключевого уравнения**: алгоритм Сугиямы или Берлекэмпа—Месси — $O(\delta^2)$.
- **Корни многочлена локаторов** X_i^{-1}: процедура Ченя — перебор $O(n\delta/2)$.
- **Значения ошибок**: формула Форни — $O(\delta^2)$.

Итого декодирование БЧХ с исправлением $\lfloor\frac{\delta-1}{2}\rfloor$ ошибок — $O(n\delta+\delta^2)$.

8.4.8 Мягкое декодирование кодов БЧХ

Метод Чейза-2 (декодирование около жестких решений):

1. Найти τ наименее надежных позиций принятого вектора $(y_0,\ldots,y_{n-1})$; пусть это позиции $0,\ldots,\tau-1$.
2. Пусть $\hat y_i \in \mathrm{GF}(q)$ — жесткие решения. Перебрать все q^τ тестовых векторов $(x_0,\ldots,x_{\tau-1},\hat y_\tau,\ldots,\hat y_{n-1})$ с $x_i \in \mathrm{GF}(q)$.
3. Каждый тестовый вектор продекодировать жестким декодером, исправляющим $\frac{\delta-1}{2}$ ошибок.
4. Из полученных кодовых слов выбрать наиболее правдоподобное для принятого $(y_0,\ldots,y_{n-1})$.

Сложность $O(q^\tau\delta^2)$ (2^τ для двоичных кодов).

Метод Пиндии (декодирование с мягким выходом). Апостериорные логарифмические отношения правдоподобия

$$L_i = \log\frac{\sum_{c\in C:c_i=0}\prod_{j=0}^{n-1}P(c_j\mid y_j)}{\sum_{c\in C:c_i=1}\prod_{j=0}^{n-1}P(c_j\mid y_j)} \approx \log\frac{\max_{c\in C:c_i=0}\prod_j P(c_j\mid y_j)}{\max_{c\in C:c_i=1}\prod_j P(c_j\mid y_j)} = \min_{c\in C:c_i=1}E(c,y) - \min_{c\in C:c_i=0}E(c,y),$$

где $E(c,y)$ — корреляционная невязка (глава об универсальных методах декодирования): сумма $|y_j|$ по позициям, где c_j не совпадает с жестким решением. Точный максимум по всему коду недостижим, поэтому список L кодовых слов-кандидатов, полученный алгоритмом Чейза-2 (или Тала—Варди для полярных кодов), аппроксимирует его:

$$L_i \approx \min_{c\in L:c_i=1}E(c,y) - \min_{c\in L:c_i=0}E(c,y).$$

Внешние ЛОПП для итеративных схем (чтобы исключить двойной учет канальной информации):

$$\tilde L_i = \begin{cases} L_i - \log\frac{P(y_i\mid c_i=0)}{P(y_i\mid c_i=1)} & \text{если в } L \text{ есть слова с } c_i'=0 \text{ и } c_i''=1 \\ \beta(1-2\hat c_i) & \text{иначе}\end{cases},$$

где $\hat c$ — наиболее правдоподобное слово списка, β — параметр, подбираемый экспериментально.

Сравнение (коды длины 256): мягкое декодирование БЧХ (Чейз-2, box-and-match) и списочное декодирование полярных кодов и подкодов дают близкие кривые вероятности ошибки; среднее число

операций Чейза-2 для расширенного кода БЧХ $(256, 239)$ около 10^3 против около $6 \cdot 10^3$ у полярных декодеров; расширенные примитивные коды БЧХ в узком смысле имеют большее минимальное расстояние, чем полярные коды той же длины и размерности.

8.5 Приложения

8.5.1 Квадратично-вычетные коды

Число y называется **квадратичным вычетом** по модулю простого n, если сравнение $x^2 \equiv y \bmod n$ разрешимо; иначе — квадратичный невычет. Вычетами являются квадраты $1^2, 2^2, \ldots, \left(\frac{n-1}{2}\right)^2 \bmod n$; произведение вычетов — вычет.

> **Определение 24.** **Квадратично-вычетный код** — циклический код простой длины n над $\mathrm{GF}(p)$, где p — квадратичный вычет по модулю n, с порождающим многочленом $g_1(x) = \prod_{i \in Q}(x - \alpha^i)$ (или $(x-1)g_1(x)$) либо $g_2(x) = \prod_{i \in N}(x - \alpha^i)$ (или $(x-1)g_2(x)$), где α — примитивный корень степени n из 1, Q — множество квадратичных вычетов, N — невычетов по модулю n.

Минимальное расстояние КВ-кода удовлетворяет $d^2 \geq n$; при $n = 4s - 1$, $s \in \mathbb{N}$, справедливо более сильное $d^2 - d + 1 \geq n$.

8.5.2 QR-коды

Код Quick Response (1994) представляет данные черными и белыми точками; защита от ошибок считывания — коды Рида—Соломона с различными параметрами. Избыточность позволяет восстанавливать данные даже при частично замаскированном коде, что дает возможность художественных QR-кодов.

8.5.3 Кодирование для BluRay

Пикетный код: пользовательские данные, закодированные $(248, 216, 33)$ кодом РС над $\mathrm{GF}(2^8)$, перемежаются со служебными данными, закодированными $(62, 30, 33)$ кодом РС над $\mathrm{GF}(2^8)$ (служебный код выявляет пакеты ошибок). Длинный пакет ошибок канала после перемежения превращается в набор коротких пакетов, исправимых компонентными кодами.

8.5.4 CRC

Циклический код с проверкой избыточности (Cyclic Redundancy Check) используется для **обнаружения** ошибок. Контрольная сумма для многочлена данных $a(x)$, $\deg a \leq K - 1$:

$$r(x) \equiv x^{N-K} a(x) \pmod{g(x)}, \quad N - K = \deg g(x) \text{ проверочных символов}, \quad g(x) \mid x^n - 1,$$

где N — общая длина, K — число символов данных; требуется $K \leq n - \deg g(x)$: при $K > n - \deg g(x)$ получилось бы слово длиннее n с неизвестным (вероятно, плохим) минимальным расстоянием. Таблицы хороших порождающих многочленов CRC публикует, например, Koopman (users.ece.cmu.edu/ koopman/crc).

8.6 Выводы

- Циклические коды допускают еще более компактное задание, чем произвольные линейные коды: одним порождающим многочленом.
- Конструкция БЧХ позволяет строить коды с гарантированным минимальным расстоянием $d \geq \delta$; коды Рида—Соломона — МДР-коды на границе Синглтона.
- Существуют алгоритмы декодирования БЧХ с исправлением $\left\lfloor \frac{\delta - 1}{2} \right\rfloor$ ошибок сложностью $O(n\delta + \delta^2)$: ПГЦ, Сугиямы, Берлекэмпа—Месси с формулой Форни.
- Расширенные примитивные коды БЧХ в узком смысле достигают предела Шеннона для двоичного стирающего канала; метод Чейза—Пиндии дает мягкое декодирование.

9 Альтернантные коды и криптосистема Мак-Элиса

9.1 Обобщенные коды Рида—Соломона

9.1.1 Коды Рида—Соломона через значения многочлена

Напомним определение кода БЧХ из предыдущей главы: **кодом БЧХ** над полем $\mathrm{GF}(q)$ длины n с конструктивным расстоянием δ называется циклический код наибольшей возможной размерности, у которого порождающий многочлен имеет корнями степени примитивного корня $\alpha \in \mathrm{GF}(q^m)$ с показателями $b, b+1, \ldots, b+\delta-2$ (α — элемент поля расширения порядка n). В силу теоремы Лагранжа порядок любого элемента делит порядок группы, поэтому n должно делить $q^m - 1$; если такое m подобрать нельзя, кода БЧХ данной длины не существует. Частные случаи:

- $n = q^m - 1$ — **примитивный** код БЧХ;
- $b = 1$ — код БЧХ **в узком смысле**;
- $m = 1$ — код Рида—Соломона (корни лежат в самом поле $\mathrm{GF}(q)$).

Теорема 45. Кодовое слово $c = (c_0, \ldots, c_{n-1})$ кода Рида—Соломона над $\mathrm{GF}(q)$ в узком смысле тогда и только тогда, когда оно состоит из значений многочлена: $c_i = f(\alpha^i)$ для $0 \le i < n$, где $f(x)$ — многочлен над $\mathrm{GF}(q)$ степени меньше k. Кратко: $c = \mathrm{ev}(f)$, где $\mathrm{ev}(f) = (f(\alpha^0), \ldots, f(\alpha^{n-1}))$.

Доказательство. Пусть n — порядок α, то есть $\alpha^n = 1$. Тогда для $0 < i < n$

$$0 = \alpha^{in} - 1 = (\alpha^i - 1)\sum_{j=0}^{n-1} \alpha^{ij},$$

и так как $\alpha^i \ne 1$ при $0 < i < n$, имеем $\sum_{j=0}^{n-1} \alpha^{ij} = 0$ — суммы степеней корня из единицы обнуляются.

По определению циклического кода, c — кодовое слово тогда и только тогда, когда $cH^T = 0$, что для кода БЧХ в узком смысле равносильно условиям $\sum_{i=0}^{n-1} c_i \alpha^{ij} = 0$ для $1 \le j \le n-k$. Если $c_i = f(\alpha^i)$, где $f(x) = \sum_{t=0}^{k-1} f_t x^t$, то

$$\sum_{i=0}^{n-1} f(\alpha^i)\alpha^{ij} = \sum_{i=0}^{n-1}\left(\sum_{t=0}^{k-1} f_t \alpha^{it}\right)\alpha^{ij} = \sum_{t=0}^{k-1} f_t \sum_{i=0}^{n-1} \alpha^{i(j+t)} = 0,$$

поскольку $1 \le j + t \le n - k + k - 1 < n$, то есть $\alpha^{j+t} \ne 1$. Значит, множество $\{\mathrm{ev}(f) \mid \deg f < k\}$ содержится в коде.

Обратно, векторы $\mathrm{ev}(f)$ для $\deg f < k$ — это в точности линейные комбинации строк матрицы Вандермонда (α^{it}), $0 \le t < k$, $0 \le i < n$; их ровно q^k, то есть столько же, сколько кодовых слов, значит совпадение множеств полное. □

9.1.2 Второе определение кодов Рида—Соломона

Определение 25. **(n, k, n-k+1) кодом Рида—Соломона** называется множество векторов $c = (c_0, \ldots, c_{n-1})$, где $c_i = f(a_i)$ для различных элементов $a_0, \ldots, a_{n-1} \in \mathrm{GF}(q)$ (**локаторов** — они нумеруют позиции кодового слова) и произвольного многочлена $f(x)$ над $\mathrm{GF}(q)$ степени меньше k.

Теорема 46. Минимальное расстояние этого кода равно $n - k + 1$.

Доказательство. Пусть $d \le n - k$. Тогда существуют различные многочлены $f^{(1)}(x)$, $f^{(2)}(x)$ степеней меньше k, совпадающие в $n - d \ge k$ точках a_i. Разность $v(x) = f^{(1)}(x) - f^{(2)}(x)$ имеет

степень меньше k, но имеет не менее k корней, значит $v(x) = 0$ — противоречие: одинаковые кодовые слова получаются только из одинаковых многочленов, то есть $d > n - k$. Верхняя граница $d \leq n - k + 1$ следует из границы Синглтона. □

Циклический код Рида—Соломона в узком смысле получается выбором локаторов в виде степеней примитивного корня: $a_i = \alpha^i$.

9.1.3 Обобщенные коды РС

Определение 26. **Обобщенным (n, k, d = n - k + 1) кодом Рида—Соломона** $\mathrm{GRS}(n, k, a, u)$ называется множество векторов $(c_0 u_0, ..., c_{n-1} u_{n-1})$, где $(c_0, ..., c_{n-1})$ — кодовое слово $(n, k, n - k + 1)$ кода Рида—Соломона, то есть $c_i = f(a_i)$, $\deg f < k$, a_i различны, а $u_0, ..., u_{n-1}$ — произвольные **ненулевые** константы поля (домножение позиций на весовые коэффициенты).

Для декодирования обобщенных кодов применимы те же алгоритмы, что и для обычных кодов Рида—Соломона: домножение на константы учитывается простым пересчетом значений и локаторов ошибок.

9.2 Альтернантные коды

9.2.1 Определение

Определение 27. **Альтернантным кодом** длины n над полем $\mathrm{GF}(q)$ называется код $A(n, r, a, u)$ с проверочной матрицей

$$H = \begin{bmatrix} a_0^0 & a_1^0 & ... & a_{n-1}^0 \\ a_0^1 & a_1^1 & ... & a_{n-1}^1 \\ \vdots & \vdots & \vdots & \vdots \\ a_0^{r-1} & a_1^{r-1} & ... & a_{n-1}^{r-1} \end{bmatrix} \mathrm{diag}(u_0, ..., u_{n-1}),$$

где $a_0, ..., a_{n-1} \in \mathrm{GF}(q^m)$ — различные элементы (**локаторы**), $u_0, ..., u_{n-1} \in \mathrm{GF}(q^m) \setminus \{0\}$ — ненулевые весовые коэффициенты, r — число проверочных строк. Матрица Вандермонда слева — это в точности проверочная матрица обобщенного кода Рида—Соломона $\mathrm{GRS}(n, n - r, a, v)$ при подходящем выборе весов v.

Иначе говоря, альтернантный код — это **ограничение на подполе** $\mathrm{GF}(q)$ обобщенного кода РС над полем расширения $\mathrm{GF}(q^m)$:

$$A(n, r, a, u) = \mathrm{GRS}(n, n - r, a, v) \cap \mathrm{GF}(q)^n.$$

Чтобы получить проверочную матрицу над $\mathrm{GF}(q)$ (код определен над меньшим полем, а элементы H лежат в $\mathrm{GF}(q^m)$), каждый элемент $H_{i,j}$ представляется по базису поля расширения: $H_{i,j} = \sum_{s=0}^{m-1} H_{i,j,s} \alpha^s$, и заменяется столбцом $\left(H_{i,j,0}, ..., H_{i,j,m-1}\right)^T$; матрица разрастается с r строк до rm строк.

Параметры альтернантного кода:

$$d \geq r + 1, \quad n - r \geq k \geq n - mr.$$

Нижняя граница размерности $k \geq n - mr$ следует из того, что у матрицы H над $\mathrm{GF}(q)$ не более mr независимых строк (каждая строка над $\mathrm{GF}(q^m)$ разворачивается в m строк над $\mathrm{GF}(q)$). Минимальное расстояние $d \geq r + 1$ наследуется от кода РС. Для декодирования используются алгоритмы декодирования кодов Рида—Соломона: альтернантный код — подкод кода РС, поэтому декодер РС, исправляющий t ошибок, исправляет их и в альтернантном коде.

9.2.2 Существование хороших альтернантных кодов

Коды БЧХ асимптотически плохи (их относительное расстояние стремится к нулю при фиксированной скорости). Для альтернантных кодов верна теорема о существовании кодов, лежащих на границе Варшамова—Гилберта.

Теорема 47. Пусть m делит $n-h$. Существует альтернантный $(n, k \geq h, d \geq \delta)$ код над $\mathrm{GF}(q)$ такой, что

$$\sum_{i=1}^{\delta-1} (q-1)^i \binom{n}{i} < (q^m-1)^{(n-h)/m}.$$

Доказательство. Зафиксируем вектор локаторов $a = (a_0, ..., a_{n-1})$. Оценим, скольким различным обобщенным кодам РС может принадлежать фиксированный ненулевой вектор $y \in \mathrm{GF}(q)^n$.

Вектор y принадлежит $\mathrm{GRS}(n, k', a, v)$ тогда и только тогда, когда $y_i/v_i = f(a_i)$ для некоторого многочлена степени меньше k'. Построим такой код явно: выберем произвольно первые k' весовых коэффициентов $v_0, ..., v_{k'-1}$ (каждый — один из q^m-1 ненулевых элементов), построим по точкам $(a_i, y_i/v_i)$, $0 \leq i < k'$, интерполяционный многочлен $f(x)$ степени меньше k', после чего определим оставшиеся веса условием $v_i = y_i/f(a_i)$ при $k' \leq i < n$. Если при этом получилось $y_i = 0$, положим $v_i = 1$. Если же $y_i = 0$, но $f(a_i) \neq 0$, либо наоборот — выбор первых k' весов считается неудачным. Отсюда

$$|\{\mathrm{GRS}(n, k', a, v) : y \in \mathrm{GRS}(n, k', a, v)\}| \leq (q^m-1)^{k'}.$$

Рассмотрим альтернантный код $A(n, (n-h)/m, a, u) = \mathrm{GRS}(n, n-(n-h)/m, a, v) \cap \mathrm{GF}(q)^n$. Его размерность $k \geq h$, поскольку проверочных условий над $\mathrm{GF}(q^m)$ всего $(n-h)/m$, а над $\mathrm{GF}(q)$ их не более $n-h$.

Плохим для нас считаем код с $d < \delta$: он содержит ненулевое кодовое слово y веса $i < \delta$, а таких векторов $\sum_{i=1}^{\delta-1} \binom{n}{i}(q-1)^i$. Каждый такой вектор лежит не более чем в $(q^m-1)^{n-(n-h)/m}$ кодах семейства, поэтому

$$|\{C = A(n, (n-h)/m, a, u) : d_C < \delta\}| \leq (q^m-1)^{n-(n-h)/m} \sum_{i=1}^{\delta-1} (q-1)^i \binom{n}{i}.$$

Всего же различных весов v имеется не менее $(q^m-1)^n$, то есть столько различных кодов $\mathrm{GRS}(n, n-(n-h)/m, a, v)$. Если все они были бы плохими, выполнялось бы противоположное неравенство теоремы; значит, при выполнении неравенства теоремы существует хороший код.□

9.2.3 Связь с границей Варшамова—Гилберта

Теорема 48 (граница Варшамова—Гилберта). Если $q^r > \sum_{i=0}^{d-2} \binom{n-1}{i}(q-1)^i$, то существует линейный код над $\mathrm{GF}(q)$ длины n с минимальным расстоянием не менее d и не более чем $r = n-k$ проверочными символами.

Доказательство существования хороших альтернантных кодов согласуется с этой границей: при подходящих параметрах

$$\sum_{i=0}^{d-2} \binom{n-1}{i}(q-1)^i < \sum_{i=0}^{\delta-1} \binom{n}{i}(q-1)^i < (q^m-1)^{(n-h)/m} < q^{n-h},$$

то есть альтернантные коды достигают качества случайных линейных кодов — в отличие от кодов БЧХ, асимптотически плохих.

9.3 Коды Гоппы

9.3.1 Определение

Определение 28. Пусть задан многочлен $G(x)$ над $\mathrm{GF}(q^m)$ (**многочлен Гоппы**) степени r и различные элементы $a_0, \dots, a_{n-1} \in \mathrm{GF}(q^m)$, причем $G(a_i) \neq 0$ для всех i (позиции не совпадают с корнями многочлена Гоппы). **Кодом Гоппы** называется множество векторов $(c_0, \dots, c_{n-1}) \in \mathrm{GF}(q)^n$, удовлетворяющих условию

$$\sum_{i=0}^{n-1} c_i (x - a_i)^{-1} \equiv 0 \pmod{G(x)},$$

где $(x - a_i)^{-1}$ — многочлен, обратный двучлену $x - a_i$ в кольце вычетов по модулю $G(x)$ (он существует, так как $G(a_i) \neq 0$).

9.3.2 Коды Гоппы альтернантны

Многочлен $x - a_i$ обратим по модулю $G(x) = \sum_{s=0}^{r} g_s x^s$, причем разность

$$\frac{(G(x) - G(a_i))}{x - a_i} = g_r(x^{r-1} + x^{r-2}a_i + \dots + a_i^{r-1}) + g_{r-1}(x^{r-2} + \dots + a_i^{r-2}) + \dots + g_2(x + a_i) + g_1$$

— многочлен степени меньше r. Прямая проверка дает

$$(x - a_i)^{-1} = -\frac{(G(x) - G(a_i))}{x - a_i} \cdot G^{-1}(a_i),$$

где $G^{-1}(a_i) = 1/G(a_i)$ — обратный элемент поля. Группируя члены при степенях x, условие определения кода переписывается как

$$0 \equiv \sum_{i=0}^{n-1} \frac{c_i}{G(a_i)} \cdot \frac{G(x) - G(a_i)}{x - a_i} \Leftrightarrow 0 = \sum_{i=0}^{n-1} \frac{c_i}{G(a_i)} \sum_{j=0}^{k} g_{r-k+j} a_i^j, \quad 0 \leq k < r.$$

Отсюда читается проверочная матрица:

$$H = \begin{bmatrix} g_r & g_{r-1} & \dots & g_1 \\ g_r & g_{r-2} & \dots & g_2 \\ \vdots & \vdots & \vdots & \vdots \\ g_r & \dots & \dots & g_r \end{bmatrix} \begin{bmatrix} a_0^0 & a_1^0 & \dots & a_{n-1}^0 \\ a_0^1 & a_1^1 & \dots & a_{n-1}^1 \\ \vdots & \vdots & \vdots & \vdots \\ a_0^{r-1} & a_1^{r-1} & \dots & a_{n-1}^{r-1} \end{bmatrix} \mathrm{diag}\left(\frac{1}{G(a_0)}, \dots, \frac{1}{G(a_{n-1})}\right).$$

Левый множитель — ганкелева матрица коэффициентов G — обратима, поэтому ее можно исключить из рассмотрения. Оставшиеся сомножители — матрица Вандермонда по локаторам a_i и диагональная матрица весов $1/G(a_i)$ — образуют проверочную матрицу альтернантного кода $A(n, r, a, u)$ с $u_i = 1/G(a_i)$. Значит, каждый код Гоппы — альтернантный код, и для него работают алгоритмы декодирования РС.

9.3.3 Двоичные коды Гоппы

Для двоичных кодов границу расстояния удается улучшить. Пусть ненулевые символы кодового слова $(c_0, \dots, c_{n-1}) \in \mathrm{GF}(2)^n$ расположены в позициях $l_1, \dots, l_w$ (w — вес слова). Введем многочлен $f(x) = \prod_{i=1}^{w} \left(x - a_{l_i}\right)$, корнями которого являются локаторы ненулевых позиций. Тогда

$$0 \equiv \sum_{i=1}^{n} \frac{c_i}{x - a_i} = \sum_{i=1}^{w} \frac{1}{x - a_{l_i}} = \frac{f'(x)}{f(x)} \pmod{G(x)},$$

где $f'(x)$ — формальная производная многочлена $f(x)$ по x (сумма одночленов по одному на каждый сомножитель). Многочлены $f(x)$ и $G(x)$ взаимно просты (корни G не являются локаторами), поэтому из сравнения следует $G(x) \mid f'(x)$ — делимость без остатка.

В поле характеристики 2 формальная производная содержит только четные степени, а значит, является полным квадратом: $f'(x) = u^2(x)$ для некоторого многочлена $u(x)$. Пусть $Q(x)$ — полный квадрат наименьшей степени, делящийся на $G(x)$. Тогда $Q(x) \mid f'(x)$ для всякого кодового слова, и, следовательно, замена $G(x)$ на $Q(x)$ в определении кода не изменит множество кодовых слов. Отсюда улучшенная оценка минимального расстояния:

$$d \geq \deg Q(x) + 1.$$

Если исходный многочлен Гоппы **свободен от квадратов** (не делится ни на какой квадрат многочлена), то $Q(x) = G^2(x)$ и

$$d \geq 2 \deg G(x) + 1;$$

такие коды называются **сепарабельными** кодами Гоппы. Именно они используются в криптографии: при фиксированном числе проверок $r = \deg G$ расстояние вдвое больше, чем у общего альтернантного кода.

9.4 Криптосистема Мак-Элиса

9.4.1 Постквантовая мотивация

Задача: как Алисе безопасно передать Бобу сообщение m по открытому каналу? Схема с открытым ключом: у Боба есть открытый (публичный) ключ P и закрытый (секретный) ключ S; Алиса отправляет $c = f(P, m)$, Боб восстанавливает $m = g(S, c)$. Требования:

1. корректность: $g(S, f(P, m)) = m$;
2. восстановить m, зная P и c, вычислительно сложно;
3. восстановить S, зная P, вычислительно сложно.

Классические теоретико-числовые криптосистемы (RSA, протокол Диффи—Хеллмана) основаны на факторизации целых чисел и дискретном логарифме. Классическим компьютерам факторизация числа z методом обобщенного решета числового поля требует субэкспоненциальной сложности, но квантовый компьютер решает ее алгоритмом Шора за полиномиальное время (экспериментально продемонстрировано разложение 15-битного числа на 5-кубитном устройстве). Поэтому такие системы уязвимы к квантовым атакам, и постквантовая криптография ищет задачи, сложные **и для квантовых** компьютеров.

9.4.2 Схема Мак-Элиса (1978)

Ключевая идея: замаскировать код с известным эффективным декодером так, чтобы он выглядел как произвольный линейный код, декодировать который — NP-полная задача.

Пусть $G = (I \mid A)$ — порождающая матрица в систематическом виде кода, для которого известен эффективный алгоритм исправления t ошибок (в схеме Мак-Элиса — код Гоппы, ограничение на подполе некоторого обобщенного кода РС; число различных кодов Гоппы велико).

- **Секретный ключ**: обратимая матрица Q (маскирующая) и перестановочная матрица P (перемешивающая позиции), а также параметры кода.
- **Открытый ключ**: пара $[\Gamma = QGP, t]$, где Γ — порождающая матрица «замаскированного» кода, t — заявленная корректирующая способность.
- **Шифрование** сообщения x: Алиса вычисляет $y = x\Gamma + e$, добавляя случайный вектор ошибок e веса не более t.
- **Дешифрование** криптограммы y: Боб вычисляет $y' = yP^{-1} = xQG + eP^{-1} = xQG + e'$; так как перестановка не меняет вес, $\mathrm{wt}(e') \leq t$, и известный Бобу эффективный декодер кода с матрицей G находит e'; далее $y' - e' = xQG = (xQ \mid xQA)$ — систематическая запись, откуда сразу читается xQ и, по обратимости Q, сообщение x.

Стойкость:

- поиск секретного ключа по открытому — задача об эквивалентности кодов (для кодов РС она решается просто, но для кодов Гоппы эффективных алгоритмов не известно);
- восстановление сообщения по криптограмме — задача декодирования линейного кода, NP-полная.

Недостаток: большой размер открытого ключа — порядка 512 Кбит, так как передается вся матрица Г.

9.4.3 Криптосистема Нидеррайтера

Двойственная схема, работающая с проверочной матрицей. Пусть H — проверочная матрица линейного блокового кода над $\mathrm{GF}(q)$, $q > 2$, с известным алгоритмом декодирования, исправляющим t ошибок.

- **Открытый ключ**: $P = QH\Pi$, где Q — обратимая матрица, Π — перестановочная матрица.
- **Секретный ключ**: тройка $[Q, H, \Pi]$.
- **Шифрование**: символы сообщения размещаются в случайно выбранных t позициях вектора e веса t (остальные элементы нулевые); криптограмма $S = Pe$ — синдром вектора ошибок.
- **Дешифрование**: Боб вычисляет $S' = Q^{-1}S$, декодирует синдром S' (алгоритм кода с матрицей H находит вектор ошибок $e' = \Pi e$) и извлекает данные из ненулевых символов вектора $\Pi^{-1}e' = e$.

Криптосистемы Мак-Элиса и Нидеррайтера эквивалентны в смысле безопасности. Нидеррайтер предлагал использовать обобщенные коды Рида—Соломона, однако Сидельников и Шестаков показали: для кодов РС секретный ключ восстанавливается по открытому с полиномиальной сложностью — система на основе кодов РС небезопасна. Коды Гоппы этой атаке не поддаются (структура скрыта ограничением на подполе).

9.4.4 Согласование ключа

Матрица Г в схеме Мак-Элиса слишком велика для передачи; для выработки общего секретного ключа используется компактный вариант на основе Нидеррайтера. Пусть проверочная матрица кода Гоппы приведена к виду $H = Q(I \mid T)$, где Q — обратимая матрица, а T — матрица размера $r \times (n - r)$ (систематическая форма: единичная подматрица плюс остаток).

- **Открытый ключ**: пара $[T, t]$; **секретный ключ**: параметры кода Гоппы (многочлен G и локаторы).
- Алиса генерирует случайный вектор $e \in \mathrm{GF}(2)^n$ веса t, вычисляет синдром $S = (I \mid T)e^T$ и отправляет S Бобу.
- Боб формирует вектор $y = (S^T \mid 0)$ — это вектор с синдромом S — и декодирует его в коде Гоппы, получая ближайшее кодовое слово c, после чего вычисляет вектор ошибок $e = y - c$.
- Теперь и Алиса, и Боб знают один и тот же вектор e — его используют в качестве общего секрета (например, пропустив через хеш-функцию).

Перехватчик, зная S и T, должен декодировать случайный линейный код — NP-полная задача.

9.5 Выводы

- Альтернантные коды — простое обобщение кодов БЧХ (ограничение на подполе обобщенного кода РС); к ним применимы все алгоритмы декодирования БЧХ/РС.
- Коды БЧХ асимптотически плохи, но существуют альтернантные коды (в том числе коды Гоппы), лежащие на границе Варшамова—Гилберта; для сепарабельных двоичных кодов Гоппы $d \geq 2\deg G + 1$.
- Теоретико-числовые криптосистемы с открытым ключом уязвимы к квантовым атакам (алгоритм Шора).
- Криптосистема Мак-Элиса основана на NP-полной задаче декодирования линейного блокового кода и рассматривается как ведущий кандидат для постквантовой криптографии (с кодами Гоппы; РС-вариант Нидеррайтера взломан Сидельниковым—Шестаковым). Главный недостаток — большой размер открытого ключа.

10 Коды с малой плотностью проверок на четность (МППЧ, LDPC)

10.1 Понятие МППЧ

Коды с малой плотностью проверок на четность (МППЧ; английский термин — low-density parity-check codes, LDPC) — это (n, k) линейные коды, обладающие **разреженной** проверочной матрицей H размера $m \times n$ ($m \geq n - k$): в ней мало единиц, чаще всего число единиц в каждой строке и каждом столбце ограничено константой, не зависящей от n.

У всякого линейного блокового кода есть много различных проверочных и порождающих матриц, и не все они разрежены; требование МППЧ — существование **хотя бы одной** разреженной проверочной матрицы. Каждое кодовое слово удовлетворяет системе линейных уравнений $cH^T = 0$, в каждом из которых мало слагаемых — это и делает возможным итеративное декодирование с трудоемкостью, растущей линейно по длине кода.

10.1.1 Граф Таннера

Проверочной матрице H сопоставляется двудольный **граф Таннера**: $H_{i,j} = 1$ тогда и только тогда, когда в графе есть ребро между i-м проверочным узлом и j-м символьным узлом. Символьные узлы соответствуют столбцам H (символам кодового слова), проверочные узлы — строкам H (уравнениям проверки на четность).

10.2 Регулярные и нерегулярные коды

> **Определение 29**. Код называется **(i, j)-регулярным**, если существует проверочная матрица, у которой вес каждого столбца равен i (каждый символ входит ровно в i проверок), а вес каждой строки равен j (каждая проверка связывает ровно j символов). Соответствующий граф Таннера также называется (i, j)-регулярным. **Степень узла** графа — число инцидентных ему ребер.

Нерегулярные коды описываются распределениями степеней узлов в терминах ребер: полиномы

$$\lambda(x) = \sum_{i \geq 1} \lambda_i x^{i-1}, \quad \rho(x) = \sum_{i \geq 1} \rho_i x^{i-1},$$

где λ_i — доля всех ребер графа Таннера, инцидентных символьным узлам степени i, а ρ_i — доля ребер, инцидентных проверочным узлам степени i; нормировки $\lambda(1) = 1, \rho(1) = 1$ означают, что доли суммируются в единицу.

Важно: регулярность и нерегулярность — характеристики проверочной матрицы, а не самого кода; распределение степеней может быть изменено линейными преобразованиями строк.

10.2.1 Связь параметров

Пусть E — число ребер графа Таннера. Тогда число символьных узлов степени i равно $V_i = \lambda_i E / i$ (каждый такой узел несет i ребер), число проверочных узлов степени i равно $C_i = \rho_i E / i$. Доли узлов степени i среди всех узлов того же типа:

$$\Lambda_i = \frac{\lambda_i E / i}{\sum_{j \geq 1} \lambda_j E / j} = \frac{\lambda_i / i}{\int_0^1 \lambda(x)\,\mathrm{d}x}, \quad P_i = \frac{\rho_i / i}{\int_0^1 \rho(x)\,\mathrm{d}x},$$

и полиномы долей узлов: $\Lambda(x) = \sum_{i \geq 1} \Lambda_i x^{i-1}$, $P(x) = \sum_{i \geq 1} P_i x^{i-1}$. Относительная размерность проверочной матрицы:

$$\frac{m}{n} = \frac{\sum_{i \geq 1} C_i}{\sum_{j \geq 1} V_j} = \frac{\int_0^1 \rho(x)\,\mathrm{d}x}{\int_0^1 \lambda(x)\,\mathrm{d}x}.$$

Скорость кода может быть **больше** $1 - m/n$, если в проверочной матрице есть линейно зависимые строки.

10.3 Декодирование

10.3.1 Двоичный симметричный канал: алгоритм инвертирования битов

Простейший итеративный декодер (бит-флип):

1. Пусть $y = mG + e$ — принятое слово (m — информационный вектор, e — вектор ошибок).
2. Вычислить синдром $S = yH^T = eH^T$; если $S = 0$, работа завершена.
3. Выделить множество **неудовлетворенных проверок** — ненулевых компонент синдрома (проверочных узлов графа Таннера с нечетным числом ошибочных соседей).
4. Инвертировать каждый символ кодового слова, входящий не менее чем в T неудовлетворенных проверок (порог T — параметр алгоритма).
5. Повторять с шага 2, пока не превышено максимальное число итераций.

Работа алгоритма опирается на разреженность: с большой вероятностью каждое проверочное уравнение содержит не более одного ошибочного символа, и этот символ выделяется на голосовании.

10.3.2 Исправление стираний: распространение доверия

В двоичном стирающем канале символы либо принимаются достоверно, либо стираются (позиция стирания известна). Восстановление идет по проверочным уравнениям: если в уравнении стерт только один символ, его значение однозначно вычисляется из остальных; найденные значения подставляются в другие уравнения, запуская каскад. Итеративный алгоритм с такой логикой — **алгоритм распространения доверия** (его анализ для стираний и блокирующие множества обсуждаются ниже).

10.3.3 Постановка задачи распространения доверия

Пусть y_i — результат передачи символов c_i кодового слова по каналу без памяти, известны переходные вероятности $p(y_i \mid c_i = x)$, $x \in \{0, 1\}$. Задача декодирования по максимуму правдоподобия:

$$\hat{c} = \arg\max_{c : cH^T = 0} \prod_{i=1}^{n} p(y_i \mid c_i)$$

— поиск наиболее вероятного решения системы линейных однородных уравнений $cH^T = 0$. Точный перебор экспоненциален, поэтому ищем **наиболее вероятные значения отдельных символов** c_i при условии выполнения всех проверок. Точный расчет возможен, если граф Таннера — дерево с корнем в i-м символьном узле; итеративный алгоритм применяет формулы дерева к графам с циклами, получая приближение.

> **Лемма 6** (Галлагера). Пусть $a_1, ..., a_m$ — независимые случайные величины, принимающие значения 0 или 1, с $P\{a_i = 1\} = p_i$. Тогда

$$P\{a_1 + a_2 + ... + a_m \equiv 0 \bmod 2\} = \frac{1}{2}\left(1 + \prod_{i=1}^{m}(1 - 2p_i)\right).$$

Доказательство. При $m = 1$ утверждение очевидно. Пусть оно верно для $m - 1$ величины. Тогда по формуле полной вероятности

$$\begin{aligned} P\{a_1 + ... + a_m \equiv 0\} &= P\{a_1 + ... + a_{m-1} \equiv 0\}(1 - p_m) + P\{a_1 + ... + a_{m-1} \equiv 1\}p_m \\ &= \frac{1}{2}\left(1 + \prod_{i=1}^{m-1}(1 - 2p_i)\right)(1 - p_m) + \frac{1}{2}\left(1 - \prod_{i=1}^{m-1}(1 - 2p_i)\right)p_m \\ &= \frac{1}{2}\left(1 + \prod_{i=1}^{m}(1 - 2p_i)\right). \end{aligned}$$

□

10.3.4 Алгоритм распространения доверия (вероятностная форма)

Введем обозначения. Пусть C_i — множество проверок, включающих символ c_i (соседи i-го символьного узла), V_j — множество символов j-й проверки (соседи j-го проверочного узла). Основные величины:

- $q_{i(b)} = P\{c_i = b \mid Y, C_i\}$ — вероятность того, что i-й символ равен $b \in \{0,1\}$ при условии принятого вектора $Y = (y_1, ..., y_n)$ и выполнения всех проверок из C_i;
- $r_{ji}(b)$ — вероятность того, что j-я проверка выполнена, при условии, что i-й символ (входящий в нее) равен b, и принят вектор Y;
- $q_{ij}(b) = P\{c_i = b \mid Y, C_i \setminus \{j\}\}$ — то же, что q_i, но без учета сообщения от проверки j (чтобы информация не возвращалась по тому же ребру).

По формуле Байеса (множители-нормировки α_i, α_{ij} не влияют на сравнение гипотез):

$$q_{i(b)} = \alpha_i P\{c_i = b \mid y_i\} \prod_{j' \in C_i} r_{j'i}(b), \quad q_{ij}(b) = \alpha_{ij} P\{c_i = b \mid y_i\} \prod_{j' \in C_i \setminus \{j\}} r_{j'i}(b).$$

Обновление проверочных узлов опирается на лемму Галлагера: проверка выполняется при $c_i = 0$, если сумма по модулю 2 прочих символов проверки равна 0, то есть

$$r_{ji}(0) = \frac{1}{2}\left(1 + \prod_{i' \in V_j \setminus \{i\}} \big(1 - 2q_{i'j}(1)\big)\right), \quad r_{ji}(1) = 1 - r_{ji}(0).$$

После каждой итерации формируются оценки $\hat{c}_i = 1$ при $q_{i(1)} > q_{i(0)}$ (иначе 0), и проверяется $\hat{c}H^T = 0$: при успехе декодирование завершено.

10.3.5 ЛОПП-форма (алгоритм сумма-произведение)

Введем логарифмические отношения правдоподобия:

$$L(c_i) = \ln \frac{P\{c_i = 0 \mid y_i\}}{P\{c_i = 1 \mid y_i\}}, \quad L\big(r_{ji}\big) = \ln \frac{r_{ji}(0)}{r_{ji}(1)}, \quad L\big(q_{ij}\big) = \ln \frac{q_{ij}(0)}{q_{ij}(1)}.$$

Обновление символьных узлов становится суммой:

$$L\big(q_{ij}\big) = L(c_i) + \sum_{j' \in C_i \setminus \{j\}} L\big(r_{j'i}\big).$$

Для обновления проверочных узлов используем тождество $\tanh\big(\frac{1}{2}\ln(p_0/p_1)\big) = (p_0 - p_1)/(p_0 + p_1) = 1 - 2p_1$. Тогда $1 - 2r_{ji}(1) = \prod_{i' \in V_j \setminus \{i\}} \big(1 - 2q_{i'j}(1)\big)$ переписывается как

$$\tanh\left(\frac{1}{2}L\big(r_{ji}\big)\right) = \prod_{i' \in V_j \setminus \{i\}} \tanh\left(\frac{1}{2}L\big(q_{i'j}\big)\right),$$

то есть **произведение гиперболических тангенсов**: алгоритм называется сумма-произведение, поскольку символьные узлы суммируют ЛОПП, а проверочные перемножают тангенсы.

Разложив $L\big(q_{ij}\big) = a_{ij} b_{ij}$ на знак $a_{ij} = \operatorname{sgn}\big(L\big(q_{ij}\big)\big)$ и модуль $b_{ij} = \big|L\big(q_{ij}\big)\big|$, получаем

$$\tanh\left(\frac{1}{2}L\big(r_{ji}\big)\right) = \left(\prod_{i' \in V_j \setminus \{i\}} a_{i'j}\right) \prod_{i' \in V_j \setminus \{i\}} \tanh\left(\frac{1}{2} b_{i'j}\right),$$

откуда (произведение знаков эквивалентно операции XOR)

$$L\big(r_{ji}\big) = \left(\prod_{i' \in V_j \setminus \{i\}} a_{i'j}\right) \cdot 2\tanh^{-1}\left(\prod_{i' \in V_j \setminus \{i\}} \tanh\left(\frac{1}{2} b_{i'j}\right)\right).$$

10.3.6 Функция Галлагера и компактная форма

Функция Галлагера:

$$\varphi(x) = -\ln\tanh(x/2), \quad x > 0$$

— монотонно убывающая функция, $\varphi(0) = \infty$, $\varphi(x) \to 0$ при $x \to \infty$. Поскольку $2\tanh^{-1}(z) = \varphi^{-1}(-\ln z)$, обновление проверочного узла записывается компактно:

$$L(r_{ji}) = \left(\prod_{i' \in V_j \setminus \{i\}} a_{i'j}\right) \cdot \varphi^{-1}\left(\sum_{i' \in V_j \setminus \{i\}} \varphi(b_{i'j})\right).$$

10.3.7 Алгоритм передачи сообщений (сводка)

1. Вычислить канальные ЛОПП $L(c_i) = \ln \frac{p\{c_i=0 \mid y_i\}}{p\{c_i=1 \mid y_i\}}$.
2. Инициализация: $L(q_{ij}) \downarrow L(c_i)$ для всех ребер.
3. **Обработка проверочных узлов**: $L(r_{ji}) = \left(\prod_{i' \in V_j \setminus \{i\}} \operatorname{sgn}(L(q_{i'j}))\right) \varphi\left(\sum_{i' \in V_j \setminus \{i\}} \varphi(|L(q_{i'j})|)\right)$ для всех j и $i \in V_j$.
4. **Обработка символьных узлов**: $L(q_{ij}) = L(c_i) + \sum_{j' \in C_i \setminus \{j\}} L(r_{j'i})$.
5. **Оценка символов**: $L(q_i) = L(c_i) + \sum_{j' \in C_i} L(r_{j'i})$, $\hat{c}_i = 1$ при $L(q_i) < 0$, иначе 0.
6. Если $\hat{c}H^T \neq 0$ и лимит итераций не исчерпан, вернуться к шагу 3; иначе вернуть $\hat{c}$.

Трудоемкость итерации линейна по числу единиц в H, то есть по n при фиксированной плотности.

10.3.8 Упрощенный декодер min-sum

Так как φ монотонно убывает, $\varphi^{-1}\left(\sum_i \varphi(b_i)\right) \approx \min_i b_i$ (наибольший вклад в сумму дает наименьшее b_i). Отбрасывая остальные слагаемые, получаем **декодер min-sum** (минимум-суммы):

$$L(r_{ji}) \approx \left(\prod_{i' \in V_j \setminus \{i\}} a_{i'j}\right) \cdot \min_{i' \in V_j \setminus \{i\}} b_{i'j}.$$

Для канала с АБГШ $L(c_i) = 2y_i/\sigma^2$; множитель $2/\sigma^2$ можно исключить, так как он не влияет на знаки и порядки при min-sum. Более точная аппроксимация: $L(r_{ji}) \approx \left(\prod a_{i'j}\right)\left(\alpha \min_i b_{i'j} - \beta\right)$ с поправками α, β, подбираемыми для каждого кода.

10.3.9 Послойный алгоритм

При обработке очередного проверочного узла можно сразу использовать уточненные ЛОПП, полученные при обработке предыдущих узлов, — это ускоряет сходимость (алгоритм layered BP): значения L^c_{ji} хранятся и вычитаются/прибавляются к L^v_i до и после обработки каждого проверочного узла; проверка критерия остановки выполняется в начале каждого цикла итераций.

10.3.10 Субоптимальность: циклы в графе Таннера

Алгоритм распространения доверия предполагает статистическую независимость сообщений, передаваемых по графу Таннера. В реальных кодах это нарушается после нескольких итераций из-за **циклов** графа: сообщение возвращается к узлу, от которого оно зависимо. При построении кодов избегают коротких циклов (особенно длины 4), максимизируя **обхват** графа — длину кратчайшего цикла. Для очень длинных кодов влиянием циклов можно пренебречь. Отметим: не существует кодов с ациклическими графами Таннера скорости $R \geq 0.5$ и расстоянием $d > 2$; коды с $R < 0.5$, $d > 2$ и ациклическими графами получаются из таковых с $R \geq 0.5$, $d \leq 2$ повторением некоторых символов.

10.3.11 Блокирующие множества (stopping sets)

Некоторые наборы стертых символов не восстанавливаются итеративным алгоритмом, хотя восстанавливаются декодером максимума правдоподобия (методом Гаусса). **Блокирующее множество** — минимальный набор стертых символьных узлов, невосстановимый алгоритмом передачи сообщений. Свойства:

- блокирующее множество состоит из символьных узлов, каждый сосед которых подключен к множеству не менее чем двумя ребрами;

- всякое блокирующее множество содержит цикл, но не всякий цикл порождает блокирующее множество;
- преобразование строк проверочной матрицы изменяет набор блокирующих множеств;
- множество ненулевых позиций любого кодового слова (все его соседи подключены четное число раз) является блокирующим множеством; обратное неверно.

10.3.12 Множества-ловушки

Пусть дан граф Таннера G с множеством символьных узлов V, множеством проверочных узлов C и множеством ребер $E \subset V \times C$. Для $\tilde{V} \subset V$ **подграф, порожденный $\tilde{V}$**, содержит символьные узлы из $\tilde{V}$, все проверочные узлы, соединенные хотя бы с одним узлом $\tilde{V}$, и все соответствующие ребра.

> **Определение 30**. Множество символьных узлов $\tilde{V}$ называется **(a, b)-ловушкой**, если $|\tilde{V}| = a$ и в порожденном подграфе имеется b проверочных узлов нечетной степени.

Ошибки алгоритма распространения доверия, как правило, обусловлены наличием множеств-ловушек в графе Таннера: множество ошибочных решений удерживается, потому что каждая ошибка поддерживается нечетным числом неудовлетворенных проверок.

10.4 Эволюция плотностей

10.4.1 Идея

Если передавалось нулевое кодовое слово, канальные ЛОПП $L(c_i)$ — независимые случайные величины с некоторым распределением. Пример: для АБГШ $y_i = (-1)^{c_i} + \eta_i$, $\eta_i \sim N(0, \sigma^2)$, имеем $L(c_i) = 2y_i/\sigma^2 \sim N(2/\sigma^2, 4/\sigma^2)$. Алгоритм распространения доверия задает детерминированные преобразования случайных величин; вычислим распределения сообщений $L(r_{ji})$, $L(q_{ij})$ на каждой итерации. Вероятность ошибки декодирования тесно связана с $P\{L(q_{ij}) < 0\}$: если эта вероятность убывает с числом итераций, декодер сходится.

10.4.2 Преобразования случайных величин

Если X, Y — независимые случайные величины с плотностями $p_{X(x)}$, $p_{Y(y)}$, то плотность суммы — свертка:

$$p_{X+Y}(z) = (p_X \otimes p_Y)(z) = \int_{-\infty}^{\infty} p_{X(z-y)} p_{Y(y)} \, \mathrm{d}y,$$

где $\otimes$ — операция свертки. Для монотонно убывающей функции f: $P\{f(X) < x\} = P\{X > f^{-1}(x)\} = 1 - P_{X(f^{-1}(x))}$, откуда плотность

$$p_{f(X)}(x) = -p_{X(f^{-1}(x))}(f^{-1})'(x).$$

10.4.3 Распределение сообщений символьных узлов

Предположим, что длина кода стремится к бесконечности, графы усредняются по ансамблю с заданными $\lambda(x)$, $\rho(x)$, и входящие сообщения каждого узла независимы и одинаково распределены. Обозначения: $p^l_{C(x)}$ — плотность сообщений $L(r_{j'i})$ от проверочных узлов к символьным на итерации l; $p^l_{V(x)}$ — плотность сообщений $L(q_{j'i})$ от символьных к проверочным; $p_{L(x)}$ — плотность канальных ЛОПП $L(c_i)$.

Символьный узел степени t суммирует $t-1$ сообщений и канальное ЛОПП: плотность суммы — свертка $p_{C^{l-1}}^{\otimes(t-1)}$ (свертка $t-1$ экземпляров p_C^{l-1} с собой). Усреднение по распределению степеней (в терминах ребер, λ_t — вероятность, что случайное ребро ведет в узел степени t) дает

$$p_V^l = p_L \otimes \sum_{t \geq 1} \lambda_t p_{C^{l-1}}^{\otimes(t-1)} = p_L \otimes \lambda(p_C^{l-1}),$$

где полином λ действует на функцию-плотность как оператор смеси сверток.

10.4.4 Распределение сообщений проверочных узлов

Обновление проверочного узла обрабатывает знаки и модули одновременно, поэтому вводится **двумерное преобразование** ЛОПП:

$$\Gamma\big(L(r_{i'j})\big) = \Big(1_{L(r_{i'j})<0}, \varphi\big(|L(r_{i'j})|\big)\Big),$$

где 1_X — индикаторная функция (1 при выполнении условия X, иначе 0). Тогда

$$L(r_{ji}) = \Gamma^{-1}\left(\sum_{i'\in V_j\setminus\{i\}} \Gamma\big(L(q_{i'j})\big)\right)$$

— сумма двумерных величин (знаки складываются по модулю 2, положительные части суммируются арифметически). Функция распределения такой величины:

$$G(s,x) = 1_{s=0}P\big\{L(q_{i'j}) > \varphi(x)\big\} + 1_{s=1}P\big\{L(q_{i'j}) < -\varphi(x)\big\}, \quad s\in\{0,1\}, x\geq 0,$$

с плотностью $g(s,x) = 1_{s=0}g_0(x) + 1_{s=1}g_1(x)$. Сумма двух величин такого вида с плотностями g и f имеет плотность

$$h(s,x) = 1_{s=0}(g_0\otimes f_0 + g_1\otimes f_1)(x) + 1_{s=1}(g_1\otimes f_0 + g_0\otimes f_1)(x).$$

Пусть $\gamma\big(p_V^l\big)$ — преобразование плотности сообщений символьных узлов в пару (g_0, g_1). Проверочный узел степени i (в терминах ребер, с вероятностью ρ_i) суммирует $i-1$ слагаемых, поэтому

$$p_C^l = \gamma^{-1}\left(\sum_{i\geq 1}\rho_i\big(\gamma\big(p_V^l\big)\big)^{\otimes(i-1)}\right) = \gamma^{-1}\big(\rho\big(\gamma\big(p_V^l\big)\big)\big).$$

10.4.5 Рекурсия эволюции плотностей

$$p_V^l = \begin{cases} p_L \\ l = 0 \\ p_L \otimes \lambda\big(\gamma^{-1}\rho\big(\gamma\big(p_V^{l-1}\big)\big)\big) \\ l > 0 \end{cases}.$$

Вероятность ошибки (доля ошибочных сообщений) на итерации l:

$$P_e^l = \int_{-\infty}^{0} p_{V(x)}^l \,\mathrm{d}x.$$

При достаточно слабых ограничениях верны утверждения:

- последовательность p_V^l сходится к неподвижной точке, зависящей только от начального распределения p_L; $\lim_{l\to\infty} P_e^l = 0$ тогда и только тогда, когда p_V^l сходится к дельта-функции Дирака в точке $+\infty$;
- если $\lambda'(0)\rho'(1) > e/r$, где $r = -\ln\Big(\int_{-\infty}^{\infty} p_{L(x)}e^{-x/2}\,\mathrm{d}x\Big)$, то существует $\xi > 0$ такое, что $P_e^l > \xi$ при всех l — декодер не сходится;
- если $\lambda'(0)\rho'(1) < e/r$, то существует $\xi > 0$ такое, что из $P_e^l \leq \xi$ для некоторого l следует $P_e^{l'} \to 0$ при $l'\to\infty$ — попадание в окрестность решения гарантирует сходимость.

10.4.6 Порог итеративного декодирования

Пусть качество канала однозначно характеризуется параметром δ, возрастающим при ухудшении канала (например, дисперсия шума). Ключевой результат: для всех $\varepsilon > 0$ и $\delta < \delta^*$ существуют $n(\varepsilon,\delta)$ и $l(\varepsilon,\delta)$ такие, что для **почти всех** кодов (λ,ρ)-ансамбля длины $n > n(\varepsilon,\delta)$ вероятность ошибки P_e^l после $l(\varepsilon,\delta)$ итераций не превышает ε. «Почти все» означает, что доля кодов без этого свойства экспоненциально мала с ростом n.

Величина δ^* называется **порогом** итеративного декодирования. Для каждого δ сходимость проверяется запуском эволюции плотностей до $P_e^{l_0} < \varepsilon$; порог δ^* ищется дихотомией (двоичным поиском). δ^* — нелинейная невыпуклая функция от $\lambda(x)$, $\rho(x)$, поэтому оптимизация

$$\delta^*(\lambda, \rho, p_L) \to \max, \quad \text{при } \lambda(1) = \rho(1) = 1, \quad R \geq 1 - \frac{\int_0^1 \rho(x)\,\mathrm{d}x}{\int_0^1 \lambda(x)\,\mathrm{d}x}$$

ведется генетическими алгоритмами, методом отжига и т.п. (Richardson, Shokrollahi, Urbanke).

10.4.7 Предостережения

- реальные графы Таннера имеют циклы, независимость сообщений нарушается, и формулы декодера и эволюции плотностей становятся неточными;
- эволюция плотностей дает результаты, усредненные по ансамблю; конкретный граф ансамбля может вести себя существенно хуже;
- оптимизированные распределения $\lambda(x)$, $\rho(x)$ требуют доработки для практического применения;
- вероятность ошибки распространения доверия не может быть меньше вероятности ошибки декодирования по максимуму правдоподобия, которая определяется минимальным расстоянием и никак не учитывается в эволюции плотностей;
- нерегулярные коды, оптимизированные описанным методом, хороши при малых ОСШ, но с ростом ОСШ скорость убывания вероятности ошибки может резко снизиться — **эффект полки** (error floor), связанный с малыми расстояниями и ловушками.

10.5 Конструкции кодов

10.5.1 Конструкция Галлагера

Проверочная матрица составляется горизонтально из g блоков:

$$H = \begin{bmatrix} H_1 & H_2 & \dots & H_g \end{bmatrix},$$

где H_1 — матрица размера $m \times n$, $n = mr$, содержащая единицы в позициях $ir, \dots, (i+1)r - 1$ каждой строки i (разбиение на m последовательных групп по r столбцов), а остальные блоки получаются из H_1 перестановками столбцов. При $g \geq 3$ минимальные расстояния достаточно хороши. Коды регулярны; при подборе перестановок нужно избегать циклов длины 4 в графе Таннера.

10.5.2 Блочно-циркулянтные коды

Проверочная матрица состоит из блоков размера $p \times p$ — матриц циклического сдвига $I(a)$ (единичная матрица с циклическим сдвигом строк на a позиций); блоки с $a = 0$ — просто единичные. Такая структура удобна для аппаратной реализации. Узлы образуют цикл длины $2s$, если выполняется необходимое и достаточное условие

$$\sum_{k=0}^{s-1} \left(p_{j_k, l_k} - p_{j_{k+1}, l_k} \right) \equiv 0 \bmod p,$$

где $p_{j,l}$ — сдвиг блока на пересечении j-й группы строк и l-й группы столбцов; проверяя суммы сдвигов, можно строить коды без коротких циклов.

10.5.3 Прогрессивное наращивание ребер (PEG)

Итеративный жадный алгоритм построения графа Таннера: сначала по заданным $\lambda(x)$, $\rho(x)$ вычисляется число узлов каждой степени; создается пустой граф; для каждого символьного узла требуемой степени (обрабатывая узлы в порядке возрастания степени) ребра добавляются i раз, причем на каждом шаге выбирается проверочный узел с наименьшей текущей степенью, добавление ребра к которому порождает цикл максимально возможной длины; неоднозначности разрешаются случайно. Начало с узлов наименьшей степени позволяет максимизировать обхват циклов, проходящих через них. Коды не имеют алгебраической структуры, но обладают хорошими свойствами графа.

10.5.4 Протографы

Берется небольшой **протограф** — код с малым числом узлов — и «масштабируется»: создаются Q копий протографа, а ребра между одноименными узлами копий «переключаются». На уровне матриц каждая единица проверочной матрицы протографа заменяется $Q \times Q$ перестановочной матрицей (как правило, циклического сдвига на l_{ij} позиций, причем сдвиги масштабируются пропорционально Q), а нуль — нулевой матрицей той же размерности. Протографы дают структурированные коды с высокой плотностью единиц в малом протографе и большой длиной циклов; применяются в стандарте 5G NR (базовые графы BG 1 и BG 2).

10.5.5 Сравнение корректирующей способности

Моделирование LDPC-кодов 5G (BG 2, (1296, 720); BG 1, (8256, 4224)) с 10–60 итерациями в сравнении с полярными подкодами равной длины (последовательное списочное декодирование, размеры списка до 2^{11}) показывает: при коротких длинах сравнимая корректирующая способность достигается сопоставимой сложностью; с ростом длины LDPC с итеративным декодированием остаются практичным эталоном.

10.6 Выводы

- МППЧ — класс линейных блоковых кодов с разреженными проверочными матрицами и широким диапазоном параметров; декодирование распространения доверия имеет линейную сложность.
- МППЧ допускают асимптотический анализ (эволюция плотностей), оптимизацию распределений степеней и достижение порога, сколь угодно близкого к пропускной способности канала.
- Оценка минимального расстояния LDPC-кодов затруднена; на практике проявляется эффект полки, а гарантии по расстоянию дают только специальные конструкции.

11 Коды для стирающего канала

11.1 Постановка задачи

11.1.1 IP-сеть как стирающий канал

В IP-сетях происходят потери пакетов. Если нужна надежная доставка, потерянные пакеты передаются заново, но у перепосылок есть недостатки:

- протокол TCP интерпретирует потери пакетов как признак перегрузки сети и снижает скорость передачи;
- на каналах с большим произведением «скорость-задержка» (long fat pipe) возможны простои и избыточные перепосылки;
- при широковещательной передаче (broadcast, multicast) перепосылки невозможны или затруднены: разные пользователи нуждаются в повторной передаче разных пакетов, и объем служебного трафика резко растет с числом пользователей.

Классические приемы (карусель данных, иерархические перепосылки) этих проблем не решают. Поэтому IP-сеть моделируется как **двоичный стирающий канал**: пакет либо доставлен верно, либо потерян (позиция потери известна приемнику).

11.1.2 Теоретический предел

Пропускная способность q-ичного стирающего канала с вероятностью стирания p равна $C = 1 - p$ символов на использование канала. Значит, если данные занимают k символов и надлежащим образом закодированы, при $k \to \infty$ достаточно получить **любые** k символов бесконечного потока кодовых символов, чтобы восстановить данные.

11.1.3 Коды Рида—Соломона в стирающем канале

Код Рида—Соломона $\mathrm{RS}(n, k)$ — множество векторов $(f(x_0), ..., f(x_{n-1}))$ по значениям многочленов степени меньше k над $\mathrm{GF}(q)$ в различных точках x_i. Любые k принятых символов образуют

информационную совокупность: по точкам $\left(x_{i_1}, f\left(x_{i_1}\right)\right), \ldots, \left(x_{i_k}, f\left(x_{i_k}\right)\right)$ интерполяцией Лагранжа восстанавливается $f(x)$, а с ней и всё кодовое слово:

$$f(x) = \sum_{j=1}^{k} f\left(x_{i_j}\right) \prod_{t \neq j} \frac{x - x_{i_t}}{x_{i_j} - x_{i_t}}.$$

Недостатки кодов РС в стирающем канале:

- если потеряно более $n - k$ символов, дополнительные проверочные символы взять негде; повторная передача того же кодового слова дает приемнику копии уже принятых символов, что бесполезно;
- длина ограничена: $n \leq q + 1$ (точки должны быть различны в поле $\mathrm{GF}(q)$ или расширении), нарастить длину нельзя;
- при высоком уровне потерь коды РС неоптимальны;
- сложность интерполяции $O(k^2)$, требуются операции в большом поле $\mathrm{GF}(q)$.

11.1.4 Цифровой фонтан

Идея **цифрового фонтана**: передатчик из k информационных символов генерирует бесконечную последовательность кодовых символов; любой пользователь, собравший **любые** k из них (плюс небольшой запас), восстанавливает данные. Код имеет потенциально бесконечную длину; такие коды называют **бесскоростными** (rateless). Приемник может присоединиться к передаче в любой момент и не сообщает передатчику, какие именно символы он принял, — обратная связь не нужна.

11.1.5 Обратимость случайных матриц

Декодирование по k принятым символам $y = xG$ возможно, если соответствующая подматрица порождающей матрицы обратима. Оценим вероятность обратимости случайной $k \times k$ двоичной матрицы A: i-я строка не должна попасть в линейную оболочку $i - 1$ предыдущих, что дает вероятность $1 - 2^{i-1}/2^k$, и

$$P(k) = \prod_{i=1}^{k} \left(1 - 2^{i-k-1}\right) \underset{(k \to \infty)}{\to} \prod_{i=1}^{\infty} (1 - 2^{-i}) \approx 0.2888.$$

Даже случайные плотные матрицы обратимы лишь в 29% случаев, поэтому подматрицы должны выбираться конструктивно.

11.1.6 Задача о мячах и корзинах

Модель передачи без кодирования: k информационных пакетов («корзины»), каждый переданный пакет случайно выбирается среди k («мяч падает в любую корзину равновероятно»). Вероятность, что корзина пуста после N бросков:

$$\left(1 - \frac{1}{k}\right)^N \approx e^{-N/k},$$

вероятность, что не осталось пустых корзин:

$$\left(1 - e^{-N/k}\right)^k \approx 1 - k e^{-N/k} = 1 - \delta,$$

откуда для гарантии приема всех пакетов с вероятностью $1 - \delta$ нужно $N \approx k \ln(k/\delta)$ принятых пакетов — в $\ln(k/\delta)$ раз больше предела Шеннона (k). Пример: $k = 1000$, $\delta = 10^{-3}$ — тринадцатикратная потеря эффективности. Нужен код, работающий в $1 + \varepsilon$ раз от предела.

11.2 LT-коды

11.2.1 Кодирование: преобразование Луби

LT-код (Luby Transform) задается над k пакетами одинаковой длины. Кодирование (преобразование Луби) повторяется нужное число раз:

1. выбрать целое число d в соответствии с распределением $\rho(d)$;
2. сложить (по модулю 2) d случайно выбранных информационных пакетов и передать сумму вместе с описанием, какие пакеты складывались.

Кодовые символы порождаются независимо; длина кода не ограничена. Порождающая матрица разрежена: каждый кодовый символ — сумма d информационных.

Условия декодируемости:

- **необходимое**: каждый информационный пакет должен входить хотя бы в один принятый кодовый пакет; всего слагаемых (с повторами) должно быть $N \approx k \ln(k/\delta)$, что с запасом выполняется при разумном ρ; вероятность декодируемости $1 - \delta$;
- **достаточное**: обратимость подматрицы порождающей матрицы, соответствующей принятым символам.

11.2.2 Декодирование: волна

Каждый принятый пакет знает свой состав. Строится двудольный граф: символьные узлы (принятые кодовые пакеты) и информационные узлы; ребро указывает, что информационный пакет входил в слагаемые кодового. Алгоритм:

1. выбрать символьный узел степени 1 (его значение — сразу значение одного информационного пакета); символьные узлы степени 1 называются **активными**, их множество — **волной декодирования**;
2. восстановить соответствующий информационный пакет;
3. вычесть его из всех связанных с ним кодовых пакетов и удалить его узел с ребрами из графа; степени смежных символьных узлов уменьшаются на 1, порождая новую волну;
4. повторять, пока волна не иссякнет.

Волна прокатывается по графу, восстанавливая по одному информационному пакету за шаг. Важно: итеративный алгоритм восстанавливает **не все** исправимые конфигурации стираний (в отличие от решения системы методом Гаусса).

11.2.3 Принципы оптимизации распределения степеней

Распределение $\rho(d)$ — вероятность того, что кодовый символ равен сумме ровно d информационных; при независимых потерях пакетов распределение степеней **принятых** символов совпадает с ρ. Цель — восстановление всех k пакетов по минимально возможному числу K принятых символов. Требования (отчасти противоречивые):

- волна декодирования должна быть достаточно велика, чтобы декодер не застрял, но как можно меньше во избежание дублей (минимизация избыточности $K - k$);
- степени кодовых символов должны быть минимально возможными для снижения сложности кодирования и декодирования.

11.2.4 Идеальное волновое (солитонное) распределение

Каждый информационный символ должен быть использован хотя бы раз; общее число слагаемых $O(k \ln k)$ при $O(k)$ закодированных символах дает среднюю степень символьного узла $\sum_{d=1}^{k} d\rho(d) \approx \ln k$. **Идеальное волновое распределение**:

$$\rho(d) = \begin{cases} \frac{1}{k} & d = 1 \\ \frac{1}{d(d-1)} & 1 < d \le k \end{cases}, \quad \sum_{d=1}^{k} \rho(d) = \frac{1}{k} + \sum_{d=2}^{k} \left(\frac{1}{d-1} - \frac{1}{d} \right) = 1.$$

Название объясняется аналогией с солитоном — структурно устойчивой уединенной волной: размер волны декодирования должен оставаться постоянным на протяжении всего декодирования.

11.2.5 Анализ декодера LT-кода

На каждом шаге восстанавливается ровно один информационный символ; успешное декодирование заканчивается за k шагов. Пусть $q(i, L)$ — вероятность того, что заданный символьный узел степени

i **активируется** (его степень падает до 1), когда осталось продекодировать L информационных символов, то есть на шаге $k-L$. При $2 \le i \le k$, $k-i+1 \ge L \ge 1$ активация означает: $i-2$ соседей узла находятся среди $k-L-1$ ранее восстановленных, один сосед восстанавливается на текущем шаге, и один остается среди L невосстановленных:

$$q(i,L) = \frac{\binom{i-2}{k-L-1}\binom{1}{L}}{\binom{i}{k}} = \frac{i(i-1)L\prod_{j=0}^{i-3}(k-L-1-j)}{\prod_{j=0}^{i-1}(k-j)};$$

в остальных случаях $q(i,L)=0$; при $i=1$: $q(1,k)=1$. Тогда $r(i,L)=\rho(i)q(i,L)$ — вероятность того, что узел имеет степень i и активируется при остатке L, а

$$r(L) = \sum_i r(i,L) = \sum_i \rho(i)q(i,L)$$

— вероятность активации произвольного узла при остатке L.

11.2.6 Анализ идеального волнового распределения

> **Теорема 49**. Для идеального волнового распределения $\rho(i)=1/k$ при $i=1$ и $\rho(i)=1/(i(i-1))$ при $2 \le i \le k$ вероятность активации на каждом шаге постоянна: $r(L)=1/k$.

Доказательство. При $L=k$: $r(k)=\rho(1)q(1,k)=1/k$. При $L<k$ подставляем ρ:

$$r(L) = \sum_{i=2}^{k} \frac{i(i-1)L\prod_{j=0}^{i-3}(k-L-1-j)}{\prod_{j=0}^{i-1}(k-j)} \cdot \frac{1}{i(i-1)} = \frac{L}{\prod_{j=0}^{k-1}(k-j)} \sum_{i=2}^{k} \left(\frac{\prod_{j=0}^{i-3}(k-L-1-j)}{\prod_{j=i}^{k-1}(k-j)} \right) = \frac{L}{k!} \sum_{i=2}^{k-L+1} \frac{(k-i)!(k-L-1)!}{(k-L-i+1)!}.$$

Замена $j=k-L-i+1$ переводит сумму в пределы $0 \le j \le k-L-1$, причем $k-i=L+j-1$:

$$\sum_{j=0}^{k-L-1} \frac{(L+j-1)!}{j!} = \frac{L(k-L-1)!(L-1)!}{k!} \sum_{j=0}^{k-L-1} \binom{L-1+j}{j} = \frac{L(k-L-1)!(L-1)!}{k!} \binom{k-1}{k-L-1} = \frac{L(L-1)!(k-L-1)!}{k!} \cdot \frac{(k$$

где использовано тождество «швабры»: $\sum_{j=0}^{M}\binom{L-1+j}{j} = \binom{L+M}{M}$ при $M=k-L-1$. □

Если принято ровно k символов, на каждом шаге в среднем активируется $r(L)k=1$ узел — волна не гаснет, декодирование успешно **в среднем**. Но математическое ожидание, равное 1, допускает и значения меньше 1: случайные отклонения могут погасить волну, поэтому идеальное распределение на практике неустойчиво.

11.2.7 Устойчивое волновое распределение

Чтобы с вероятностью $1-\delta$ волна не упала до нуля, создается запас кодовых символов степени 1. **Устойчивое волновое распределение**:

$$\mu(d) = \frac{\rho(d)+\tau(d)}{\beta}, \quad \tau(d) = \begin{cases} R/(dk) & 1 \le d \le \frac{k}{R}-1 \\ R\ln(R/\delta)/k & d = \frac{k}{R} \\ 0 & \frac{k}{R}+1 \le d \le k \end{cases},$$

где $R = c\ln(k/\delta)\sqrt{k}$ с константой $c \approx 1$, а $\beta = \sum_{i=1}^{k}(\rho(i)+\tau(i))$ — нормировка. Добавка τ повышает долю символов степени 1 и добавляет немного символов степени $\frac{k}{R}$ (связывающих разные части данных).

> **Теорема 50** (Луби). При использовании устойчивого распределения восстановление данных по любому множеству из $K > k$ принятых символов может быть выполнено с вероятностью $1-\delta$, где избыточность

$$\frac{K-k}{k} \sim \ln^2 \frac{k/\delta}{\sqrt{k}} \underset{(k\to\infty)}{\to} 0.$$

Избыточность стремится к нулю с ростом k, хотя и медленнее, чем у идеального кода: декодирование требует $K = k + O\left(\ln^2(k/\delta)\sqrt{k}\right)$ символов.

11.3 Хищные коды (Raptor)

Для сравнительно небольших k и малых δ избыточность LT-кодов (порядка $\ln^2(k/\delta)\sqrt{k}$) слишком велика. **Хищные коды** (RAPid TORnado codes, Raptor) добавляют внешний код:

1. перед преобразованием Луби данные кодируются линейным блоковым кодом (например, БЧХ или МППЧ), способным исправлять большое число стираний (**внешний код**);
2. итеративный LT-декодер может отказаться от восстановления некоторого числа символов данных; эти символы восстанавливаются декодером внешнего кода;
3. поскольку LT-декодер не восстанавливает многие исправимые конфигурации стираний, вместо итеративного алгоритма применяется декодирование по максимуму правдоподобия — решение системы $uGA = y$, где G — порождающая матрица внешнего кода, A — матрица преобразования Луби; для сохранения разреженности A используется модификация метода Гаусса.

RaptorQ: дальнейшее улучшение за счет введения в преобразование Луби небольшого числа коэффициентов из $\mathrm{GF}(2^m)$, $m > 1$. Хищные коды включены в стандарт мобильной связи LTE (4G) и широко используются в системах доставки контента.

11.4 Выводы

- IP-сеть естественно моделируется как стирающий канал; перепосылки (TCP) при широковещательной доставке и больших задержках неэффективны.
- Коды Рида—Соломона решают задачу лишь частично: ограничение длины $n \le q+1$, сложность $O(k^2)$ и бесполезность повторов делают их неоптимальными при высоких потерях.
- LT-коды реализуют цифровой фонтан: простое кодирование преобразованием Луби, линейное итеративное декодирование, избыточность, стремящаяся к нулю (теорема Луби).
- Хищные коды (Raptor, RaptorQ) сочетают LT-преобразование с внешним кодом и декодированием по максимуму правдоподобия, доводя избыточность до приемлемых значений при малых k; они стандартизированы в LTE.

12 Кодовая модуляция

12.1 Теоретико-информационные основы

12.1.1 Пропускная способность канала без памяти

Пропускная способность канала без памяти с входным алфавитом $\mathcal{X}$ и выходным алфавитом $\mathcal{Y}$:

$$C = \max_{p_X} \int_{\mathcal{Y}} \int_{\mathcal{X}} p_{XY}(x,y) \log_2 \frac{p_{XY}(x,y)}{p_{X(x)} p_{Y(y)}} \,\mathrm{d}x \,\mathrm{d}y,$$

где X, Y — случайные величины, соответствующие входу и выходу канала; $p_{XY}(x,y)$ — совместное распределение; $p_{X(x)}$ — распределение передаваемых символов, по которому ведется максимизация. Если X или Y дискретны, соответствующий интеграл заменяется суммой. $p_{Y|X}(y|x) = p_{XY}(x,y)/p_{X(x)}$ — плотность выхода канала при заданном входе (для непрерывного Y) или функция переходных вероятностей (для дискретного Y). **Прямая теорема кодирования**: существуют коды достаточно большой длины, обеспечивающие передачу по каналу $p_{Y|X}(y|x)$ со скоростью меньше C и сколь угодно малой вероятностью ошибки.

12.1.2 Пропускная способность некоторых каналов

Обозначим p_X^* ∗ оптимальное распределение входа.

- **Двоичный симметричный канал**: $\mathcal{X} = \mathcal{Y} = \{0, 1\},\ p_{Y|X}(y|x) = p^{|y-x|}\,(1-p)^{1-|y-x|}$; оптимально равномерное распределение $p_X^* * = 1/2$, и

$$C = 1 + p\log_2 p + (1-p)\log_2(1-p).$$

- **АБГШ с двоичной амплитудно-импульсной модуляцией**: $\mathcal{X} = \{-1, 1\}$, $\mathcal{Y} = \mathbb{R}$,

$$p_{Y|X}(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-x)^2}{2\sigma^2}}, \quad p_X^* * = \frac{1}{2}, \quad C = \int_{-\infty}^{\infty} p_{Y|X}(y|0)\log_2 \frac{2p_{Y|X}(y|0)}{p_{Y|X}(y|0) + p_{Y|X}(y|1)}\,\mathrm{d}y.$$

- **АБГШ с непрерывным входом** $\mathcal{X} = \mathbb{R}$ и ограничением на среднюю мощность $M\big[X^2\big] = \sigma_X^2$ (оптимален гауссовский вход):

$$C = \frac{1}{2}\log_2\left(1 + \frac{\sigma_X^2}{\sigma^2}\right), \quad p_X^* * (x) = \frac{1}{\sqrt{2\pi\sigma_X^2}} e^{-x^2/2\sigma_X^2}.$$

12.2 Методы модуляции

Реализовать $\mathcal{X} = \mathbb{R}$ на практике сложно; используют конечные алфавиты мощности $|\mathcal{X}| = M = 2^t$ (t бит на символ):

- **Амплитудно-импульсная модуляция (АИМ)**: $\mathcal{X}_{[\mathrm{AM}]} = \{d(2j + 1 - M) \mid j = 0, 1, ..., M-1\}$, где шаг решетки $d = \sigma_X\sqrt{3/(M^2-1)}$ подбирается так, чтобы средняя мощность равнялась σ_X^2;
- **Фазовая модуляция (ФМ, PSK)**: $\mathcal{X}_{[\mathrm{PSK}]} = \{\cos(2\pi j/M) - i\sin(2\pi j/M)\}$ — точки на окружности единичного радиуса;
- **Квадратурно-амплитудная модуляция (КАМ, QAM)**: решетка

$$\mathcal{X}_{[\mathrm{QAM}]} = \Big\{d\big(2j' + 1 - \sqrt{M}\big) + id\big(2j'' + 1 - \sqrt{M}\big) \mid j', j'' = 0, 1, ..., \sqrt{M} - 1\Big\}.$$

Кривые пропускной способности при фиксированном ОСШ E_s/N_0 показывают: чем больше M, тем ближе ограниченный алфавит к неограниченному гауссовскому каналу, но проигрыш малых алфавитов заметен лишь при высоких ОСШ; ДСК дает наихудшую кривую.

12.2.1 Шейпинг

Энтропия дискретной случайной величины X: $H(X) = -\sum_x p_{X(x)}\log_2 p_{X(x)}$ — сколько бит информации дает одна реализация. Задача шейпинга: при заданной скорости $R = H(X)$ бит на символ и АИМ-алфавите минимизировать среднюю энергию $M\big[X^2\big]$. Оптимальное распределение — **Максвелла—Больцмана** (дискретная гауссовская):

$$p_{X(x)} = Ke^{-\alpha x^2}, \quad x \in \mathcal{X},$$

где α определяется скоростью R, K — нормировочная константа. Шейпинг дает энергетический выигрыш до $\pi e/6 \approx 1.53$ дБ и применяется в оптоволоконных системах. Из-за сложности реализации и чувствительности к флуктуациям канала шейпингом часто пренебрегают, используя равномерное распределение передаваемых символов.

12.3 Битоперемеженная кодовая модуляция

12.3.1 Код Грея

Предположим, шейпинг не используется: символы распределены равномерно и однозначно определяются данными. Тогда элементам сигнального множества нужно приписать битовые метки так, чтобы типовые ошибки канала минимально искажали данные. Наиболее вероятное ошибочное событие в АБГШ — замена переданного символа ближайшим в метрике Евклида. ***q*-ичный код Грея** длины t — последовательность различных q-ичных векторов, в которой смежные элементы различаются ровно в одной позиции; метки соседних по сигнальному множеству символов отличаются одним битом, и типовая ошибка канала портит один бит метки.

12.3.2 Схема битоперемеженной кодовой модуляции

1. разметить 2^t-ичное сигнальное множество кодом Грея;
2. закодировать данные двоичным линейным блоковым кодом длины $n = n't$;
3. переставить символы кодового слова предопределенным образом (перемежение);
4. отобразить группы битов $c_{it}, ..., c_{it+t-1}$ на один символ сигнального множества.

Зачем перемежитель: вероятности ошибки разных битов метки различаются, и события ошибок битов одного символа зависимы. Перестановка должна исключить отображение плохо защищенных символов кодового слова на ненадежные биты меток либо обеспечить равновероятное назначение надежных и ненадежных битов; перестановка допускает оптимизацию с учетом структуры кода. Если код достаточно «случаен», перестановку можно исключить. Схема проста, но не позволяет достичь предела Шеннона.

12.3.3 Демодуляция и ЛОПП

Демодулятор вычисляет логарифмические отношения правдоподобия отдельных битов метки. Пусть $\mathcal{X}_{i,b}$ — подмножество сигнального множества, содержащее символы, i-й бит метки которых равен b:

$$L_i = \ln \frac{P\{c_i = 0 \mid y\}}{P\{c_i = 1 \mid y\}} = \ln \frac{\sum_{x \in \mathcal{X}_{i,0}} p_{X|Y}(x|y)}{\sum_{x \in \mathcal{X}_{i,1}} p_{X|Y}(x|y)} \approx \ln \frac{\max_{x \in \mathcal{X}_{i,0}} p_{X|Y}(x|y)}{\max_{x \in \mathcal{X}_{i,1}} p_{X|Y}(x|y)}.$$

Приближение max-log заменяет суммирование по гипотезам на максимум правдоподобия; график зависимости $L_{i(y)}$ от принятого значения y кусочно-непрерывен и знакопеременен по областям решения. Свойства: различные биты метки имеют различные вероятности ошибки, и события ошибок в разных битах одного символа зависимы — эти эффекты учитываются перемежителем и кодом.

12.4 Многоуровневые коды

12.4.1 Определение

Многоуровневые коды — обобщенные каскадные коды (Блох, Зяблов, Зиновьев) с **нелинейными** внутренними кодами длины 1: внутренний «код» $B_1 = \mathcal{X}$, и кодеры внутренних кодов отображают символы полей $\mathrm{GF}(q)$ на подмножества сигнального множества $\mathcal{X}$; конкретный внутренний код B_{i+1} определяется результатом кодирования B_i.

- внешние (N, K_i, D_i) коды A_i над полями $\mathrm{GF}(q^{m_i})$, $1 \le i \le s$;
- вложенные внутренние коды $B_1 \sup B_2 \sup \ldots \sup B_s$ (подмножества сигнального множества), $k_i - k_{i+1} = m_i$.

Кодирование:

1. закодировать данные внешними кодами и записать кодовые слова в $s \times N$ матрицу X;
2. заменить элементы i-й строки X их векторным представлением (столбцы длины m_i), получив $k_1 \times N$ матрицу Y;
3. умножить каждый столбец Y на порождающую матрицу B (кодирование в коде B_1);
4. полученная $n \times N$ матрица — кодовое слово: линейный $\left(Nn, \sum_{i=1}^{s} K_i m_i, \geq \min_i d_i D_i\right)$ код над $\mathrm{GF}(q)$, где d_i — минимальные расстояния Хемминга внутренних «кодовых» разбиений.

Каждый внешний код отвечает за один **уровень** разбиения сигнального множества. Пример для $\mathcal{X} = \{-7, -5, -3, -1, 1, 3, 5, 7\}$:

$$\mathcal{X} = B_2^0 \cup B_2^1, \quad B_2^0 = \{-7, -3, 1, 5\}, \quad B_2^1 = \{-5, -1, 3, 7\}, \quad B_2^0 = \{-7, 1\} \cup \{-3, 5\}, \quad B_2^1 = \{-5, 3\} \cup \{-1, 7\},$$

и символы кодовых слов i-го внешнего кода формируют i-й бит метки. Разметка подмножеств влияет на свойства кода; чаще всего используется **разметка Унгербёка**, максимизирующая минимальное расстояние Евклида каждого вложенного внутреннего кода.

12.4.2 Многошаговое декодирование

Декодирование последовательное по уровням: на j-м шаге приемник уже знает решения $\hat{c}_0, ..., \hat{c}_{j-1}$ предыдущих уровней и вычисляет ЛОПП текущего уровня, ограничивая гипотезы подмножеством сигнального множества, совместимым с ними:

$$L_i^j = \ln \frac{\sum_{x \in \mathcal{X}(\hat{c}_{0,i}, ..., \hat{c}_{j-1,i}, 0)} p_{X|Y}(x \mid y_i)}{\sum_{x \in \mathcal{X}(\hat{c}_{0,i}, ..., \hat{c}_{j-1,i}, 1)} p_{X|Y}(x \mid y_i)} \approx \ln \frac{\max_{x \in \mathcal{X}(..., 0)} p_{X|Y}(x|y_i)}{\max_{x \in \mathcal{X}(..., 1)} p_{X|Y}(x|y_i)},$$

где $\mathcal{X}(a_0, ..., a_j)$ — подмножество $\mathcal{X}$ с младшими битами меток, равными $a_0, ..., a_j$. Ошибка декодирования на одном из шагов не может быть исправлена на последующих — прямая аналогия с алгоритмом последовательного исключения полярных кодов.

12.4.3 Построение многоуровневых кодов

- кодовые слова j-го уровня передаются по **виртуальному битовому подканалу**; скорость внешнего кода уровня не должна превышать пропускную способность этого подканала;
- если внешние коды достигают предела Шеннона, то и многоуровневый код достигает его (при длине, стремящейся к бесконечности);
- квадрат минимального расстояния Евклида многоуровневого кода $d \geq \min_j \left(d_j^2 D_j \right)$, где d_j^2 — квадрат минимального расстояния Евклида внутреннего кода B_j, D_j — минимальное расстояние Хемминга внешнего кода уровня j; следует обеспечивать $d_j^2 D_j \approx \text{const}$ по уровням;
- если скорость внешнего кода превышает пропускную способность своего подканала, многошаговое декодирование неудовлетворительно; правило **равных вероятностей ошибки** — вероятности ошибки компонентных кодов на разных шагах должны быть примерно одинаковыми; подбор требует многочисленных экспериментов.

12.4.4 Шейпинг с помощью многоуровневых кодов

Часть информационных символов внешних кодов может подбираться на передатчике так, чтобы минимизировать энергию получившегося кодового слова. Пусть $x^{(j)}$ — информационные символы уровня j с полезными данными, $z^{(j)}$ — свободные символы: кодовые слова внешних кодов

$$c^{(j)} = x^{(j)} G_j + z^{(j)} \tilde{G}_j, \quad G_j = \begin{bmatrix} G_j \\ \tilde{G}_j \end{bmatrix}$$

— порождающая матрица кода A_j (систематическая часть и шейпинг-часть $\tilde{G}_j$). Внешние коды превращаются в смежные классы кодов $\tilde{A}_j$, порождаемых $\tilde{G}_j$. Шейпинг осуществляется запуском многошагового декодера **на стороне передатчика**: сначала выбираются z уровней с наименьшей энергией, затем данные передаются. При расчете ЛОПП на приемнике неравномерное распределение символов кодового слова учитывается заменой $p_{X|Y}$ на взвешенное произведение $p_{X(x)} p_{X|Y}(x|y)$ — графики L^j заметно отличаются от случая равномерного распределения, что и дает энергетический выигрыш.

12.5 Выводы

- Высокоскоростная передача данных требует кодовой модуляции — совместного проектирования кода и сигнального множества.
- **Битоперемеженная кодовая модуляция**: простота реализации, но не достигает предела Шеннона.
- **Многоуровневые коды**: могут достигать предела Шеннона; построение кода нетривиально, многошаговый декодер не обеспечивает декодирование по максимуму правдоподобия и чувствителен к изменению характеристик канала.
- **Шейпинг** дает энергетический выигрыш до 1.53 дБ за счет снижения средней энергии передаваемых символов (распределение Максвелла—Больцмана); практичная реализация — шейпинг многоуровневыми кодами.

13 Сетевое кодирование

13.1 Постановка задачи

13.1.1 Мотивирующие примеры

Обмен через спутник. Двум пользователям нужно обменяться сообщениями a и b; спутник принимает сообщения от обоих и широковещательно ретранслирует их. При пересылке без преобразований требуется 4 цикла (a вниз, b вниз, a вверх, b вверх). Если спутник может арифметически преобразовывать проходящие пакеты, он ретранслирует сумму $a + b$ (в поле характеристики 2 — побитовый XOR): каждый пользователь восстанавливает чужое сообщение вычитанием своего ($b = (a + b) - a$). Обмен занимает 3 цикла.

Мультикаст. Источник s должен доставить два пакета a, b двум приемникам t_1, t_2; в сети есть «узкое место» — канал, через который проходят данные к обоим приемникам. При пересылке и репликации пакетов этот канал задействуется дважды (сначала a, потом b); при сетевом кодировании достаточно передать по нему один пакет $a + b$: каждый приемник, получивший $(a, a + b)$ или $(b, a + b)$, восстанавливает оба пакета. Канал задействуется один раз.

13.1.2 Преимущества сетевого кодирования

- повышение пропускной способности сетей;
- снижение вычислительной сложности: построение оптимального дерева мультикаста (задача Штейнера) — NP-сложная задача, тогда как оптимальный сетевой код строится с полиномиальной сложностью;
- снижение затрат на пересылку подтверждений и повторную отправку потерянных пакетов.

13.2 Теоретические пределы

13.2.1 Потоки

Пусть сеть — ориентированный граф $G = (V, E)$ с m дугами и n узлами. **Матрица инцидентности** M: $M_{i,j} = 1$, если дуга j выходит из узла i ($i = \mathrm{tail}(j)$), $M_{i,j} = -1$, если дуга j входит в узел i ($i = \mathrm{head}(j)$), и 0 иначе. Обозначим $x_j \geq 0$ — объем данных, передаваемых по j-й дуге; d_i — объем данных, порождаемых (положительный) или потребляемых (отрицательный) узлом i. **Закон сохранения потока** (закон Кирхгофа): в каждом узле разность исходящего и входящего потоков равна потреблению:

$$\sum_{j \in A:\ \mathrm{tail}(j) = i} x_j - \sum_{j \in A:\ \mathrm{head}(j) = i} x_j = d_i, \quad 1 \leq i \leq n, \quad \text{или} \quad Mx = d.$$

Условие разрешимости: $\sum_{i=1}^{n} d_i = 0$ (сколько порождено, столько и потреблено). Множество допустимых потоков — аффинное пространство $\mathcal{X} = X^{(0)} + \sum_{j \geq 1} \alpha_j X^{(j)}$ (частное решение плюс линейная оболочка решений однородной системы).

13.2.2 Поток с минимальной стоимостью

Пусть c_j — стоимость передачи единицы потока по дуге j. Задача:

$$\min c^T x \quad \text{при} \quad Mx = d, \quad x \geq 0.$$

Пример: $c_j = 1$, $d_s = 1$, $d_t = -1$, остальные $d_i = 0$ — это поиск кратчайшего пути из s в t.

13.2.3 Разрезы и теорема Форда—Фалкерсона

***s-t* разрез** — подмножество узлов $S \subset V$ такое, что $s \in S$, $t^{\neg} \in S$. Дуга j **пересекает** разрез S, если $\mathrm{tail}(j) \in S, \mathrm{head}(j)^{\neg} \in S$; $E(S)$ — множество дуг, пересекающих разрез. Пусть z_j — пропускная способность дуги j (поток $x_j \leq z_j$). **Величина разреза**:

$$v(S) = \sum_{j \in E(S)} z_j.$$

Теорема 51 (о максимальном потоке и минимальном разрезе). Величина минимального s-t разреза равна величине максимального потока от s к t:

$$\max\{r : Mx = rd, 0 \le x_j \le z_j\} = \min\{v(S) : S \text{ — } s\text{ -t разрез}\},$$

где d содержит 1 на позиции s, -1 на позиции t и нули на прочих позициях.

Максимальный поток строится алгоритмом Форда—Фалкерсона.

13.2.4 Мультикаст как задача о потоках

Для мультикаста (один источник — несколько приемников) возможны три подхода:

1. сведение к передаче «один к одному»;
2. репликация пакетов на узлах — построение **деревьев мультикаста**;
3. преобразование (кодирование) пакетов на узлах — сетевое кодирование.

***s*-*T* дерево мультикаста** — дерево с корнем s, из которого существует ориентированный путь в каждый узел множества приемников T. Дерево задается вектором инцидентности x^k: $x^k_j = 1$, когда дуга j входит в k-е дерево. Пусть r_k — скорость передачи по k-му дереву. Максимальная скорость мультикаста с репликацией — решение задачи линейного программирования:

$$\max \sum_k r_k \quad \text{при} \quad \sum_k r_k x^k \le z, \quad r_k \ge 0.$$

Пример: три дерева передают со скоростью 0.5 каждое — максимальная скорость мультикаста к двум приемникам равна 1.5. При **сетевом кодировании** каждому терминалу $t \in T$ назначается свой поток x_t с вектором источника d_{st}, и достижимая скорость

$$\max r \quad \text{при} \quad Mx_t = rd_{st}, \quad x_t \ge 0, \quad x_t \le z, \quad t \in T$$

определяется теоремой о максимальном потоке и минимальном разрезе: скорость равна величине минимального разреза, отделяющего источник от приемников. В приведенном примере: сохранение и пересылка (store and forward) дает скорость 1, деревья мультикаста — 1.5, сетевое кодирование — 2 (величина минимального разреза).

13.3 Линейные сетевые коды

13.3.1 Кодирование

Данные представляются многочленами над конечным полем: пакет — последовательность M слов, то есть вектор $a \in \mathrm{GF}(q)^M$, или многочлен $a(x) \in \mathrm{GF}(q)[x]$. Многочленное представление позволяет складывать потоки, идущие по путям различной длины: сложение многочленов не зависит от временных сдвигов. Поколение из N пакетов — матрица $A^T = (a_1, \ldots, a_N)$ (строки — пакеты). **Закодированный пакет**:

$$b = A^T c = \sum_{i=1}^{N} c_i a_i, \quad c_i \in \mathrm{GF}(q),$$

где c — вектор коэффициентов, пересылаемый вместе с пакетом b (в заголовке). Промежуточные узлы перекодируют принятые пакеты: получив k пакетов $(x_1, \ldots, x_k)$, транзитный узел вычисляет

$$x' = \sum_{i=1}^{k} c'_i x_i, \quad c'_i \in \mathrm{GF}(q) \text{ случайны},$$

и применяет то же преобразование к вектору коэффициентов: коэффициенты пакета складываются по тем же правилам, что и сами пакеты (линейность).

13.3.2 Декодирование

Пусть приемник получил k пакетов, собранных в матрицу

$$X = \begin{bmatrix} x_1^T \\ \vdots \\ x_k^T \end{bmatrix} = \begin{bmatrix} c_1^T & b_1^T \\ \vdots & \vdots \\ c_k^T & b_k^T \end{bmatrix} = (C, B),$$

где C — матрица коэффициентов, B — матрица данных. Если C обратима над $\mathrm{GF}(q)$, приемник вычисляет $XC^{-1} = (I, A)$ и восстанавливает всё поколение данных A.

При достаточно большом q коэффициенты c_i, c_i' можно выбирать **случайно** — псевдослучайное линейное сетевое кодирование: вероятность необратимости C мала (аналогично вероятности обратимости случайной матрицы из главы о стирающем канале), и протокол не требует координации узлов. Это простой способ реализации сетевого кодирования, применяемый на практике (например, в протоколах передачи данных в одноранговых сетях).

13.4 Выводы

- Пропускная способность компьютерных сетей может быть существенно повышена за счет сетевого кодирования: скорость мультикаста достигает величины минимального разреза, тогда как сохранение-пересылка и деревья мультикаста дают строго меньшие скорости.
- Сетевое кодирование заменяет NP-сложную задачу построения оптимального дерева мультикаста полиномиально решаемой задачей.
- Псевдослучайное линейное сетевое кодирование (случайные коэффициенты в большом поле, передаваемые в заголовках пакетов) — простая и практичная реализация.

14 Регенерирующие коды

14.1 Мотивация

Производительность географически распределенных систем хранения данных (СХД) определяется объемом сетевого трафика, который они генерируют. При восстановлении данных необязательно передавать все содержимое задействованных узлов; кроме того, стертые символы можно заменять другими, если при этом сохраняется корректирующая способность кода. **Регенерирующие коды** — коды, оптимизирующие объем данных, передаваемых при восстановлении отказавшего узла.

14.2 Сетевое кодирование и СХД

14.2.1 Граф информационных потоков

Модель восстановления описывается ориентированным ациклическим **графом информационных потоков** с тремя типами узлов:

1. узел-источник S;
2. узлы-хранилища (для каждого устройства — «писатель» $x_i^{[\mathrm{in}]}$ и «читатель» $x_i^{[\mathrm{out}]}$);
3. сборщики данных «DC».

Пропускные способности дуг: $S \to x_i^{[\mathrm{in}]}$ — бесконечная (источник выгружает данные без ограничений); $x_i^{[\mathrm{in}]} \to x_i^{[\mathrm{out}]}$ — α (объем данных, хранимых на устройстве); $x_i^{[\mathrm{out}]} \to \mathrm{DC}$ — бесконечная; для вновь подключаемого узла j: $x_i^{[\mathrm{out}]} \to x_j^{[\mathrm{in}]}$ — β (объем данных, передаваемый с одного выжившего узла). Источник отключается после первоначальной загрузки. При отказе одного из узлов подключаемый узел получает β бит с любых $D \le n-1$ выживших узлов (всего $\gamma = D\beta$); любые k узлов-хранилищ должны позволять восстановить все данные. Каждая последовательность отказов и подключений задает свой граф; их множество — $G(n, k, D, \alpha, \gamma)$.

14.2.2 Граница Димакиса и ее точки

Теорема 52 (Димакис и др.). Для любого $\alpha \geq \alpha^* *$ достижимы точки $(n, k, D, \alpha, \gamma)$ — причем с помощью линейных сетевых кодов; точки с $\alpha < \alpha^* *$ недостижимы, где

$$\alpha^* * (n, k, D, \gamma) = \begin{cases} M/k & f(0) \leq \gamma < \infty \\ \frac{M - g(i)\gamma}{k - i} & f(i) \leq \gamma < f(i-1) \end{cases},$$

$$f(i) = \frac{2MD}{(2k - i - 1)i + 2k(D - k + 1)}, \quad g(i) = \frac{(2D - 2k + i + 1)i}{2D},$$

где M — объем хранимых полезных данных, $D \leq n - 1, 0 < i < k$. Минимальный объем данных, передаваемых при восстановлении: $\gamma_{\min} = f(k-1)$.

Граница описывает компромисс между объемом хранения α и трафиком восстановления γ (кривая, убывающая от M/k при большом γ к большим α при минимальном γ).

14.2.3 Особые точки: MSR и MBR

- **Коды с минимальным хранением** (MSR, minimum-storage regenerating): $\alpha = M/k$, $\gamma = \frac{MD}{k(D-k+1)}$ — храним как можно меньше, платим трафиком.
- **Коды с минимальным трафиком** (MBR, minimum-bandwidth regenerating): $\alpha = \gamma = \frac{2MD}{2kD - k^2 + k}$ — минимальный трафик восстановления при увеличенном хранении.

Здесь α — объем данных на одном устройстве, $\gamma = D\beta$ — общий объем передачи при восстановлении. Для (n, k) МДР-кода ($d = n - k + 1$) с $D = n - 1$ оптимальный трафик

$$\gamma_{[\text{MDS}]} = \frac{M}{k} \cdot \frac{n-1}{n-k}.$$

Помимо трафика γ, важен объем данных $\varphi_{[\text{MDS}]}$, которые нужно **считать** с носителей при восстановлении: МДР-код с оптимальным считыванием — код, для восстановления одного символа которого достаточно прочитать ровно $\varphi_{[\text{MDS}]} = \gamma_{[\text{MDS}]}$ бит данных.

14.2.4 Размер поля и практические приемы

Теорема 53. Для $(k + r, k, r + 1)$ МДР-кода: с оптимальным считыванием $k \leq r \log_r l$; с оптимальным объемом передачи $k \leq \log_r l$, где l — емкость узла в символах поля $\mathrm{GF}(q)$. Существуют коды, достигающие этих границ.

Большие поля ограничивают минимальный объем данных, кодируемых таким способом. Практические приемы работы с большими полями:

- **полосы** (stripes): группировка S кодовых слов; на каждом устройстве хранятся блоки из S символов различных кодовых слов над $\mathrm{GF}(2^m)$, над которыми выполняются одни и те же операции;
- **подполосы** (substripes): каждый бит элемента $\mathrm{GF}(2^m)$ сопоставляется блоку из S битов, и все операции выполняются над такими блоками целиком.

14.3 Комбинирующая конструкция (piggybacking)

14.3.1 Общая схема

Пусть дан (n, k) код с функциями посимвольного кодирования $f_{i(u)}$, $1 \leq i \leq n$ (u — информационный вектор, f_i — символ i-го узла). Берутся α кодовых слов базового кода для независимых сообщений $a, b, ..., z$ и вводятся **функции комбинирования** $g_{i,j}(u)$, $2 \leq i \leq \alpha$, $1 \leq j \leq n$: на узле j хранится

$$f_1 \text{ слоя 1: } f_1(a); \quad \text{слои } 2, ..., \alpha : f_{i(b)} + g_{2,j}(a), ..., f_{i(z)} + g_{\alpha,j}(a, ..., y).$$

То есть к «побочным» слоям подмешиваются комбинации символов других сообщений. Свойства: комбинированный код сохраняет минимальное расстояние исходного; обобщение — в пределах каждого узла выполняется обратимое преобразование символов базового кода.

Пример восстановления. При восстановлении узла 1 вместо α подсимволов скачивается меньше (например, на 25%): из смешанных символов вычитаются известные комбинации, после чего восстанавливаются исходные символы по обычным формулам кода.

14.3.2 Метод комбинирования 1

Пусть $\alpha = 2$, базовый код — систематический линейный МДР-код: $f_{i(u)} = u_i$ для $1 \le i \le k$ (информационные узлы), $f_{k+i}(u) = p_i^T u$ для проверочных узлов (p_i — векторы коэффициентов проверок). Информационные символы разбиваются на r почти равных подмножеств $S_1, ..., S_r$. Для проверочных узлов вводятся векторы: q_i — вектор p_i, «обнуленный» вне блока S_i, и $v_r = p_r - q_r$; во втором слое хранятся смешанные символы $p_i^T b + q_i^T a$ (узлы $k+2, ..., k+r$) и $v_r^T a - p_r^T b$ (узел $k+r$, после обратимого преобразования в узле).

Восстановление стертого информационного символа $l \neg \in S_r$: скачать $k-1$ информационных символов и $p_1^T b$, восстановить вектор b; скачать смешанный символ $p_2^T b + q_2^T a$, вычесть $p_2^T b$; скачать a_i, $i \in S_1 \setminus \{l\}$, и вычислить a_l из $q_2^T a$ (у q_2 единственный ненулевой блок S_1). Объем скачанных данных $k + |S_1|$ — существенно меньше $2k$. Для $l \in S_r$ аналогично используется v_r и остальные смешанные символы; объем $k + |S_r| + r - 2$.

Обобщение для проверочных символов. Берется m кодовых слов (узел хранит $2m$ подсимволов); суммы проверочных символов четных подполос подмешиваются к проверочному подсимволу узла $k+1$ следующей подполосы. Восстановление информационных символов не меняется; восстановление символа узла $k+1$ — по определению из всех информационных; восстановление символа узла $k+i$ ($i \ge 2$) — скачиванием информационных символов нечетных подполос, вычитанием их вкладов из подсимволов узла $k+1$ и последним вычитанием известных сумм проверочных.

14.3.3 Метод комбинирования 2

Берется $2r-3$ кодовых слов базового $(k+r, k, r+1)$ кода с информационными векторами $a_1, ..., a_{2r-3}$. Систематические узлы делятся на $r-1$ почти равных подмножеств; вводятся полиномиальные комбинации $v_i = a_{r-1} + i a_{r-2} + i^2 a_{r-3} + ... + i^{r-2} a_1$ и блочные векторы q_{ij} (вектор p_i, обнуленный вне j-го блока). Для последних $r-1$ проверочных узлов выполняются аналогичные обратимые преобразования. Пример: восстановление одного из 10 узлов кода $(14, 10)$ требует 20 скачанных подсимволов вместо 30 информационных.

14.4 Конструкция на основе произведения матриц

14.4.1 Постановка

Оптимальные точки: $\alpha_{[\text{MSR}]} = M/k$, $\gamma_{[\text{MSR}]} = \frac{MD}{k(D-k+1)}$; $\alpha_{[\text{MBR}]} = \gamma_{[\text{MBR}]} = \frac{2MD}{2kD-k^2+k}$. Положим $\beta = 1$ (с каждого выжившего узла скачивается один символ); тогда $\alpha_{[\text{MSR}]} = D - k + 1$, $\alpha_{[\text{MBR}]} = D$.

Коды на основе произведения матриц: кодовое слово — матрица C размера $n \times \alpha$, i-я строка — α символов i-го узла:

$$C = \Psi X,$$

где Ψ — матрица кодирования размера $n \times D$ (ψ_i — ее i-я строка, вектор кодирования узла i), X — матрица размера $D \times \alpha$, содержащая M символов полезных данных (возможно, с повторениями). Восстановление данных по k узлам — выделение подматрицы $\Psi_{[\text{DC}]}$ (строки задействованных узлов) и обращение; восстановление узла f — запрос с D узлов значений $\psi_{h_j} X \mu_f$, где μ_f — подвектор строки ψ_f^T. Коды допускают систематический вид.

14.4.2 Код с минимальным трафиком (MBR)

При $\beta = 1$: $\alpha_{[\text{MBR}]} = D$, и объем данных $M = k\alpha - \binom{k}{2} = \binom{k+1}{2} + k(D-k)$ (первое слагаемое — из $k\alpha$ символов k узлов вычитаются повторы). Пусть S — симметричная $k \times k$ матрица, верхнетреугольная часть которой (включая диагональ) хранит $\binom{k+1}{2}$ символов данных (элементы $S_{ij} = S_{ji}$ — повторы); оставшиеся $k(D-k)$ символов — в матрице T размера $k \times (D-k)$. Данные:

$$X = \begin{bmatrix} S & T \\ T^T & \end{bmatrix} \quad \text{нижний блок повторяет } T.$$

Матрица кодирования $\Psi = (\Phi, \Delta)$, где Φ — матрица размера $n \times k$ с линейно независимыми любыми k строками, Δ — размера $n \times (D-k)$, причем любые D строк Ψ линейно независимы (пример: Ψ — матрица Вандермонда). Кодирование: $C = \Psi X$.

Теорема 54. Восстановление одного узла MBR-кода выполняется скачиванием одного символа с любых D из $n-1$ оставшихся узлов.

Доказательство. Пусть отказал узел f с вектором $\psi_f X$. С узлов $h_1, \ldots, h_D$ скачиваются скаляры $\left(\psi_{h_j} X\right)\psi_f^T$. Столбцы $\psi_{h_1}^T, \ldots, \psi_{h_D}^T$ образуют обратимую матрицу (любые D строк Ψ линейно независимы), поэтому восстанавливающий узел находит $X\psi_f^T = \psi_f X$ — ровно то, что хранилось на узле f; с каждого узла передан один символ, итого $\gamma = D$. □

Теорема 55. Все M символов данных MBR-кода восстанавливаются по любым k узлам.

Доказательство. Пусть $\Psi_{[\text{DC}]} = \left(\Phi_{[\text{DC}]}, \Delta_{[\text{DC}]}\right)$ — подматрица строк задействованных узлов. С узлов читается

$$\Psi_{[\text{DC}]} X = \left(\Phi_{[\text{DC}]} S + \Delta_{[\text{DC}]} T^T, \quad \Phi_{[\text{DC}]} T\right).$$

Матрица $\Phi_{[\text{DC}]}$ обратима (любые k строк Φ линейно независимы), поэтому из второго блока находится T, вычисляется $\Delta_{[\text{DC}]} T^T$ и восстанавливается S. □

Систематический MBR-код: $\Psi = \left(I_k, \tilde{\Phi}, \tilde{\Delta}\right)$ (первые k узлов систематические), где матрица $\left(\tilde{\Phi}, \tilde{\Delta}\right)$ — матрица Коши с элементами $1/\left(x_i - y_j\right)$: все ее квадратные подматрицы обратимы, а определитель $l \times l$ подматрицы вычисляется по формуле

$$\det Q = \frac{\prod_{i=2}^{l} \prod_{j=1}^{i-1} (x_i - x_j)(y_j - y_i)}{\prod_{i=1}^{l} \prod_{j=1}^{l} (x_i - y_j)} \neq 0$$

при различных x_i, y_j.

14.4.3 Код с минимальным хранением (MSR)

Пусть $D = 2k-2$, $\alpha = D - k + 1 = k - 1$, $M = k\alpha = \alpha(\alpha+1)$. Отметим: не существуют линейных MSR-кодов с $\beta = 1$ при $D < 2k-3$. Данные: $X = (S_1, S_2)^T$, где S_1, S_2 — симметричные $\alpha \times \alpha$ матрицы, верхнетреугольные части которых хранят по $\binom{\alpha+1}{2}$ символов. Матрица кодирования:

$$\Psi = (\Phi, \Lambda\Phi),$$

где Φ — матрица размера $n \times \alpha$ с линейно независимыми любыми α строками, Λ — диагональная матрица $n \times n$ с **различными** диагональными элементами; любые D строк Ψ линейно независимы. Пример: строки $\psi_i = \left(1, x_i, \ldots, x_i^{D-1}\right)$ матрицы Вандермонда с $x_i = g^{i-1}$, g — образующий элемент мультипликативной группы порядка не менее $n(D-k+1)$, и $\Lambda = \operatorname{diag}(x_1^\alpha, \ldots, x_n^\alpha)$.

Теорема 56. Точное восстановление отказавшего узла MSR-кода выполняется скачиванием одного символа с любых $D = 2k - 2$ из оставшихся $n - 1$ узлов.

Доказательство. Отказавший узел f хранил

$$\psi_f X = (\varphi_f, \lambda_f \varphi_f) \begin{bmatrix} S_1 \\ S_2 \end{bmatrix} = \varphi_f S_1 + \lambda_f \varphi_f S_2,$$

где φ_f — строка Φ, λ_f — диагональный элемент Λ. С узлов $h_1, \ldots, h_D$ запрашиваются скаляры $\psi_{h_j} X \varphi_f^T$. Из обратимости матрицы из строк ψ_{h_j} восстанавливающий узел находит

$$X \varphi_f^T = \begin{bmatrix} S_1 \varphi_f^T \\ S_2 \varphi_f^T \end{bmatrix}.$$

Тогда $\varphi_f S_1^T + \lambda_f \varphi_f S_2^T = \varphi_f S_1 + \lambda_f \varphi_f S_2 = \psi_f X$ — узел восстановлен точно; передано по одному символу с каждого из D узлов. □

Теорема 57. Все M символов MSR-кода восстанавливаются по любым $k = \alpha + 1$ узлам.

Доказательство. С k узлов читается $\Psi_{[\mathrm{DC}]} X = \Phi_{[\mathrm{DC}]} S_1 + \Lambda_{[\mathrm{DC}]} \Phi_{[\mathrm{DC}]} S_2$ ($\Lambda_{[\mathrm{DC}]}$ — соответствующая подматрица Λ). Умножим справа на $\Phi_{[\mathrm{DC}]}^T$:

$$Z = \left(\Phi_{[\mathrm{DC}]} S_1 + \Lambda_{[\mathrm{DC}]} \Phi_{[\mathrm{DC}]} S_2\right) \Phi_{[\mathrm{DC}]}^T = \underbrace{\Phi_{[\mathrm{DC}]} S_1 \Phi_{[\mathrm{DC}]}^T}_{P} + \underbrace{\Lambda_{[\mathrm{DC}]} \Phi_{[\mathrm{DC}]} S_2 \Phi_{[\mathrm{DC}]}^T}_{Q}.$$

Матрицы P и Q симметричны, а $Z_{ij} = P_{ij} + \lambda_i Q_{ij}$ и $Z_{ji} = P_{ij} + \lambda_j Q_{ij}$ — система двух уравнений с различными $\lambda_i \neq \lambda_j$, из которой находятся все недиагональные P_{ij}, Q_{ij}. Строка i матрицы P без диагонали равна $\tilde{\varphi}_i S_1 \tilde{\Phi}^T$, где $\tilde{\Phi}$ — матрица из прочих строк $\Phi_{[\mathrm{DC}]}$; она обратима (размер $\alpha \times \alpha$), поэтому находятся $\tilde{\varphi}_i S_1$ для всех i, а с ними — симметричная S_1. Аналогично восстанавливается S_2. □

14.4.4 MSR-коды при $D \geq 2k - 2$

Базовая конструкция покрывает $D = 2k - 2$. Расширение: если существует (n', k') MSR-код с $D' = D + 1$ и систематическим кодированием, то укорочение на первый символ (верхнюю строку кодовых матриц) дает $(n = n' - 1, k = k' - 1)$ MSR-код с восстановлением по D узлам; соотношение $D' = ak' + b$ дает $D = ak + b + (a - 1)$. Комбинируя такие укорочения с готовыми оптимальными кодами, строят MSR-коды для всего диапазона $2k - 2 \leq D \leq n - 1$.

14.5 Восстановление в кодах Рида—Соломона

14.5.1 МДР-коды и дуальные коды

Код с максимальным достижимым расстоянием — $(n, k, n - k + 1)$ код. Код, дуальный к коду с порождающей матрицей G, имеет проверочную матрицу G; дуальным к $(n, k, n - k + 1)$ МДР-коду является $(n, n - k, k + 1)$ МДР-код. Для МДР-кода всегда можно записать $G = (I \mid A)P$, $H = (A^T \mid -I)P$ с общей перестановочной матрицей P: любые $n - k$ столбцов H линейно независимы, поэтому любые k столбцов G линейно независимы, и G служит проверочной матрицей кода с расстоянием $k + 1$.

Обобщенный код Рида—Соломона: $C = \{(\lambda_1 f(x_1), \ldots, \lambda_n f(x_n)) \mid \deg f < k\}$, где λ_i — ненулевые весовые коэффициенты, x_i — различные локаторы. **Дуальным** к $(n, k, n - k + 1)$ коду РС является обобщенный $(n, n - k, k + 1)$ код РС с весами $\lambda_i' = \prod_{j \neq i} (x_i - x_j)^{-1}$ (точнее, с весами, обратными произведениям разностей локаторов).

14.5.2 Подполя конечных полей

Лемма 7. Для натуральных $p \geq 2$, $r, s \geq 1$: $(p^s - 1)$ делит $(p^r - 1)$ тогда и только тогда, когда s делит r.

Доказательство. Пусть $r = Qs + R$, $0 \leq R < s$. Тогда $\frac{p^r-1}{p^s-1} = p^R \frac{p^{Qs}-1}{p^s-1} + \frac{p^R-1}{p^s-1}$. Первое слагаемое целое всегда ($p^{Qs} - 1$ делится на $p^s - 1$), поэтому делимость равносильна делимости $p^R - 1$ на $p^s - 1$, то есть $R = 0$. □

Теорема 58. $\mathrm{GF}(p^s)$ является подполем $\mathrm{GF}(p^r)$ тогда и только тогда, когда $s \mid r$.

Доказательство. x лежит в $\mathrm{GF}(q)$ тогда и только тогда, когда $x^q - x = 0$. Ненулевой элемент $\mathrm{GF}(p^s)$ удовлетворяет $x^{p^s-1} = 1$. Если $s \mid r$, то $Q = \frac{p^r-1}{p^s-1}$ целое и $x^{p^r-1} = \left(x^{p^s-1}\right)^Q = 1$, значит $x \in \mathrm{GF}(p^r)$. Обратно, если $\mathrm{GF}(p^s) \subset \mathrm{GF}(p^r)$, мультипликативная группа первого содержится во второй, и по лемме $s \mid r$. □

14.5.3 След элемента

Определение 31. Пусть $F = \mathrm{GF}(q^t)$, $B = \mathrm{GF}(q)$. **Следом** элемента $\beta \in F$ называется

$$\mathrm{Tr}_{F/B}(\beta) = \sum_{i=0}^{t-1} \beta^{q^i}.$$

Свойства: $\mathrm{Tr}_{F/B}(a + b) = \mathrm{Tr}_{F/B}(a) + \mathrm{Tr}_{F/B}(b)$ (аддитивность); $\mathrm{Tr}_{F/B}(xy) = x\,\mathrm{Tr}_{F/B}(y)$ для $x \in B$, $y \in F$ (B-линейность); $\mathrm{Tr}_{F/B}(\beta) \in B$ (след лежит в меньшем поле).

14.5.4 Линеаризованные многочлены

Определение 32. **Линеаризованным** называется многочлен $L(x) = \sum_i l_i x^{p^i}$, $l_i \in \mathrm{GF}(p^m)$. Он линеен над $\mathrm{GF}(p)$: $L(a + b) = L(a) + L(b)$ и $L(ax) = aL(x)$ для $a \in \mathrm{GF}(p)$.

Для линейного подпространства $W \subset \mathrm{GF}(p^m)$ многочлен $L_{W(x)} = \prod_{w \in W}(x - w)$ — линеаризованный (его корни — в точности элементы W).

Теорема 59. Если $\dim W = k$, то образ L_W: $\left\{L_{W(x)} \mid x \in \mathrm{GF}(p^m)\right\}$ имеет размерность $m - k$.

Доказательство. Пусть $w_0, \ldots, w_{k-1}$ — базис W, дополненный до базиса всего $\mathrm{GF}(p^m)$. Тогда $L_{W\left(\sum_{i=0}^{m-1} x_i w_i\right)} = \sum_{i=k}^{m-1} x_i L_{W(w_i)}$, так как $L_{W(w_i)} = 0$ при $i < k$. Образ — линейная оболочка $m - k$ векторов $L_{W(w_i)}$; если бы их было меньше $m - k$, нашлась бы нетривиальная комбинация с $L_{W(\sum x_i w_i)} = 0$, то есть корень вне W — противоречие. Значит размерность ровно $m - k$. □

14.5.5 Линейные схемы восстановления

Пусть дан линейный (n, k) код C над полем F, кодовые слова — значения функций f в точках-локаторах $A = (a_1, \ldots, a_n)$. **Линейная схема восстановления** одного стирания над подполем $B = \mathrm{GF}(q) \subset F = \mathrm{GF}(q^m)$ задается:

- набором **запросов** $Q_{a(\alpha)} \subset F$ — для каждой нестертой позиции α;
- линейным алгоритмом, вычисляющим $f(a) = \sum_{i=1}^{m} \lambda_i \nu_i$ ($\lambda_i \in B$) по базису $\nu_1, \ldots, \nu_m$ поля F над B, где коэффициенты λ_i — B-линейные комбинации значений следа $\mathrm{Tr}_{F/B}(\gamma f(\alpha))$ по запросам $\gamma \in Q_{a(\alpha)}$.

График восстановления — общее число подсимволов поля B, передаваемых с узлов:

$$b = \max_{a \in A} \sum_{\alpha \in A \setminus \{a\}} |Q_{a(\alpha)}|;$$

локальность схемы: $D = \max_{a \in A} \sum_{\alpha \in A \setminus \{a\}} 1_{Q_{a(\alpha)} \neq \varnothing}$ — с какого максимального числа узлов читаются данные.

Основная идея: представить стертый символ линейной комбинацией нестертых

$$\zeta f(a) = \sum_{\alpha \in A \setminus \{a\}} \mu_{a,\zeta}(\alpha) f(\alpha)$$

для нескольких $\zeta \in F$; применяя след (он линеен), получаем $\mathrm{Tr}_{F/B}(\zeta f(a)) = \sum_\alpha \mathrm{Tr}_{F/B}(\mu f(\alpha))$ — вычислимо по запросам; набрав следы для достаточно многих ζ, восстанавливаем $f(a)$ по дуальному базису.

Базисы поля. $\mathrm{GF}(p^m)$ — m-мерное пространство над $\mathrm{GF}(p)$. Примеры базисов: стандартный $(1, \alpha, ..., \alpha^{m-1}$ для примитивного $\alpha)$ и **нормальный** $(\gamma, \gamma^p, ..., \gamma^{p^{m-1}})$. Если $\zeta_1, ..., \zeta_t$ — базис F над B, то **дуальным** называется базис $\nu_1, ..., \nu_t$ с условием ортогональности следа: $\mathrm{Tr}_{F/B}(\zeta_i \nu_l) = 0$ при $i \neq l$ и $= 1$ при $i = l$ — по нему коэффициенты $\lambda_i = \mathrm{Tr}_{F/B}(\zeta_i f(a))$ сразу дают $f(a) = \sum_i \lambda_i \nu_i$.

14.5.6 Алгоритм линейного восстановления

Алгоритм 11 (LinRepair).

1. Выбрать набор $Z = \{\zeta_1, ..., \zeta_m\}$ линейно независимых над B элементов F.
2. Найти коэффициенты $\mu_{a,\zeta}(\alpha)$, $\alpha \neq a$, $\zeta \in Z$, в представлениях $\zeta f(a) = \sum_{\alpha \neq a} \mu_{a,\zeta}(\alpha) f(\alpha)$.
3. Для каждой позиции α взять базис $\tilde{Q}_{a(\alpha)}$ линейной оболочки $\{\mu_{a,\zeta}(\alpha) \mid \zeta \in Z\}$ над B и запросить значения $\mathrm{Tr}_{F/B}(\gamma f(\alpha))$, $\gamma \in \tilde{Q}_{a(\alpha)}$.
4. Пользуясь B-линейностью следа, вычислить $\mathrm{Tr}_{F/B}\big(\mu_{a,\zeta}(\alpha) f(\alpha)\big)$, затем $\mathrm{Tr}_{F/B}(\zeta f(a)) = \sum_{\alpha \neq a} \mathrm{Tr}_{F/B}\big(\mu_{a,\zeta}(\alpha) f(\alpha)\big)$ для всех $\zeta \in Z$.
5. Возвратить $f(a) = \sum_{i=1}^m \mathrm{Tr}_{F/B}(\zeta_i f(a)) \nu_i$, где ν_i — базис, дуальный к Z.

Корректность опирается на то, что равенство следов для всех кодовых слов и всех домножений на $\gamma \in F$ влечет само равенство $\zeta_i f(a) = \sum_\alpha \mu_{a,\zeta_i}(\alpha) f(\alpha)$; при этом трафик равен $b = \max_a \sum_\alpha \dim_B \{\mu_{a,\zeta}(\alpha) \mid \zeta \in Z\} \leq \max_a \sum_\alpha |Q_{a(\alpha)}|$.

14.5.7 Схема восстановления для кодов РС через линеаризованные многочлены

Пусть дан $(n, k, n-k+1)$ код РС над $F = \mathrm{GF}(q^m)$, $n - k \geq q^l$, $1 \leq l \leq m$; фиксируем l-мерное подпространство $W \subset \mathrm{GF}(q^m)$. Для восстановления стертого символа с локатором a используются многочлены

$$g_{i(x)} = \frac{L_{W(\beta_i(x-a))}}{x - a}, \quad 0 \leq i < l, \quad \beta_i \text{— базис } \mathrm{GF}(q^m) \text{ над } \mathrm{GF}(q),$$

где $L_{W(x)} = \sum_i L_i x^{q^i} = x L_0 + x^2 M(x)$, $L_0 = \prod_{w \in W \setminus \{0\}} w$ (произведение ненулевых элементов W). Поскольку $x - a$ — сомножитель $L_{W(\beta_{i(x-a)})}$, степени низки: $\deg g_i = q^l - 1 < n - k$, то есть g_i — кодовые слова дуального кода (многочлены степени меньше $n - k$ задают проверочные условия). Прямая проверка:

$$g_{i(a)} = L_0 \beta_i \neq 0, \quad \dim\{g_{i(a)}\} = l \text{ максимален}; \quad \dim\{g_{i(\alpha)} \mid \alpha \neq a\} \leq \dim\{L_{W(y)} \mid y \in \mathrm{GF}(q^m)\} = m - l$$

— в стертой позиции значения многочленов максимально независимы, в остальных — лежат в подпространстве коразмерности l.

Теорема 60. Восстановление одного стирания в $(n, k, n-k+1)$ коде РС с локаторами A над $F = \mathrm{GF}(q^m)$, $q^m \geq n \geq n - k \geq q^l$, требует передачи не более $(n-1)(m-l)$ элементов $B = \mathrm{GF}(q)$.

Доказательство. Схема LinRepair с Z = базисом и коэффициентами из дуального кода $\mathrm{GRS}(n, n-k, \lambda)$, $\lambda_\alpha = \prod_{b \neq \alpha}(b-\alpha)$:

1. вычислить $\mu_{\alpha,i} = \lambda_\alpha g_{i(\alpha)}$, $0 \le i < m$; в каждой позиции α выбрать максимальное линейно независимое подмножество $\tilde{Q}_{a(\alpha)}$ — не более $m-l$ элементов;
2. получить с $n-1$ узлов следы $\mathrm{Tr}_{F/B}(\zeta f(\alpha))$, $\zeta \in \tilde{Q}_{a(\alpha)}$;
3. вычислить $\mathrm{Tr}_{F/B}\big(\mu_{a,i}(\alpha) f(\alpha)\big)$ как B-линейные комбинации, затем $\mathrm{Tr}_{F/B}(\zeta_i f(a))$ по аддитивности следа;
4. восстановить $f(a)$ по дуальному базису.

С каждого из $n-1$ узлов передается не более $m-l$ элементов B, итого $(n-1)(m-l)$. □

14.5.8 Нижняя граница трафика

Теорема 61. Всякая линейная схема восстановления кода РС $\mathrm{RS}(n,k)$ над $\mathrm{GF}(q^m)$, работающая в поле $\mathrm{GF}(q)$, требует передачи не менее $b = t\lfloor b_a \rfloor + (n-1-t)\lceil b_a \rceil$ символов $\mathrm{GF}(q)$, где

$$b_a = \log_q\left(\frac{(n-1)q^m}{(n-k-1)(q^m-1)+(n-1)}\right), \quad t = \begin{cases} n-1 & b_a \in \mathbb{Z} \\ \left(\frac{T-(n-1)q^{-\lceil b_a \rceil}}{q^{-\lfloor b_a \rfloor}-q^{-\lceil b_a \rceil}}\right) & b_a \neg \in \mathbb{Z}, \end{cases} \quad T = \frac{(r-1)(q^m-1)+(n-1)}{q^m} \text{ при } r = n-k.$$

При $n = q^m$, $n-k = q^l$ получаем $b_a = m-l$ и $b = (n-1)(m-l)$: построенная схема восстановления **оптимальна**.

14.6 Выводы

- Параметры СХД описывает компромисс «хранение-трафик» (граница Димакиса); ключевые точки — коды с минимальным хранением (MSR) и минимальным трафиком (MBR).
- Комбинирующая конструкция (piggybacking) снижает трафик восстановления систематических МДР-кодов ценой простых обратимых преобразований в узлах.
- Конструкция на основе произведения матриц дает оптимальные MBR- и MSR-коды: восстановление узла скачиванием одного символа с каждого из D выживших узлов, восстановление данных по любым k узлам.
- Линейные схемы восстановления на основе следа и линеаризованных многочленов восстанавливают стирание в коде Рида—Соломона с трафиком $(n-1)(m-l)$ элементов подпола, что совпадает с нижней границей — схема оптимальна.